

UNIVERSIDADE FEDERAL DE PELOTAS
Centro de Desenvolvimento Tecnológico
Programa de Pós-Graduação em Computação



Dissertação

**ParseH2IA: Desenvolvimento de um Parser para Análise de Dependências
Sintáticas do Português Brasileiro com Aprendizado Profundo**

Guilherme Dallmann Lima

Pelotas, 2026

Guilherme Dallmann Lima

**ParseH2IA: Desenvolvimento de um Parser para Análise de Dependências
Sintáticas do Português Brasileiro com Aprendizado Profundo**

Dissertação apresentada ao Programa de Pós-Graduação em Computação do Centro de Desenvolvimento Tecnológico da Universidade Federal de Pelotas, como requisito parcial à obtenção do título de Mestre em Ciência da Computação.

Orientador: Prof^a. Dr^a. Larissa Astrogildo de Freitas
Coorientador: Prof. Dr. Ulisses Brisolara Corrêa

Pelotas, 2026

Insira AQUI a ficha catalográfica
Quando finalizado o trabalho, deve ser
solicitada através do Sistema Cobalto
Biblioteca – Cadastro – Ficha catalográfica.

Guilherme Dallmann Lima

**ParseH2IA: Desenvolvimento de um Parser para Análise de Dependências
Sintáticas do Português Brasileiro com Aprendizado Profundo**

Dissertação aprovada, como requisito parcial, para obtenção do grau de Mestre em Ciência da Computação, Programa de Pós-Graduação em Computação, Centro de Desenvolvimento Tecnológico, Universidade Federal de Pelotas.

Data da Defesa: 30 de fevereiro de 2019

Banca Examinadora:

Prof. Dr. Marilton Sanchotene de Aguiar (orientador)

Doutor em Computação pela Universidade Federal do Rio Grande do Sul.

Prof. Dr. Paulo Roberto Ferreira Jr.

Doutor em Computação pela Universidade Federal do Rio Grande do Sul.

Prof. Dr. Ricardo Matsumura Araujo

Doutor em Computação pela Universidade Federal do Rio Grande do Sul.

Prof. Dr. Luciano da Silva Pinto

Doutor em Biotecnologia pela Universidade Federal de Pelotas.

Dedico...

AGRADECIMENTOS

Agradeço...

Só sei que nada sei.

— SÓCRATES

RESUMO

LIMA, Guilherme Dallmann. **ParseH2IA: Desenvolvimento de um Parser para Análise de Dependências Sintáticas do Português Brasileiro com Aprendizado Profundo**. Orientador: Larissa Astrogildo de Freitas. 2026. 165 f. Dissertação (Mestrado em Ciência da Computação) – Centro de Desenvolvimento Tecnológico, Universidade Federal de Pelotas, Pelotas, 2026.

A análise de dependências sintáticas é uma tarefa fundamental do Processamento de Linguagem Natural (PLN), responsável por identificar as relações gramaticais entre as palavras de uma sentença e produzir uma representação estrutural que serve de base para tarefas mais complexas, como extração de informação, análise de sentimento e tradução automática. Para o português brasileiro, o estado da arte é representado pelo PortParser, que combina o encoder BERTimbau Large com uma camada BiLSTM bidirecional e mecanismo biaffine, atingindo 96,08% de *Unlabeled Attachment Score* (UAS) e 94,61% de *Labeled Attachment Score* (LAS) no corpus Porttinari. Esta dissertação apresenta o **ParseH2IA**, um *parser* de dependências para o português brasileiro que suprime o BiLSTM presente no PortParser e conecta o encoder diretamente a cabeçotes de predição com ajuste fino completo de todos os parâmetros. O trabalho conduz uma comparação sistemática de quatro encoders pré-treinados, BERTimbau Large, BERTimbau Base, mBERT e JabuticaBERT, em cruzamento com duas arquiteturas de cabeçote (linear e biaffine) e dois regimes de treinamento (aprendizado multi-tarefa com UPOS e ablação sem UPOS), totalizando 16 configurações experimentais. O cabeçote linear projeta a saída do encoder por três camadas densas independentes para HEAD, DEPREL e UPOS; o cabeçote biaffine utiliza quatro MLPs especializadas e dois mecanismos de atenção biaffine para HEAD e DEPREL. Os hiperparâmetros de cada configuração foram otimizados independentemente por busca bayesiana com validação cruzada em cinco partições. O melhor resultado foi obtido pela configuração BERTimbau Large com cabeçote linear e aprendizado multi-tarefa, alcançando UAS de 94,99%, LAS de 93,84% e acurácia de UPOS de 99,28%, a menos de 1,1 pontos percentuais do PortParser em UAS. O achado mais relevante é a **superioridade consistente do cabeçote linear sobre o biaffine** em três dos quatro encoders avaliados, com margens de até 5,78 pontos percentuais, contrariando a expectativa da literatura para cenários de ajuste fino completo. Esse resultado é interpretado à luz das características do corpus de avaliação: com apenas 5.893 sentenças de treinamento no Porttinari, o cabeçote biaffine, com suas quatro MLPs e matrizes bilineares, apresenta maior risco de sobreajuste em comparação ao cabeçote linear, cuja menor capacidade paramétrica atua como regularização implícita e resulta

em generalização mais estável. Como contribuição metodológica, o trabalho propõe uma análise de instabilidade do treinamento por rastreamento de predições por época, quantificando eventos de *forgetting* (acerto $\rightarrow$ erro) e *recovery* (erro $\rightarrow$ acerto) por rótulo de dependência. Essa análise revela que o cabeçote biaffine é sistematicamente mais instável durante a convergência, e que rótulos de alta frequência com fronteiras de decisão difusas, *nmod* e *obl*, concentram a maior parte dos eventos de instabilidade, independentemente da arquitetura utilizada. Os resultados demonstram que é possível construir *parsers* de dependências competitivos para o português brasileiro sem o BiLSTM intermediário, com cabeçotes conectados diretamente ao encoder com ajuste fino completo, e estabelecem a preferência pelo cabeçote linear em cenários de corpus limitado como diretriz prática para o desenvolvimento de sistemas de PLN.

Palavras-chave: Análise de Dependência Sintática Processamento de Linguagem Natural Transformers Português Brasileiro

ABSTRACT

LIMA, Guilherme Dallmann. **ParseH2IA: Development of a Parser for Syntactic Dependency Analysis of Brazilian Portuguese with Deep Learning**. Advisor: Larissa Astrogildo de Freitas. 2026. 165 f. Dissertation (Masters in Computer Science) – Technology Development Center, Federal University of Pelotas, Pelotas, 2026.

Syntactic dependency parsing is a fundamental Natural Language Processing (NLP) task that identifies grammatical relations between words in a sentence and produces a structural representation serving as a foundation for higher-level tasks such as information extraction, sentiment analysis, and machine translation. For Brazilian Portuguese, the state of the art is represented by PortParser, which combines the BERTimbau Large encoder with a bidirectional BiLSTM layer and a biaffine mechanism, achieving 96.08% Unlabeled Attachment Score (UAS) and 94.61% Labeled Attachment Score (LAS) on the Porttinari corpus. This dissertation presents **ParseH2IA**, a dependency parser for Brazilian Portuguese that removes the BiLSTM present in PortParser and connects the encoder directly to prediction heads with full fine-tuning of all parameters. The work conducts a systematic comparison of four pre-trained encoders, BERTimbau Large, BERTimbau Base, mBERT, and JabuticaBERT, crossed with two head architectures (linear and biaffine) and two training regimes (multi-task learning with UPOS and ablation without UPOS), totaling 16 experimental configurations. The linear head projects the encoder output through three independent dense layers for HEAD, DEPREL, and UPOS; the biaffine head uses four specialized MLPs and two biaffine attention mechanisms for HEAD and DEPREL. Hyperparameters for each configuration were independently optimized through Bayesian search with five-fold cross-validation. The best result was achieved by BERTimbau Large with a linear head and multi-task learning, reaching 94.99% UAS, 93.84% LAS, and 99.28% UPOS accuracy, within 1.1 percentage points of PortParser in UAS. The most significant finding is the **consistent superiority of the linear head over the biaffine head** in three of the four encoders evaluated, with margins of up to 5.78 percentage points in UAS, contradicting the literature's expectation for full fine-tuning scenarios. This result is interpreted in light of the corpus characteristics: with only 5,893 training sentences in Porttinari, the biaffine head, with its four MLPs and bilinear weight matrices, presents a higher risk of overfitting compared to the linear head, whose lower parametric capacity acts as implicit regularization and yields more stable generalization. As a methodological contribution, this work proposes a training instability analysis based on epoch-level prediction tracking, quantifying *forgetting* events (correct-to-incorrect transitions) and

recovery events (incorrect-to-correct transitions) per dependency label. This analysis reveals that the biaffine head is systematically more unstable during convergence, and that high-frequency labels with diffuse decision boundaries, `nmod` and `obl`, concentrate most instability events regardless of architecture. The results demonstrate that competitive dependency parsers for Brazilian Portuguese can be built without the intermediate BiLSTM, with heads connected directly to a fully fine-tuned encoder, and establish a preference for the linear head in limited-corpus scenarios as a practical guideline for NLP system development.

Keywords: Syntactic Dependency Parsing Natural Language Processing Transformers
Brazilian Portuguese

LISTA DE FIGURAS

Figura 1	Estrato da Linguagem.	37
Figura 2	Diagrama Tarefas NLU e NLG.	39
Figura 3	Escala de Ordens.	40
Figura 4	Exemplo de estruturação da escala de ordens.	41
Figura 5	Exemplo de estruturação com colchetes.	42
Figura 6	Exemplo de estruturação com árvore.	43
Figura 7	Exemplo de uma Árvore de Dependência Sintática.	46
Figura 8	Representação parentética de sentença	46
Figura 9	Exemplo de uma Árvore de Dependência Sintática.	46
Figura 10	Arquitetura Transformers (Vaswani; Shazeer; Parmar; Uszkoreit; Jones; Gomez; Kaiser; Polosukhin, 2017a).	53
Figura 11	Exemplo de um vetor de embeddings.	54
Figura 12	Scaled Dot-Product Attention (Vaswani; Shazeer; Parmar; Uszkoreit; Jones; Gomez; Kaiser; Polosukhin, 2017a).	56
Figura 13	Multi Head Attention (Vaswani; Shazeer; Parmar; Uszkoreit; Jones; Gomez; Kaiser; Polosukhin, 2017a).	56
Figura 14	Perceptron.	59
Figura 15	Perceptron Multicamadas.	60
Figura 16	Arquitetura BERT.	61
Figura 17	Processo de classificação de um token com o BERT.	62
Figura 18	Arquitetura Apenas Decodificadora.	64
Figura 19	Arquitetura Hard Parameter Sharing.	65
Figura 20	Arquitetura Soft Parameter Sharing.	66
Figura 21	Pipeline Metodologia.	71
Figura 22	Frequência UPOS conjunto de Treino.	73
Figura 23	Frequência UPOS conjunto de Desenvolvimento.	73
Figura 24	Frequência UPOS conjunto de Teste.	73
Figura 25	Frequência DEPREL conjunto de Treino.	74
Figura 26	Frequência DEPREL conjunto de Desenvolvimento.	74
Figura 27	Frequência DEPREL conjunto de Teste.	74
Figura 28	Box plot do conjunto de treino, mostrando a distribuição do tamanho das sentenças em tokens, incluindo outliers.	75
Figura 29	Box plot do conjunto de desenvolvimento, mostrando a distribuição do tamanho das sentenças em tokens, incluindo outliers.	75
Figura 30	Box plot do conjunto de teste, mostrando a distribuição do tamanho das sentenças em tokens, incluindo outliers.	75

Figura 31	Arquitetura multitarefa baseada em camadas lineares.	83
Figura 32	Arquitetura multitarefa com mecanismo biaffine para predição de HEAD e DEPREL.	85
Figura 33	Exemplo da aplicação da técnica de validação cruzada com $k = 5$. .	88
Figura 34	Interface de monitoramento dos experimentos utilizando WandB. . .	89
Figura 35	Comparação de desempenho (UAS e LAS) de todos os modelos avaliados, ordenados por UAS decrescente. As linhas tracejadas indicam o desempenho do PortParser como referência (UAS = 96,08% , LAS = 94,61%). cU = com UPOS (MTL); sU = sem UPOS (ablação).	93
Figura 36	Curvas de UAS por época para os modelos Linear MTL de cada encoder. O eixo vertical indica o UAS (%) no conjunto de teste ao final de cada época.	94
Figura 37	UAS (%) por variante de arquitetura (Lin/Biaff × cU/sU) e encoder. A superioridade do cabeçote linear sobre o biaffine é consistente em três dos quatro encoders.	95
Figura 38	Diferença de UAS e LAS (em pontos percentuais) entre modelos MTL (com UPOS) e de ablação (sem UPOS), por variante e encoder. Valores positivos indicam que o MTL beneficia o <i>parsing</i> ; negativos indicam prejuízo.	96
Figura 39	Acurácia (%) nos rótulos DEPREL mais difíceis para os quatro melhores modelos. <i>orphan</i> e <i>dislocated</i> apresentam desempenho inferior a 50% na maioria dos modelos.	98
Figura 40	Curvas de UAS por época para as quatro variantes do BERTimbau Large. A variante Linear MTL converge de forma mais estável que a Biaffine MTL, que registra mais oscilações nas primeiras 20 épocas. .	99
Figura 41	Número de eventos de <i>forgetting</i> (acerto → erro) por rótulo DEPREL para as quatro variantes do BERTimbau Large. Rótulos de alta frequência com fronteiras de decisão difusas (<i>nmod</i> , <i>obl</i>) são os mais instáveis.	100

LISTA DE TABELAS

Tabela 1	Principais componentes do Penn Treebank	42
Tabela 2	Exemplo de sentença anotada no formato UD.	44
Tabela 3	Parser GLC para Análise Sintática	50
Tabela 4	Parser GSC para Análise Sintática	51
Tabela 5	Comparação entre os trabalhos relacionados e a proposta desta dissertação.	70
Tabela 6	Distribuição dos dados no conjunto Porttinari-base.	72
Tabela 7	Comparação entre as arquiteturas avaliadas para as tarefas de UPOS, DEPREL e HEAD.	84
Tabela 8	Hiperparâmetros fixos utilizados durante o treinamento.	87
Tabela 9	Espaço de busca utilizado na otimização de hiperparâmetros com Optuna.	87
Tabela 10	Configurações de reprodutibilidade aplicadas a todos os experimentos.	89
Tabela 11	Hiperparâmetros selecionados para cada modelo após otimização com Optuna.	90
Tabela 12	Configuração de hardware utilizada nos experimentos.	90
Tabela 13	Comparação Baseline	92
Tabela 14	Melhores resultados por modelo considerando as métricas UPOS, UAS e LAS.	92
Tabela 15	Custo computacional estimado por fase experimental (GPU: RTX 4090). Valores extraídos do WandB (hf-optuna).	107
Tabela 16	Resultados dos experimentos com mBERT (Linear)	125
Tabela 17	Resultados dos experimentos com BERTimbau-Base (Linear) . . .	125
Tabela 18	Resultados dos experimentos com BERTimbau-Large (Linear) . . .	125
Tabela 19	Resultados dos experimentos com modernJabuticaBERT-Base-1k (Linear)	126
Tabela 20	Resultados dos experimentos com mBERT (Biaffine)	126
Tabela 21	Resultados dos experimentos com BERTimbau Base (Biaffine) . . .	126
Tabela 22	Resultados dos experimentos com BERTimbau Large (Biaffine) . .	127
Tabela 23	Resultados dos experimentos com JabuticaBERT (Biaffine)	127
Tabela 24	Resultados do treinamento por época para o modelo mBERT (Linear)	128
Tabela 25	Resultados do treinamento por época - Bertimbau-Base (Linear) . .	129
Tabela 26	Resultados do treinamento por época - BERTimbau-Large (Linear)	130

Tabela 27	Resultados do treinamento por época para o modelo JabuticaBERT (Linear)	131
Tabela 28	Resultados do treinamento por época para o modelo mBERT (Biaffine)	132
Tabela 29	Resultados de treinamento do BERTimbau Base (Biaffine)	133
Tabela 30	Resultados de treinamento do BERTimbau Large (Biaffine)	134
Tabela 31	Resultados de treinamento do JabuticaBERT (Biaffine)	135
Tabela 32	Resultados do treinamento por época para o modelo mBERT (Linear) Sem UPOS	136
Tabela 33	Resultados do treinamento por época - Bertimbau-Base (Linear) Sem UPOS	137
Tabela 34	Resultados do treinamento por época - BERTimbau-Large (Linear) Sem UPOS	138
Tabela 35	Resultados do treinamento por época para o modelo JabuticaBERT (Linear) Sem UPOS	139
Tabela 36	Resultados do treinamento por época para o modelo mBERT (Biaffine) Sem UPOS	140
Tabela 37	Resultados de treinamento do BERTimbau Base (Biaffine) Sem UPOS	141
Tabela 38	Resultados de treinamento do BERTimbau Large (Biaffine) Sem UPOS	142
Tabela 39	Resultados de treinamento do JabuticaBERT (Biaffine) Sem UPOS	143
Tabela 40	Selected training epochs of PortParser.	144
Tabela 41	Resultados validação - mBert (Linear)	144
Tabela 42	Resultados validação - Bertimbau-Base (Linear)	144
Tabela 43	Resultados validação - Bertimbau-Large (Linear)	144
Tabela 44	Resultados validação - JabuticaBERT (Linear)	144
Tabela 45	Resultados validação - mBert (Biaffine)	144
Tabela 46	Resultados validação - BERTimbau Base (Biaffine)	144
Tabela 47	Resultados validação - BERTimbau Large (Biaffine)	144
Tabela 48	Resultados validação - JabuticaBERT (Biaffine)	144
Tabela 49	Resultados validação - mBert (Linear) Sem UPOS	144
Tabela 50	Resultados validação - Bertimbau-Base (Linear) Sem UPOS	144
Tabela 51	Resultados validação - Bertimbau-Large (Linear) Sem UPOS	145
Tabela 52	Resultados validação - JabuticaBERT (Linear) Sem UPOS	145
Tabela 53	Resultados validação - mBert (Biaffine) Sem UPOS	145
Tabela 54	Resultados validação - BERTimbau Base (Biaffine) Sem UPOS	145
Tabela 55	Resultados validação - BERTimbau Large (Biaffine) Sem UPOS	145
Tabela 56	Resultados validação - JabuticaBERT (Biaffine) Sem UPOS	145
Tabela 57	Resultados validação - PortParser	145
Tabela 58	Acurácia por rótulo — mBERT (mB). Lin =Linear com UPOS, Biaff =Biaffine com UPOS, sLin =sem UPOS Linear, sBiaff =sem UPOS Biaffine. Negrito =melhor por rótulo.	146
Tabela 59	Acurácia por rótulo — BERTimbau Base (BtB). Lin =Linear com UPOS, Biaff =Biaffine com UPOS, sLin =sem UPOS Linear, sBiaff =sem UPOS Biaffine. Negrito =melhor por rótulo.	147
Tabela 60	Acurácia por rótulo — BERTimbau Large (BtL). Lin =Linear com UPOS, Biaff =Biaffine com UPOS, sLin =sem UPOS Linear, sBiaff =sem UPOS Biaffine. Negrito =melhor por rótulo.	148

Tabela 61	Acurácia por rótulo — JabuticaBERT (JaB). Lin = Linear com UPOS, Biaff = Biaffine com UPOS, sLin = sem UPOS Linear, sBiaff = sem UPOS Biaffine. Negrito = melhor por rótulo.	149
Tabela 62	Instabilidade por rótulo de dependência (DEPREL) — mBERT (mB). As 20 categorias com maior número de esquecimentos (Lin) são exibidas. Fg. máximo e Ac.F máxima por linha em negrito	150
Tabela 63	Instabilidade por categoria morfossintática (UPOS) — mBERT (mB). Apenas modelos MTL. Fg. máximo e Ac.F máxima por linha em negrito	151
Tabela 64	UAS (HEAD) por época de avaliação — mBERT (mB). Valor máximo por linha em negrito	152
Tabela 65	Instabilidade por rótulo de dependência (DEPREL) — BERTimbau Base (BtB). As 20 categorias com maior número de esquecimentos (Lin) são exibidas. Fg. máximo e Ac.F máxima por linha em negrito	153
Tabela 66	Instabilidade por categoria morfossintática (UPOS) — BERTimbau Base (BtB). Apenas modelos MTL. Fg. máximo e Ac.F máxima por linha em negrito	154
Tabela 67	UAS (HEAD) por época de avaliação — BERTimbau Base (BtB). Valor máximo por linha em negrito	155
Tabela 68	Instabilidade por rótulo de dependência (DEPREL) — BERTimbau Large (BtL). As 20 categorias com maior número de esquecimentos (Lin) são exibidas. Fg. máximo e Ac.F máxima por linha em negrito	156
Tabela 69	Instabilidade por categoria morfossintática (UPOS) — BERTimbau Large (BtL). Apenas modelos MTL. Fg. máximo e Ac.F máxima por linha em negrito	157
Tabela 70	UAS (HEAD) por época de avaliação — BERTimbau Large (BtL). Valor máximo por linha em negrito	158
Tabela 71	Instabilidade por rótulo de dependência (DEPREL) — JabuticaBERT (JaB). As 20 categorias com maior número de esquecimentos (Lin) são exibidas. Fg. máximo e Ac.F máxima por linha em negrito	159
Tabela 72	Instabilidade por categoria morfossintática (UPOS) — JabuticaBERT (JaB). Apenas modelos MTL. Fg. máximo e Ac.F máxima por linha em negrito	160
Tabela 73	UAS (HEAD) por época de avaliação — JabuticaBERT (JaB). Valor máximo por linha em negrito	161

LISTA DE ABREVIATURAS E SIGLAS

IA	Inteligência Artificial
PLN	Processamento de Linguagem Natural
NLU	Compreensão da Linguagem Natural
NLG	Geração da Linguagem Natural
NLP	<i>Natural Language Processing</i>
POS	Categoria Morfossintática (<i>Part-of-Speech</i>)
UPOS	Categoria Morfossintática Universal (<i>Universal POS</i>)
NER	Reconhecimento de Entidades Nomeadas (<i>Named Entity Recognition</i>)
SRL	Rotulação de Papéis Semânticos (<i>Semantic Role Labeling</i>)
QA	Resposta a Perguntas (<i>Question Answering</i>)
ABSA	Análise de Sentimento Baseada em Aspecto
SA	Análise Sintática
DP	Análise de Dependência (<i>Dependency Parsing</i>)
CP	Análise de Constituintes (<i>Constituency Parsing</i>)
GLC	Gramáticas Livres de Contexto
GSC	Gramáticas Sensíveis ao Contexto
UD	Dependências Universais (<i>Universal Dependencies</i>)
GSD	Google Universal Dependencies (<i>treebank UD-Portuguese-GSD</i>)
HEAD	Rótulo de cabeça de arco de dependência
DEPREL	Rótulo de relação de dependência (<i>Dependency Relation</i>)
UAS	Pontuação de Anexação Não-Rotulada (<i>Unlabeled Attachment Score</i>)
LAS	Pontuação de Anexação Rotulada (<i>Labeled Attachment Score</i>)
BERT	Representações de Encoder Bidirecional a partir de Transformers (<i>Bidirectional Encoder Representations from Transformers</i>)
MLM	Modelo de Linguagem com Mascaramento (<i>Masked Language Model</i>)
NSP	Previsão de Próxima Sentença (<i>Next Sentence Prediction</i>)

RNN	Rede Neural Recorrente (<i>Recurrent Neural Network</i>)
LSTM	Memória de Longo e Curto Prazo (<i>Long Short-Term Memory</i>)
BiLSTM	LSTM Bidirecional (<i>Bidirectional LSTM</i>)
CNN	Rede Neural Convolucional (<i>Convolutional Neural Network</i>)
MLP	Perceptron Multicamada (<i>Multi-Layer Perceptron</i>)
ELU	Unidade Linear Exponencial (<i>Exponential Linear Unit</i>)
MTL	Aprendizado Multi-Tarefa (<i>Multi-Task Learning</i>)
AdamW	Otimizador Adam com Decaimento de Peso (<i>Adam Weight Decay</i>)
LoRA	Adaptação de Baixo Posto (<i>Low-Rank Adaptation</i>)
GPU	Unidade de Processamento Gráfico (<i>Graphics Processing Unit</i>)
VRAM	Memória de Vídeo (<i>Video Random Access Memory</i>)
RTX	Ray Tracing (<i>NVIDIA RTX GPU series</i>)
WandB	Weights & Biases

SUMÁRIO

1	INTRODUÇÃO	21
1.1	Motivação	22
1.2	Objetivos	23
1.3	Hipótese de Pesquisa	24
1.4	Metodologia	24
1.5	Organização da Dissertação	25
2	REFERÊNCIAL TEÓRICO	26
2.1	Inteligência Artificial	26
2.2	Linguagem	36
2.3	Processamento de Linguagem Natural	38
2.4	Análise Sintática	39
2.4.1	Análise de Constituintes	40
2.4.2	Análise de Dependências	43
2.4.3	Diferença entre Análise Constituinte e Análise de Dependência	48
2.4.4	Parser	48
2.5	Grandes Modelos de Linguagem	52
2.5.1	Transformers	52
2.5.2	Bidirectional Encoder Representations from Transformers	61
2.5.3	Ajuste-Fino	63
2.5.4	Apenas Codificador	63
2.5.5	Apenas Decodificador	64
2.6	Aprendizado Multi-Tarefa	65
3	TRABALHOS RELACIONADOS	67
4	METODOLOGIA	71
4.1	Seleção do Conjunto de Dados e das Tarefas	71
4.2	Seleção dos Modelos	76
4.3	Estado da Arte	90
5	RESULTADOS E DISCUSSÃO	91
5.1	Desempenho Geral	91
5.2	Comparação entre Encoders	92
5.3	Impacto da Arquitetura do Cabeçote	94
5.3.1	Linear versus Biaffine	94
5.3.2	Impacto do Multi-Task Learning	96
5.4	Análise por Rótulo de Dependência	97

5.4.1	Rótulos de Alta Acurácia	97
5.4.2	Rótulos Difíceis	97
5.5	Análise de Instabilidade do Treinamento	99
5.5.1	Curvas de Aprendizado do BERTimbau Large	99
5.5.2	Instabilidade por Rótulo DEPREL	100
5.6	Discussão Geral	101
5.7	Limitações	103
6	CONCLUSÃO	105
6.1	Sobre a Arquitetura Proposta	105
6.2	Sobre o Conjunto de Dados	106
6.3	Custo Computacional	106
6.4	Síntese dos Resultados	107
6.5	Contribuições	108
6.6	Trabalhos Futuros	109
6.7	Considerações Finais	109
	REFERÊNCIAS	111
.8	Classes de Dependencia	121
.9	Resultado: Busca de Hiperparametros Avaliados	124
.10	Resultado: Treinamento Modelos Avaliados	124
.11	Resultado: Validação Modelos Avaliados	124
.12	Frequência das Tags e instabilidade	124
	APÊNDICE A UM APÊNDICE	162
	ANEXO A UM ANEXO	164
	ANEXO B OUTRO ANEXO	165

1 INTRODUÇÃO

A linguagem é a principal forma de comunicação humana e constitui a base sobre a qual a maior parte do conhecimento produzido pela humanidade está registrado. Com a expansão da internet e das plataformas digitais que, em 2025, reuniam mais de 5,56 bilhões de usuários conectados, dos quais 5,24 bilhões ativos em redes sociais, a produção de conteúdo textual atingiu volumes sem precedentes na história, gerando tanto oportunidades quanto desafios para sua análise automatizada. Nesse cenário, o Processamento de Linguagem Natural (PLN) emerge como área fundamental para o desenvolvimento de sistemas capazes de compreender, estruturar e extrair significado de textos em escala.

Dentre as tarefas do PLN, a **análise sintática de dependências** (*dependency parsing*) ocupa um papel central: ela permite identificar as relações gramaticais entre as palavras de uma sentença, qual palavra é núcleo (*head*) de qual dependente, e qual função sintática (rótulo de dependência, *DEPREL*) essa ligação expressa produzindo uma representação estruturada da gramática da frase. Essa representação é fundamento para tarefas de maior complexidade semântica, como extração de informação, análise de sentimento baseada em aspecto (ABSA), resposta a perguntas e tradução automática (Jurafsky; Martin, 2023). Os fundamentos da análise sintática, do aprendizado profundo e dos modelos de linguagem que suportam este trabalho são apresentados no Capítulo 2.

O desempenho dos *parsers* de dependências avançou radicalmente com o surgimento dos modelos de linguagem pré-treinados baseados em *Transformers* (Vaswani; Shazeer; Parmar; Uszkoreit; Jones; Gomez; Kaiser; Polosukhin, 2017b), em particular o BERT (Devlin; Chang; Lee; Toutanova, 2019), cujas representações contextuais ricas permitiram que sistemas modernos atingissem acurácias próximas ao limite superior dos conjuntos de anotação humana em diversas línguas. O mecanismo de atenção biaffine proposto por Dozat; Manning (2017) tornou-se o cabeçote de predição predominante em sistemas de estado da arte, por modelar explicitamente interações entre pares de tokens candidatos a arco de dependência.

Paralelamente, estudos de *probing* (Tenney; Das; Pavlick, 2019; Hewitt; Manning,

2019; Jawahar; Sagot; Seddah, 2019) revelaram que o BERT codifica, de forma implícita e emergente, uma ampla gama de propriedades linguísticas, incluindo relações de dependência e estrutura de árvore sintática em seus vetores de representação. Essa constatação levanta uma questão de natureza empírica relevante: *se as representações do BERT já organizam sintaticamente o espaço vetorial, é necessário um cabeçote biaffine complexo, ou um cabeçote linear simples é suficiente, e potencialmente superior, mesmo em regime de ajuste fino completo?*

É sobre essa questão que esta dissertação se debruça, apresentando o sistema **ParseH2IA**: um *parser* de dependências sintáticas para o português brasileiro que remove a camada BiLSTM bidirecional presente no PortParser e conecta o encoder diretamente a cabeçotes de predição com ajuste fino completo, avaliado no corpus Porttinari.

1.1 Motivação

O português brasileiro é uma das línguas mais faladas do mundo, com mais de 200 milhões de falantes nativos, e representa volume crescente de conteúdo digital em redes sociais, plataformas jornalísticas, sistemas jurídicos e repositórios científicos. Apesar de sua relevância, o português ainda é considerado uma língua de recursos relativamente limitados (*lower-resource*) no contexto do PLN em comparação ao inglês: os conjuntos de dados anotados são menores e o volume de pesquisa publicada é proporcionalmente menor (Real; Fonseca; Pon; Cravalho; Balage filho; Ramisch, 2020).

O **PortParser** (Lopes; Brisolara; Freitas, 2024) representa o estado da arte em *parsing* de dependências para o português brasileiro: construído sobre o BERTimbau Large (Souza; Nogueira; Lotufo, 2020) com ajuste fino completo do encoder, uma camada BiLSTM bidirecional e mecanismo biaffine, atingiu 96,08% de UAS (*Unlabeled Attachment Score*) e 94,61% de LAS (*Labeled Attachment Score*) no corpus Porttinari, constituindo o principal ponto de referência da área.

A arquitetura do PortParser segue o padrão dominante na literatura: encoder contextual pré-treinado, seguido de uma camada BiLSTM para contextualização adicional, e cabeçotes biaffine para predição de arcos e rótulos. Contudo, o papel do BiLSTM nessa cadeia não está completamente elucidado: dada a já elevada capacidade de contextualização do BERT, o BiLSTM pode ser uma redundância arquitetural. Além disso, a escolha do mecanismo biaffine sobre alternativas mais simples, como projeções lineares diretas, raramente é investigada de forma controlada em cenários de corpus pequeno.

Essa lacuna motiva a investigação conduzida no ParseH2IA: **suprimir o BiLSTM** e comparar sistematicamente cabeçotes linear e biaffine conectados diretamente ao

encoder com ajuste fino completo, em quatro encoders distintos para o português, no corpus Porttinari. Se cabeçotes lineares atingirem desempenho comparável ou superior ao biaffine, isso simplifica substancialmente a arquitetura do sistema sem sacrifício de acurácia, resultado prático com implicações diretas para o desenvolvimento de ferramentas de PLN em português.

1.2 Objetivos

Objetivo Geral

Desenvolver e avaliar o sistema ParseH2IA, um *parser* de dependências sintáticas para o português brasileiro com ajuste fino completo do encoder e sem camada BiLSTM intermediária — comparando sistematicamente o impacto de quatro encoders pré-treinados e duas arquiteturas de cabeçote de predição (linear e biaffine) no desempenho do *parsing*, e propondo uma metodologia de análise da dinâmica de estabilidade do treinamento por rastreamento de predições por época.

Objetivos Específicos

1. Implementar o sistema ParseH2IA com arquitetura modular que suporte múltiplos encoders BERT e múltiplas arquiteturas de cabeçote, linear (três camadas densas independentes para HEAD, DEPREL e UPOS) e biaffine (quatro MLPs e dois mecanismos de atenção biaffine) , operando sem camada BiLSTM intermediária.
2. Avaliar quatro encoders pré-treinados, BERTimbau Large, BERTimbau Base, mBERT e JabuticabERT, no corpus Porttinari, reportando desempenho em UAS, LAS e acurácia de UPOS para cada configuração.
3. Comparar dois regimes de treinamento, multi-tarefa com UPOS (*MTL*) e ablação sem UPOS, em cruzamento com as quatro arquiteturas de encoder e dois cabeçotes, totalizando 16 configurações experimentais.
4. Conduzir busca sistemática e independente de hiperparâmetros por configuração de encoder e cabeçote, garantindo que as comparações não sejam confundidas por diferenças de otimização.
5. Desenvolver e aplicar uma metodologia de análise de instabilidade do treinamento baseada no rastreamento de predições por época, quantificando eventos de *forgetting* (acerto $\rightarrow$ erro) e *recovery* (erro $\rightarrow$ acerto) por rótulo de dependência e por token.
6. Interpretar os resultados experimentais à luz das características do corpus Porttinari e das propriedades das representações BERT, avaliando em que medida

a simplicidade arquitetural do cabeçote linear é suficiente, ou superior, ao mecanismo biaffine em cenário de corpus pequeno com ajuste fino completo.

1.3 Hipótese de Pesquisa

A hipótese central desta dissertação é que, **sem a camada BiLSTM intermediária**, um cabeçote linear é ao menos tão eficaz quanto o mecanismo biaffine para a tarefa de *parsing* de dependências para o português brasileiro com ajuste fino completo — especialmente em cenários de corpus limitado, como o Porttinari (5.893 sentenças de treino) , porque o cabeçote linear introduz menos parâmetros e menor risco de sobreajuste em comparação ao biaffine, enquanto o ajuste fino completo do encoder compensa a menor expressividade arquitetural do cabeçote.

Essa hipótese é sustentada por dois eixos de evidências. O primeiro é empírico: estudos de *probing* demonstram que sondas lineares simples conseguem recuperar relações de dependência das representações BERT com alta acurácia (Hewitt; Manning, 2019), indicando que a informação sintática é linearmente acessível no espaço de representações do encoder. O segundo é prático: o cabeçote biaffine aplica quatro MLPs e transformações bilineares que adicionam substancialmente o número de parâmetros treináveis no cabeçote; em corpus de treinamento pequeno, essa capacidade adicional pode manifestar-se como sobreajuste em vez de generalização superior, enquanto o cabeçote linear, com seus três projetores independentes, atua como regularização implícita.

Se confirmada, essa hipótese tem implicações práticas diretas: ao construir *parsers* de dependências sem BiLSTM e com corpus limitado, a arquitetura linear é preferível ao biaffine, simplificando o sistema sem sacrifício de desempenho.

1.4 Metodologia

Para atingir os objetivos propostos, a metodologia do ParseH2IA compreende quatro etapas principais, detalhadas no Capítulo 4:

1. **Implementação do sistema:** desenvolvimento da arquitetura ParseH2IA com encoder BERT com ajuste fino completo e cabeçotes modulares para HEAD, DEPREL e UPOS, suprimindo a camada BiLSTM presente no PortParser (Lopes; Brisolara; Freitas, 2024), nas variantes linear e biaffine.
2. **Busca de hiperparâmetros:** otimização bayesiana de taxa de aprendizado, *weight decay* e *warmup ratio* com validação cruzada em 5 *folds*, conduzida independentemente para cada uma das 8 combinações encoder × cabeçote.
3. **Treinamento e avaliação:** treinamento dos 16 modelos com os hiperparâmetros

ótimos e avaliação no conjunto de teste do Porttinari, com reporte de UAS, LAS e acurácia de UPOS em comparação ao PortParser como linha de base.

4. **Análise de instabilidade:** rastreamento das predições de cada token em cada época do treinamento, computação de eventos de *forgetting* e *recovery* por rótulo, e comparação da estabilidade de convergência entre arquiteturas e encoders.

1.5 Organização da Dissertação

Esta dissertação está organizada da seguinte forma:

O **Capítulo 2** apresenta o referencial teórico que fundamenta o trabalho: os conceitos de Inteligência Artificial, Aprendizado de Máquina e Aprendizado Profundo; os fundamentos da linguagem e do Processamento de Linguagem Natural; a análise sintática de constituição e de dependências; os modelos de linguagem pré-treinados baseados em *Transformers*; e os trabalhos relacionados ao *parsing* para o português brasileiro.

O **Capítulo 4** descreve a metodologia empregada no ParseH2IA: a arquitetura dos modelos, os encoders avaliados, os cabeçotes implementados, a estratégia de busca de hiperparâmetros e o processo de treinamento e avaliação.

O **Capítulo 5** apresenta e discute os resultados obtidos pelos 16 modelos treinados, comparando-os ao PortParser em UAS, LAS e acurácia de UPOS, analisando o impacto da arquitetura do cabeçote e do encoder, investigando a estabilidade de convergência e discutindo os achados à luz das características do corpus Porttinari e das representações BERT.

O **Capítulo 6** sintetiza as contribuições desta dissertação, discute as limitações do trabalho e aponta direções para investigações futuras.

Os **Apêndices** reúnem o material de suporte utilizado ao longo da dissertação: as definições completas dos rótulos de dependência do Porttinari (Apêndice .8), os resultados detalhados da busca de hiperparâmetros por configuração de modelo (Apêndice .9) e as tabelas de desempenho por época de cada modelo treinado (Apêndice .10).

2 REFERÊNCIAL TEÓRICO

Este capítulo apresenta os fundamentos teóricos essenciais para a compreensão do presente trabalho, abordando conceitos centrais da Inteligência Artificial e do Processamento de Linguagem Natural. Inicialmente, discute-se a natureza da **Inteligência Artificial e linguagem**, suas complexidades, seguida das principais técnicas de **Processamento de Linguagem Natural**, com destaque para a análise sintática como ferramenta para compreensão estrutural das sentenças. Em seguida, são apresentados os avanços recentes proporcionados pelos **grandes modelos de linguagem**, incluindo suas capacidades de realizar tarefas complexas e múltiplas simultaneamente por meio do **aprendizado multi-tarefa**.

2.1 Inteligência Artificial

A internet está infundida na vida cotidiana da população. Segundo Statista, em fevereiro de 2025, 5.56 bilhões de pessoas possuíam algum tipo de acesso à internet, número que representa 67.9% da população mundial. Desse montante, 5.24 bilhões de pessoas tinham acesso às redes sociais, o equivalente a 63.9% da população mundial. Com essa quantidade expressiva de usuários com acesso às redes sociais, surgem alguns desafios inerentes. As redes sociais são o meio de comunicação mais utilizado pela população mundial. Isso se deve às funcionalidades possíveis por meio delas, que permitem redigir comentários, compartilhar informações, experiências e opiniões em tempo real, sem nenhuma restrição ou filtro (Lima; Oliveira; Silva; Pinheiro; Luz; Freitas, 2024).

Devido a essa liberdade que as redes sociais proporcionam, há a necessidade de técnicas para avaliar essas informações redigidas, a fim de mitigar a propagação de mensagens prejudiciais que podem conter ofensas, discriminações, entre outros. Segundo o estudo (TechJury, 2023), em 2023, diariamente eram gerados 560GB em *tweets* no X¹; no WhatsApp², 41.666.667 milhões de mensagens eram enviadas di-

¹<https://x.com/>

²<https://whatsapp.com/>

ariamente; no Facebook³, 150.000 mensagens eram enviadas e recebidas todos os dias. Essa quantidade massiva de dados torna impossível a avaliação manual, sendo necessária a criação ou melhoria de técnicas para a análise desses textos. Devido à crescente massa de textos que circulam nas redes sociais, áreas como a Inteligência Artificial (do inglês *Artificial Intelligence* – AI) ganham ênfase.

A AI é um campo da computação dedicado a desenvolver sistemas capazes de executar tarefas que normalmente exigiriam inteligência humana, como percepção, tomada de decisão, aprendizado e adaptação (Dobrev, 2012). Formalmente, pode-se definir AI como um programa que, operando em um ambiente arbitrário, com suas próprias regras e desafios, se comporta de maneira a alcançar um desempenho equivalente ou até superior ao de um ser humano naquele mesmo contexto. Segundo Turing, uma máquina pode ser considerada inteligente se for capaz de enganar um interrogador humano em uma conversa por texto, a ponto de este não conseguir distinguir se está interagindo com um ser humano ou com uma máquina.

O termo AI foi cunhado por McCarthy; Minsky; Rochester; Shannon em 31 de agosto de 1955. Naquela época, o estudo enfatizava que todo aspecto do aprendizado, assim como qualquer outra característica da inteligência, poderia ser descrito com precisão suficiente para que uma máquina fosse capaz de simulá-lo. Contudo, o conceito que embasa a inteligência artificial é muito mais antigo.

Suas raízes remontam a Aristóteles (384–322 a.C.), que introduziu o raciocínio dedutivo por meio dos silogismos. Entre os séculos XIII e XIV (c. 1235–1316), Ramon Llull desenvolveu ainda mais essa ideia ao criar uma máquina conceitual chamada *Ars Magna*, projetada para responder a todas as perguntas (Rai, 2024). Já em 1642, o filósofo e matemático Blaise Pascal construiu a Pascalina, uma máquina capaz de realizar operações de adição e subtração, demonstrando que processos antes dependentes exclusivamente de seres humanos podiam ser automatizados (Rai, 2024). Em 1854, George Boole foi pioneiro na lógica proposicional ao propor um sistema algébrico capaz de representar o raciocínio lógico de forma matemática. Outros pensadores, como Gottlob Frege e Gottfried Wilhelm Leibniz, também contribuíram significativamente para essa evolução (Rai, 2024), ajudando a consolidar a ideia de que máquinas podem tomar decisões baseadas em princípios lógicos, assim como os seres humanos.

Ao longo da história, o ser humano demonstrou fascínio em criar algo capaz de replicar sua forma de pensar e tomar decisões. Esse interesse, aliado aos avanços tecnológicos, deu origem a ciclos de grande empolgação, conhecidos como “verões da AI”, seguidos por períodos de frustração e estagnação, denominados “invernos da AI”. Esses períodos estavam associados a avanços e estagnações. Esses avanços eram geralmente impulsionados pela concepção de novos conceitos e técnicas promissoras,

³<https://www.facebook.com/>

enquanto as frustrações surgiam quando tais inovações não alcançavam os resultados esperados, levando à estagnação da área.

Conforme descrito pelos autores Nilsson; Hilpisch; Yao; Zhou; Jia; Baesen; Vlasselaer; Verbeke, Russell; Norvig e Toosi; Bottino; Saboury; Siegel; Rahmim podemos dividir os ciclos de “verões da AI” e “invernos da AI” em sete grandes períodos. Além desses ciclos, houve um ciclo denominado “Gestação da AI” (de 1943 a 1956). O primeiro verão (de 1956 a 1969) marcou o entusiasmo inicial com os avanços na lógica simbólica e nas redes neurais primitivas. O primeiro inverno (de 1969 a 1980) as expectativas não foram atendidas e o financiamento foi drasticamente reduzido. O segundo verão (de 1980 a 1987) foi impulsionado pelo desenvolvimento dos sistemas especialistas. Em seguida, o segundo inverno (de 1987 a 1993) ocorreu devido à limitação desses sistemas e à perda de interesse comercial. O terceiro verão (de 1993 a 2006) apresentou avanços significativos no aprendizado de máquina e maior poder computacional. O terceiro inverno da IA (de 2006 a 2011), mais sutil, foi marcado por uma desaceleração no entusiasmo. Desde 2012, vivemos o quarto verão da AI, caracterizado por avanços expressivos em deep learning, processamento de linguagem natural e grandes modelos de linguagem, que continuam em evolução até o presente momento (2026).

Gestação da AI

O período de 1943 a 1956 é denominado Gestação da AI, pois marcou o ponto de partida formal e organizado do campo da AI. Durante essa fase, foram propostos os primeiros modelos artificiais que buscavam simular ou replicar aspectos da inteligência humana. Um dos modelos desenvolvidos nesse período, é o neurônio lógico (Mcculloch; Pitts, 1943). Esse modelo foi criado como uma abstração teórica para simular o funcionamento básico de um neurônio biológico, realizando operações lógicas com saídas binárias (0 ou 1), o que pode ser interpretado como decisões binárias simples.

Além disso, esse período foi significativo por incluir a criação do termo AI e a proposição do Teste de Turing, um experimento concebido para avaliar se uma máquina é capaz de exibir comportamento inteligente indistinguível do de um ser humano.

Nascimento e Primeiro Verão

O primeiro verão da AI foi marcado por grande entusiasmo e investimentos significativos na área, logo após sua consolidação como um campo formal da computação. John McCarthy descreveu esse período com a expressão: “Olha, mãe, sem as mãos”, refletindo o otimismo e a confiança nas possibilidades da AI.

O sucesso inicial teve início com Allen Newell e Herbert Simon e o desenvolvimento do *Logic Theorist* (Newell; Simon, 1956), considerado o primeiro programa da área de AI, projetado para imitar os métodos humanos de resolução de problemas. Esse

programa era capaz de demonstrar teoremas matemáticos, evidenciando a aplicação de raciocínio simbólico.

Outro marco desse período foi o desenvolvimento de programas para o jogo de damas, conhecido como Checkers (Samuel, 1959). Esses programas foram capazes de aprender a jogar em um nível equivalente ao de um amador forte. Durante esse processo, refutou-se a ideia, até então amplamente aceita, de que computadores só poderiam executar tarefas explicitamente programadas. O programa foi inclusive apresentado na televisão em 1956.

Nesse mesmo período, foi definida a linguagem de alto nível LISP, que se tornaria a principal linguagem de programação da AI pelas três décadas seguintes (Mccarthy, 1978).

Além disso, houve o aprimoramento do modelo neurológico artificial conhecido como Perceptron (Rosenblatt, 1958). Esse modelo tomava decisões classificando entradas de forma binária. Rosenblatt afirmava que ele seria capaz de aprender a reconhecer linguagem natural, identificar letras e palavras, e até realizar traduções automáticas.

Primeiro Inverno

O primeiro inverno da AI foi marcado por uma combinação de decepções, cortes de financiamento e crescente ceticismo em relação ao futuro da área. Esse período se estendeu do final da década de 1960 até o final da década de 1970. A frustração surgiu principalmente devido ao não cumprimento de diversas promessas ambiciosas feitas durante o primeiro verão da AI, as quais acabaram sendo vistas como fracassos.

Um exemplo emblemático foi a declaração de Herbert Simon, em 1957, afirmando que, dentro de dez anos, um computador seria campeão mundial de xadrez e que um teorema matematicamente significativo seria provado por uma máquina. Embora essas previsões eventualmente tenham se concretizado, isso só ocorreu muitos anos depois. O otimismo de Simon se baseava nos bons resultados iniciais obtidos pelos primeiros sistemas de AI.

Outro fator relevante foi o fracasso dos projetos de tradução automática. O Conselho Nacional de Pesquisa dos Estados Unidos da América financiou projetos com o objetivo de traduzir textos técnicos do russo para o inglês, buscando compreender o sucesso da missão Sputnik. No entanto, esses projetos falharam, sobretudo devido à incapacidade dos sistemas em lidar com ambiguidades linguísticas e compreender o contexto das frases. Como consequência, todo o financiamento governamental norte-americano destinado à tradução automática foi cortado, levando ao cancelamento dos projetos.

A limitação computacional também desempenhou um papel crucial. Naquela época, ainda não havia uma teoria consolidada da complexidade computacional, e

acreditava-se, de forma ingênua, que o poder computacional era ilimitado e que bastaria aprimorar os hardwares para resolver qualquer problema. Autores como Friedberg; Dunham; North acreditavam que algoritmos, então chamados de Evolução de Máquina, hoje conhecidos como algoritmos genéticos, precisavam testar todas as combinações possíveis. Quando isso se tornava inviável, a solução sugerida era simplesmente aumentar o poder de processamento.

Relatórios técnicos também contribuíram para esse período de estagnação. Um dos mais influentes foi o Relatório Lighthill (Lighthill, 2020), que expôs a incapacidade dos sistemas da época de lidar com a chamada explosão combinatória. Esse relatório levou o governo britânico a encerrar quase todo o apoio à pesquisa em inteligência artificial nas universidades do Reino Unido.

Além disso, outros trabalhos apontaram limitações fundamentais dos modelos desenvolvidos no primeiro verão da AI. Um exemplo foi a crítica ao modelo do Perceptron, que demonstrou sérias limitações ao lidar com dados que não são linearmente separáveis, como é o caso da função lógica XOR (Marvin; Seymour, 1969).

Apesar das dificuldades, o período também foi marcado por contribuições importantes que mais tarde impulsionariam novos avanços na área. Um dos conceitos introduzidos nesse contexto foi o algoritmo de retropropagação (do inglês, *backpropagation*), que se tornaria fundamental para o treinamento de redes neurais profundas nas décadas seguintes.

Segundo Verão

O segundo verão da AI ocorreu no início dos anos 1980 e se estendeu até meados da década de 1990. Durante esse período, o campo da AI voltou a atrair atenção e investimentos, principalmente devido ao sucesso dos sistemas especialistas.

Até então, acreditava-se que a AI seria de propósito geral, ou seja, que um único sistema poderia encadear etapas elementares de raciocínio para encontrar soluções completas para qualquer tipo de problema. No entanto, os fracassos ocorridos durante o primeiro inverno da AI demonstraram as limitações dessa abordagem. Como resposta, os pesquisadores passaram a concentrar seus esforços no desenvolvimento de sistemas de domínio específico.

Esses sistemas eram projetados para resolver problemas dentro de uma única classe ou área de conhecimento, o que permitia realizar etapas de raciocínio mais profundas sem enfrentar a explosão combinatória. Dessa forma, conseguiam lidar de maneira mais eficiente com os casos típicos de áreas restritas e especializadas.

Um exemplo notável é o sistema DENDRAL (Lederberg, 1965), criado para inferir a estrutura molecular de substâncias com base nas informações fornecidas por um espectrômetro de massas. O DENDRAL foi extremamente relevante por ser o primeiro sistema bem-sucedido nesse paradigma, graças ao uso de um grande volume

de regras especializadas.

Outro sistema importante foi o MYCIN (Shortliffe; Davis; Axline; Buchanan; Green; Cohen, 1975), desenvolvido para diagnosticar infecções sanguíneas. Ele utilizava cerca de 450 regras especializadas, representando um avanço significativo no uso de conhecimento médico em sistemas computacionais. Já o sistema SHRDLU foi desenvolvido com o objetivo de compreender a linguagem natural (Winograd, 1971). Ele conseguiu superar limitações anteriores, como ambiguidades e referências pronominais, mas isso só foi possível porque o modelo foi projetado especificamente para operar em um ambiente altamente restrito e controlado. Ainda assim, a dificuldade em lidar com a análise sintática fora de contextos limitados acabou gerando problemas semelhantes aos enfrentados por sistemas anteriormente.

Durante esse período, também houve avanços importantes no uso do algoritmo de aprendizado por retropropagação, que foi aplicado a diversos problemas de aprendizado supervisionado. A ampla divulgação dos bons resultados obtidos com esse algoritmo gerou um novo entusiasmo em torno da AI.

Com esses avanços e o retorno do otimismo na área, governos como o do Japão e dos Estados Unidos da América retomaram o financiamento de projetos em AI. O governo japonês, por exemplo, financiou uma iniciativa ambiciosa voltada ao desenvolvimento de computadores inteligentes baseados em AI e lógica simbólica, utilizando a linguagem Prolog como base.

Segundo Inverno

O segundo inverno da AI ocorreu no início dos anos 1990 até meados dos anos 2000. Esse período foi marcado por diversos fatores, principalmente relacionados ao mercado. Um dos principais motivos foi o colapso do mercado de máquinas LISP, que eram computadores especializados desenvolvidos exclusivamente para aplicações em AI. Com o advento das máquinas de propósito geral, capazes de executar programas de AI, as máquinas LISP tornaram-se obsoletas e perderam espaço no mercado.

Além disso, os sistemas especialistas começaram a revelar suas limitações, mostrando dificuldades para lidar com problemas mais complexos e variados. Outro fator importante foi o encerramento do ambicioso projeto japonês, que tinha como objetivo desenvolver um sistema capaz de compreender e gerar linguagem natural, mas não alcançou seus objetivos.

Por fim, os cortes de financiamento por parte do governo dos Estados Unidos da América também contribuíram para esse período de estagnação, especialmente em decorrência das promessas feitas durante o segundo verão da AI que não se concretizaram.

Terceiro Verão

O terceiro verão da AI, também conhecido às vezes como primavera da AI, ocorreu do início dos anos 2000 até 2006. Esse período foi marcado pelo avanço no desenvolvimento de métodos estatísticos e de aprendizado de máquina. Entre os algoritmos que ganharam destaque estão as Máquinas de Vetores de Suporte (do inglês, *Support Vector Machine* - SVM), redes neurais artificiais com poucas camadas devido às limitações de hardware da época, árvores de decisão bayesianas e aprendizado por reforço.

Além disso, a área de AI passou a aderir firmemente a métodos científicos rigorosos, nos quais as hipóteses precisam ser submetidas a experimentos controlados e os resultados são analisados estatisticamente para verificar sua relevância.

Esse período também foi marcado por eventos importantes que chamaram a atenção do mundo, como a vitória do sistema de AI *Deep Blue* sobre o então campeão mundial de xadrez, Garry Kasparov. Houve um retorno significativo dos investimentos em AI por parte do Japão e da Europa, bem como uma revisão e aprofundamento dos conceitos relacionados às redes neurais convolucionais.

Terceiro Inverno

O terceiro inverno da AI, diferente dos anteriores que foram longos e dramáticos, ocorreu apenas entre 2006 e 2012. Alguns dos fatores que desencadearam esse período foram as limitações dos métodos tradicionais de aprendizado de máquina, que atingiram um platô no desempenho dos sistemas. Além disso, havia uma escassez de dados e de poder computacional disponível, o que dificultava a aplicação prática da AI no mundo real. Os sistemas de AI da época não apresentavam generalização nem robustez suficientes para enfrentar problemas variados.

Esse inverno chegou ao fim com o advento de computadores mais poderosos e o uso intensivo de GPUs, que permitiram o treinamento eficiente de redes neurais profundas, resultando em melhorias significativas no desempenho desses sistemas.

Quarto Verão

O quarto verão da AI ocorre de 2012 até os dias de hoje. Esse período tem sido marcado por grandes avanços nas áreas de compreensão e geração de linguagem natural, visão computacional e aprendizado profundo.

Um marco importante foi a vitória do sistema de AI da IBM, o Watson, sobre os campeões do programa de perguntas e respostas americano *Jeopardy!*. Esse feito demonstrou avanços significativos no campo da linguagem natural. Outro evento marcante foi a vitória da AlexNet, uma rede neural profunda, no concurso *ImageNet Large Scale Visual Recognition Challenge* (ILSVRC). A margem de vitória foi tão expressiva que provou a eficácia do aprendizado profundo na tarefa de visão computacional,

marcando o renascimento das redes neurais profundas.

Com os constantes avanços nas GPUs e o surgimento das TPUs, tornou-se possível treinar redes neurais ainda maiores e mais complexas, o que levou a melhorias expressivas de desempenho em diversas áreas. Além disso, surgiram modelos baseados em arquiteturas modernas que mesclam conceitos clássicos com novas abordagens, elevando significativamente o desempenho em tarefas como compreensão e geração de linguagem natural.

Durante os períodos de entusiasmos e decepções, diversos sistemas e algoritmos foram concebidos, cada um com um comportamento específico que, ao longo dos anos, passou a ser associado a determinadas categorias. Essas categorias, de forma natural, foram vinculadas a tipos de técnicas e comportamentos desses sistemas ou algoritmos. As duas principais categorias são: Aprendizado de Máquina (do inglês *Machine Learning* – ML) e Aprendizado Profundo (do inglês *Deep Learning* – DL).

Aprendizado de Máquina

O conceito de ML foi introduzido no primeiro verão da AI por Samuel, definindo ML como um campo de estudo que dá aos computadores a habilidade de aprender sem serem explicitamente programados, atualmente há definições semelhantes, como de ML é a categoria de um conjunto de técnicas associadas a sistemas e algoritmos que aprimoram seu desempenho por meio do aprendizado a partir de experiências anteriores, utilizando métodos computacionais (Zhou, 2021). Os sistemas e algoritmos que compõem essa categoria são chamados de modelos. Esses modelos buscam representar matematicamente inferências sobre uma determinada série de características.

Existem dois processos principais empregados em modelos de ML: o processo de aprendizado e o processo de avaliação. O processo de aprendizado, comumente utilizado em modelos de ML, é geralmente dividido em duas etapas: treinamento e teste. Já o processo de avaliação, por sua vez, é composto por uma única etapa, onde o desempenho do modelo é mensurado com base em métricas específicas.

Antes dos processos de aprendizado e avaliação, é necessário realizar a obtenção e preparação dos dados que serão utilizados em ambas as etapas. Esses dados devem ser coletados por meio de observações de um determinado fenômeno ou evento. Cada observação é descrita por um conjunto de atributos ou variáveis, denominados características (*features*), que servem para explicar ou prever um determinado resultado, chamado de predição (*target* ou *label*).

Obtenção dos Dados

O processo de obtenção dos dados é de extrema importância e demanda um esforço significativo, pois a qualidade e a relevância dos dados impactam diretamente a capacidade do modelo de ML de aprender padrões e fazer previsões precisas. Por

meio desses dados, o modelo será capaz de associar determinadas características às saídas correspondentes, possibilitando assim a generalização para novos dados.

Além da obtenção dos dados, esse processo inclui etapas como seleção, limpeza, transformação e validação dos dados. Um processo mal conduzido pode introduzir viés, ruídos ou distorções que comprometem a performance do modelo em cenários do mundo real. Destacam-se algumas ações a serem observadas durante a obtenção e preparação dos dados:

- **Representatividade da amostra:** Os dados coletados devem representar o problema real e contemplar todos os cenários relevantes, evitando amostras enviesadas ou desbalanceadas.
- **Qualidade e integridade dos dados:** Identificar e corrigir dados ausentes, duplicados, inconsistentes ou ruidosos, assegurando que os dados utilizados sejam confiáveis.
- **Documentação da Origem dos Dados:** É necessário descrever a origem dos dados, a metodologia empregada para a coleta e as eventuais transformações aplicadas, como algum tipo de pré-processamento.
- **Padronização e normalização:** Normalizar ou padronizar os dados, especialmente quando os atributos apresentam escalas diferentes que possam interferir no aprendizado, é fundamental.
- **Equidade e eliminação de vieses:** Avaliar a presença de vieses históricos ou estruturais nos dados que possam comprometer a imparcialidade das previsões.
- **Conformidade ética e legal:** É fundamental respeitar as diretrizes de privacidade e proteção de dados, especialmente ao lidar com dados sensíveis.

A adoção dessas ações permite que os modelos de ML sejam robustos, reduzindo os riscos associados à má qualidade dos dados.

O ML possui quatro principais metodologias para o processo de aprendizado e avaliação conforme descrito por N; Gupta: Aprendizado Supervisionado, Aprendizado Semi-Supervisionado, Aprendizado Não Supervisionado e Aprendizado por Reforço.

Aprendizado Supervisionado

Conforme descrito por Morales; Escalante, a grande maioria dos modelos de ML que, dada uma entrada, produzem uma saída na forma de um valor-alvo, seja uma classe, ou valores contínuos, são conhecidos como pertencentes à metodologia de aprendizado supervisionado. O objetivo do aprendizado supervisionado é encontrar

uma função que mapeie entradas em saídas, de forma que essa função seja capaz de generalizar além do conjunto de amostras.

Comumente, o conjunto de amostras é dividido em partições de treino e validação. A função é aprendida a partir dessas partições, com o objetivo de realizar previsões corretas de rótulos em novas instâncias amostradas da mesma distribuição subjacente do conjunto de amostras.

O conjunto de teste é formado por instâncias provenientes da mesma distribuição que o conjunto de amostras, mas que não estão presentes nesse conjunto de amostras. Assim, o conjunto de teste é utilizado para avaliar a capacidade de generalização da função aprendida.

Alguns algoritmos que pertencem a essa categoria incluem: Árvore de Decisão, Máquinas de Vetores de Suporte (do inglês *Support Vector Machines* - SVM), classificadores baseados em instâncias, entre outros.

Aprendizado Não Supervisionado

Os métodos que segmentam automaticamente os dados são conhecidos como aprendizado não supervisionado. A ideia central por trás dessa abordagem é identificar, de forma automática, grupos ou estruturas nos dados, de modo que os elementos pertencentes ao mesmo grupo sejam semelhantes entre si segundo certos critérios, e diferentes, segundo esses mesmos critérios, dos elementos de outros grupos (Moraes; Escalante, 2022).

Alguns exemplos de algoritmos utilizados nesse contexto incluem: K-Means, DBSCAN, Hierarchical Clustering, entre outros. Métricas de similaridade frequentemente utilizadas nesses algoritmos incluem as distâncias Euclidiana, Manhattan e Minkowski.

Aprendizado Semi-Supervisionado

O aprendizado semi-supervisionado situa-se entre o aprendizado supervisionado e o não supervisionado, utilizando tanto dados rotulados quanto não rotulados para o treinamento do modelo. Essa abordagem é especialmente útil quando a obtenção de rótulos é custosa ou limitada, mas há uma grande quantidade de dados não rotulados disponíveis.

A ideia central do aprendizado semi-supervisionado é aproveitar as informações contidas nos dados não rotulados para melhorar a generalização do modelo, combinando-as com os poucos dados rotulados para guiar o processo de aprendizado. Assim, o modelo pode aprender representações mais robustas e performar melhor em tarefas de classificação ou regressão (Van engelen; Hoos, 2020).

Alguns exemplos de técnicas de aprendizado semi-supervisionado incluem: auto-encoders, métodos baseados em grafos, co-training, e algoritmos de propagação de rótulos.

Aprendizado por Reforço

O aprendizado de um modelo pode ocorrer por meio da interação com o ambiente. Essa interação fornece informações sobre as consequências das ações tomadas, as quais auxiliam na obtenção de um determinado objetivo. O aprendizado por reforço utiliza essas interações com o ambiente para aprender como se comportar de forma eficiente (Morales; Escalante, 2022).

O objetivo do aprendizado por reforço é aprender a mapear estados para ações, de modo a maximizar a recompensa esperada. As recompensas geralmente são fornecidas em estados finais ou intermediários. O modelo busca adquirir experiências úteis sobre estados, ações, transições de estado e recompensas, a fim de aprender a se comportar de maneira ótima.

Alguns algoritmos que utilizam essa metodologia incluem: Q-Learning, SARSA (do inglês *State-Action-Reward-State-Action*), métodos de Monte Carlo, Programação Dinâmica, entre outros.

Aprendizado Profundo

O aprendizado profundo (do inglês *Deep Learning* - DL) é uma subcategoria do aprendizado de máquina baseada em redes neurais artificiais com múltiplas camadas (camadas profundas), capazes de aprender representações complexas e não lineares dos dados de forma automática (Pouyanfar; Sadiq; Yan; Tian; Tao; Reyes; Shyu; Chen; Iyengar, 2018). Diferentemente dos métodos tradicionais, que exigem maior intervenção humana na definição de características, o DL extrai e transforma essas características internamente por meio de várias camadas de unidades chamadas neurônios, como o perceptron (Pouyanfar; Sadiq; Yan; Tian; Tao; Reyes; Shyu; Chen; Iyengar, 2018). Essa capacidade o torna especialmente eficaz em tarefas complexas como processamento de linguagem natural. Essa sub-categoria tem ganhado destaque devido aos grandes volumes de dados, poder computacional e melhorias nos algoritmos.

Podemos sintetizar que, em diversos momentos da história, sempre houve um tema central associado ao sucesso ou ao fracasso da área de AI: a capacidade das máquinas de compreenderem e gerarem linguagem natural.

2.2 Linguagem

A linguagem é um objeto de estudo científico profundo que integra as ciências biológicas. Pode ser definida como um método puramente humano e não instintivo de comunicar ideias, emoções e desejos por meio de símbolos produzidos voluntariamente, conforme (Sapir, 1929), sendo a espinha dorsal da comunicação humana. A língua é sistemática, constituída por um conjunto (finito ou infinito) de símbolos que podem ser reutilizados a partir de possíveis combinações definidas por regras (Chomsky, 2002).

Pode ser compreendida como um supersistema formado por sistemas menores, tais como: fonológico, gramatical (morfologia, sintaxe, léxico), semântico e contextual, os quais podem ser denominados como camadas ou estratos da linguagem (Martinet et al., 1978). Na Figura 1, podemos observar a hierarquia dos estratos da linguagem.

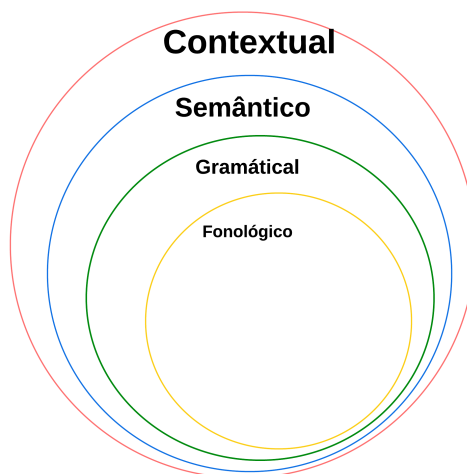


Figura 1 – Estrato da Linguagem.

Cada sistema possui atribuições que auxiliam na compreensão, interpretação e geração da linguagem, o que possibilita a comunicação entre os humanos. Conforme Martinet et al., o sistema fonológico é responsável pela organização das pausas, entonações e aspectos prosódicos da fala. O sistema semântico, por sua vez, atribui significados às frases, permitindo a distinção entre diferentes conteúdos expressos por uma mesma combinação lexical. Já o sistema contextual relaciona a estrutura linguística à situação comunicativa, determinando, por exemplo, se um enunciado deve ser interpretado como pergunta, afirmação ou ordem.

O sistema gramatical atua como uma ponte entre todos os sistemas, organizando o significado em estruturas formais que podem ser realizadas fonologicamente e interpretadas contextualmente. A gramática estabelece a ordem das palavras, as relações sintáticas e as flexões morfológicas, possibilitando que o conteúdo semântico seja codificado de forma clara e que a prosódia e a entonação sejam aplicadas adequadamente. Esse sistema garante que os níveis fonológico, semântico e contextual estejam articulados, permitindo que a comunicação seja eficiente e compreensível para todos.

Atualmente, vivemos em um mundo interconectado. Com o advento da internet, a linguagem passou a se propagar de forma abundante, rompendo barreiras até então impostas pela distância geográfica entre os povos. Essa conectividade permitiu que a pluralidade de ideias e culturas fosse amplamente disseminada, seja por meio das redes sociais, portais de notícias, fóruns ou outras plataformas digitais.

No entanto, a ausência do conhecimento sobre a língua de origem de quem comunica pode levar a diferentes interpretações, algumas positivas, outras nem tanto. Isso evidencia a complexidade da linguagem, que envolve múltiplos estratos, desde

o léxico e a sintaxe até aspectos semânticos, pragmáticos e culturais. Compreender esses níveis é fundamental para interpretar corretamente o conteúdo textual gerado em através das plataformas online.

Nesse cenário, surgem desafios significativos, sobretudo relacionados à forma como processamos, interpretamos e extraímos sentido de dados textuais. É nesse contexto que a área da computação ganha destaque, especialmente por meio de áreas, que buscam desenvolver métodos e tecnologias capazes de analisar, compreender e gerar linguagem humana de forma automatizada, tornando possível lidar a diversidade textual presentes na comunicação.

Com o atual estado da arte, diversas subáreas da IA têm ganhado relevância no meio acadêmico, entre elas destaca-se o Processamento de Linguagem Natural (do inglês *Natural Language Processing* – NLP), que tem se mostrado fundamental para a evolução dos sistemas de AI.

2.3 Processamento de Linguagem Natural

O NLP, é a área voltada ao tratamento e à interpretação da linguagem humana em formato textual (Joshi; Dabre; Kanojia; Li; Zhan; Haffari; Dippold, 2025). Trata-se de uma intersecção entre os campos da linguística e da inteligência artificial, buscando permitir que máquinas compreendam, processem e gerem linguagem de maneira significativa. O NLP abrange duas tarefas principais: a compreensão da linguagem natural (do inglês *Natural Language Understanding* – NLU) e a geração da linguagem natural (do inglês *Natural Language Generation* – NLG). A compreensão pode ser realizada por meio de subtarefas, como a análise de sentimentos, cujo objetivo é inferir, a partir do texto, qual sentimento é expresso (positivo, negativo ou neutro). Já a geração textual ocorre, em grande parte, por meio da predição da próxima palavra, abordagem amplamente adotada nas arquiteturas modernas de sistemas de NLP. A Figura 2 apresenta algumas das tarefas associadas a cada uma dessas categorias: NLU e NLG.

Para que ambas as categorias, NLU e NLG, possam apresentar bons desempenhos, é necessário que os sistemas de NLP compreendam os diferentes estratos da linguagem, como, por exemplo, a morfologia do idioma no qual o texto está sendo expresso (Joshi; Dabre; Kanojia; Li; Zhan; Haffari; Dippold, 2025). Cada idioma possui singularidades morfológicas, de modo que uma mesma palavra pode assumir interpretações diferentes conforme o contexto. Por exemplo, na frase “Preciso ir ao banco para sacar dinheiro”, banco é objeto de “ir”, relacionado a uma ação financeira; já em “Ele se sentou no banco do parque”, banco é objeto de “sentou”, indicando um assento físico. Nesse sentido, as tarefas voltadas à compreensão dos estratos da linguagem têm ganhado ênfase no campo do NLP, servindo como fundamento para avanços em

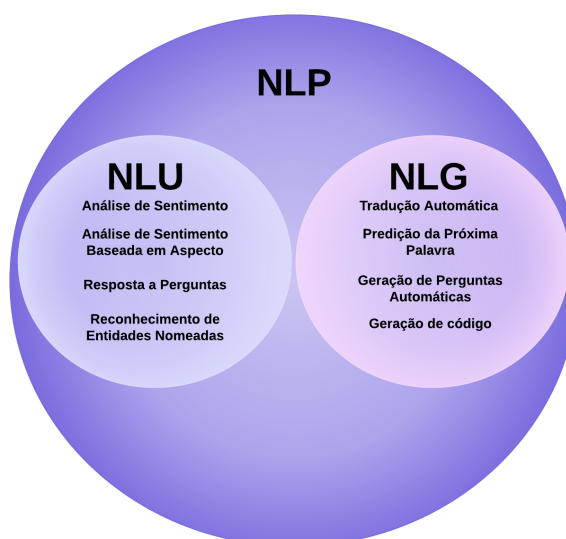


Figura 2 – Diagrama Tarefas NLU e NLG.

aplicações mais complexas. Dentre elas, destaca-se a Análise Sintática (do inglês *Syntactic Analysis* - SA).

2.4 Análise Sintática

A SA é especialmente relevante, pois permite que os sistemas compreendam tanto a estrutura gramatical singular da língua-alvo quanto os aspectos universais da sintaxe, identificando palavras que se relacionam entre si (Mel'cuk et al., 1988), ou seja, como as palavras se organizam em sintagmas e como esses sintagmas se relacionam para formar sentenças coerentes. Durante a análise, o sistema identifica diferentes tipos de sintagmas, como sintagmas nominais, verbais e adjetivais e estabelece suas funções dentro da oração (Pacheco; Guaranha, 2022). Por exemplo, na frase “O menino corre rapidamente no parque”, a análise sintática identifica o sintagma nominal “O menino” como sujeito, o sintagma verbal “corre rapidamente” como predicado e o sintagma preposicional “no parque” como adjunto adverbial de lugar.

Essa identificação detalhada dos sintagmas possibilita uma melhor captura do contexto do texto, o que auxilia na desambiguação e proporciona uma compreensão mais precisa do sentimento expresso nas orações (Otero, 2008), contribuindo significativamente para as áreas de NLU e NLG.

Além disso, a tarefa de análise sintática desempenha um papel fundamental durante o pré-processamento de tarefas mais complexas, como a Análise de Sentimento Baseada em Aspecto (ABSA) (Yuan; Wang; Yu; Zhang, 2023). A ABSA tem como objetivo identificar sentimentos expressos em relação a aspectos específicos de uma entidade em um texto. Diferentemente da análise de sentimento tradicional, que classifica o sentimento global de uma sentença ou documento, a ABSA busca associar

opiniões a características particulares.

Nesse contexto, a SA é essencial para identificar corretamente as relações gramaticais entre os elementos da frase, como sujeito, objeto e modificadores. Essas relações ajudam os sistemas a determinar a que aspecto um determinado sentimento está vinculado.

Por exemplo, na frase “A câmera do celular é excelente, mas a bateria é fraca”, a análise sintática permite reconhecer que o adjetivo “excelente” qualifica “câmera”, enquanto “fraca” refere-se à “bateria”. Sem esse mapeamento sintático, um sistema poderia interpretar erroneamente os sentimentos expressos, comprometendo a precisão da análise.

A SA pode ser realizada de duas formas distintas: uma a nível de constituinte, denominada Análise de Constituência (do inglês, *Constituency Parsing* – CP), e outra a nível de dependência, denominada Análise de Dependências (do inglês, *Dependency Parsing* – DP).

2.4.1 Análise de Constituência

A constituição é uma unidade definida pela sua função no nível superior da escala de ordens. Essa escala é composta, do mais alto ao mais baixo, por: oração, grupo/frase, palavra e morfema (Miller, 1992). O nível mais baixo da escala é o morfema, e o mais alto é a oração. Na Figura 3 é possível visualizar a escala de ordens.

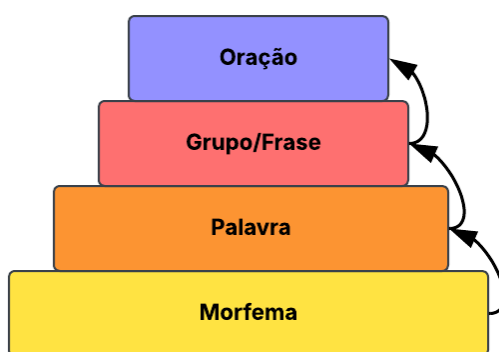


Figura 3 – Escala de Ordens.

De acordo com Enfield, os morfemas correspondem às menores unidades de significado de uma palavra; por exemplo, na palavra *brinca*, os morfemas são “brinc” e “a”. A palavra resulta da junção desses morfemas, como em “brinca”. Já o grupo ou frase é formado por conjuntos de palavras, como “A menina” (sujeito) ou “brinca no parque” (predicado), enquanto a oração representa a sentença completa, por exemplo: “A menina brinca no parque”. Na Figura 4, é possível visualizar essa estruturação da escala de ordens.

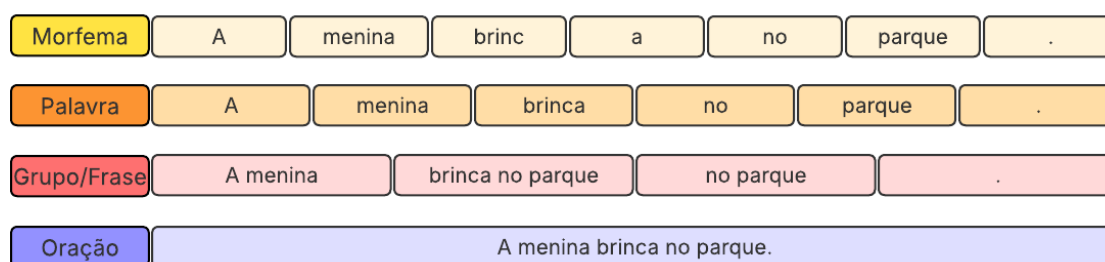


Figura 4 – Exemplo de estruturação da escala de ordens.

Além das unidades presentes na escala de ordens, que abrange apenas as unidades gramaticais, na linguagem escrita existe uma unidade grafológica denominada sentença. A sentença segue convenções prototípicas da língua escrita (Alkohlani; Alguzo; Sayed; Shenouda, 2025), isto é, inicia-se com letra maiúscula e termina com um sinal de pontuação que indica sua finalização, podendo ser esse um ponto final, um ponto de interrogação ou um ponto de exclamação.

A sentença não está contida na escala de hierarquia linguística, exatamente por seguir critérios gráficos e convencionais da escrita, como a presença de letra maiúscula no início e o ponto final. Embora semelhantes, oração e sentença diferem quanto aos critérios que as definem: enquanto a oração é caracterizada por aspectos estruturais e funcionais da língua, a sentença é determinada por convenções da linguagem escrita.

Uma sentença pode conter uma única oração, como em “A menina brinca no parque”, ou mais de uma oração, no caso de um período composto, como em “A menina brinca no parque e o menino joga bola na rua”.

A análise de constituintes busca extrair as relações entre os constituintes de uma sentença, que são representadas por meio de uma hierarquia de classes sintáticas e de seus constituintes internos (Fernández-gonzález; Gómez-rodríguez, 2022). A relação é do tipo parte-todo, entre um constituinte menor e outro situado em nível superior na escala de ordens. As classes sintáticas incluem, por exemplo, Sintagma Nominal (SN) e Sintagma Verbal (SV). Seus constituintes internos, que podem ser entendidos como “subclasses”, incluem elementos como núcleo, determinante ou adjunto, dependendo do tipo de sintagma. Por exemplo, no Sintagma Nominal “A menina”, menina é o núcleo e “A” é o determinante.

Entre 1993 e 1994, o projeto Corporas Textuais capturou dados textuais em alemão, francês e italiano, construindo um corpus denominado *Penn Treebank*, no formato *treebanks* (Natural language processing, 2025). Um corpus é definido como o conjunto de documentos que serve de base para a descrição ou o estudo de um fenômeno. Os *treebanks* são corpora linguísticos anotados sintaticamente com estruturas em árvore que representam a análise sintática das sentenças.

O projeto desenvolveu a representação para textos e suas marcações, juntamente com uma linguagem de consulta e um sistema de acesso ao corpus para exploração linguística do material textual. Os componentes da ferramenta desenvolvida são específicos para cada idioma e, ao mesmo tempo, gerais. Eles abrangem desde a análise morfossintática até a análise parcial, passando por medidas como informação mútua, *t-score*, extração e agrupamento de colocações, até marcação baseada em HMM e marcação de n-gramas. Na Tabela 1, são elencados os principais componentes que compõem o *Penn Treebank*.

Tipo de anotação	Descrição / função	Tamanho / observações
POS tagging	Etiquetagem de palavras com classes gramaticais (Part-of-Speech).	Contém 36 POS tags principais e cerca de 12 símbolos adicionais.
Estrutura sintática (constituente)	Árvores de constituintes com sintagmas (NP, VP, etc.), baseada em bracketing.	3 milhões de palavras com análise sintática.
Predicado-argumento	Marca quem faz o quê nos verbos (estrutura semântica).	2 milhões de palavras com marcação semântica.
Texto falado (speech)	Transcrição de fala espontânea ou discursos com anotações específicas.	1.6 milhões de palavras com marcações de fala.

Tabela 1 – Principais componentes do Penn Treebank

O trabalho desenvolvido pelo projeto serviu como base para o padrão das estruturas que são representadas na análise de constituição. A análise de constituição é usualmente representada de duas formas, conforme o padrão definido pelo projeto (Natural language processing, 2025): com colchetes, conforme apresentado na Figura 5, ou por meio de uma estrutura em árvore, conforme mostrado na Figura 6.

[S [NP [DET A][NOUN menina]][VP [VERB brinca][PP [ADP no][NP [NOUN parque]]]][PUNCT .]]

Figura 5 – Exemplo de estruturação com colchetes.

A análise de constituição tem como ponto de partida a unidade maior, a sentença, representada, prototipicamente, pela notação S (do inglês *sentence*). Se uma sentença tem pontuação final, o sinal de pontuação recebe a notação PUNCT (do inglês, *punctuation*) e é anotado no mesmo nível hierárquico de S. De S, derivam constituintes

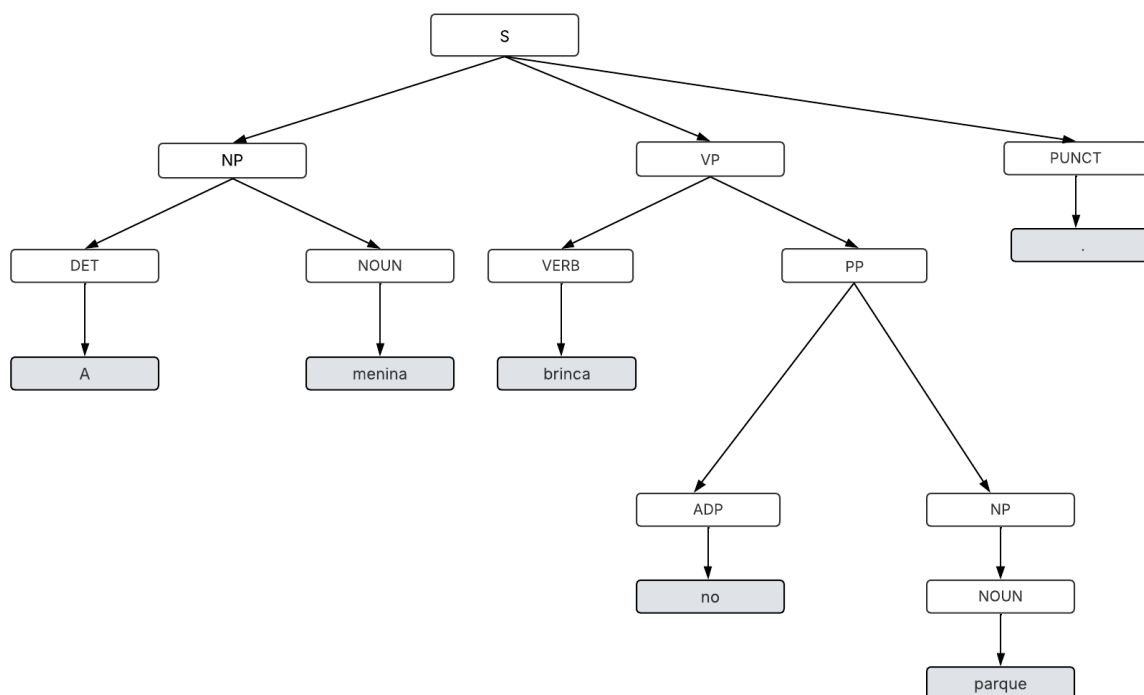


Figura 6 – Exemplo de estruturação com árvore.

cujas estruturas realizam tipicamente as funções sintáticas no nível da oração: sujeito, geralmente realizada por um constituinte com estrutura de sintagma nominal, que recebe a notação NP (do inglês *noun phrase*); e predicado, geralmente realizada por um constituinte com estrutura de sintagma verbal, com a notação VP (do inglês, *verb phrase*), podendo ter objetos com estrutura prototípica de sintagma nominal (NP). Há também frases preposicionais, anotadas como PP (do inglês, *prepositional phrase*). Nos níveis seguintes, cada constituinte é subdividido em subconstituintes até chegarmos às palavras individuais, as quais recebem a notação de sua classe: V para verbo, N para substantivo, DET para determinantes como artigos e pronomes demonstrativos, ADJ para adjetivo, ADV para advérbio, e P para preposição.

2.4.2 Análise de Dependências

Na análise de dependência, a palavra, que também pode ser denominada token, é a unidade fundamental responsável por estabelecer relações de dependência com outras palavras na sentença. O projeto *Universal Dependencies* (UD) desenvolveu anotações em *treebanks* com consistência interlinguística para diversas línguas, com o objetivo de facilitar o desenvolvimento de analisadores sintáticos multilíngues (Dependencies, 2025).

O esquema de anotação desse projeto se baseou na evolução das dependências universais de Stanford (De marneffe; Dozat; Silveira; Haverinen; Ginter; Nivre; Manning, 2014), nas tags universais das classes gramaticais do Google (Petrov; Das;

Mcdonald, 2011) e na Interlingua Interset para conjuntos de tags morfossintáticas. A ideia do projeto foi disponibilizar um conjunto universal de categorias e diretrizes que facilite a anotação consistente de construções semelhantes entre línguas, permitindo extensões específicas para cada língua quando necessário (Dependencies, 2025).

Na Tabela 2 é possível visualizar o formato em que esse conjunto de dados, no formato de um *treebank*, está estruturado.

ID	FORM	LEMMA	UPOS	XPOS	FEATS	HEAD	DEPREL	DEPS	MISC
1	O	o	DET	—	Definite=Def Gender=Masc Number=Sing PronType=Art	2	det	2:det	—
2	canal	canal	NOUN	—	Gender=Masc Number=Sing	3	nsubj	3:nsubj	—
3	terá	ter	VERB	—	Mood=Ind Number=Sing Person=3 Tense=Fut VerbForm=Fin	0	root	0:root	—
4	o	o	DET	—	Definite=Def Gender=Masc Number=Sing PronType=Art	5	det	5:det	—
5	conteúdo	conteúdo	NOUN	—	Gender=Masc Number=Sing	3	obj	3:obj 6:nsubj:pass	—
6	reformulado	reformular	VERB	—	Gender=Masc Number=Sing VerbForm=Part Voice=Pass	3	xcomp	3:xcomp	SpaceAfter=No
7	.	.	PUNCT	—	—	3	punct	3:punct	—

Tabela 2 – Exemplo de sentença anotada no formato UD.

Cada coluna do formato UD representa um aspecto específico da anotação sintática e morfológica das palavras na sentença. A coluna **ID** indica a posição da palavra na frase, geralmente numerada sequencialmente. **FORM** corresponde à forma da palavra conforme aparece no texto. **LEMMA** apresenta a forma canônica ou dicionarizada da palavra, útil para análise morfológica e semântica. **UPOS** é a etiqueta universal da classe gramatical (por exemplo, substantivo, verbo, adjetivo), enquanto **XPOS** traz uma etiqueta mais detalhada, específica para cada idioma. A coluna **FEATS** registra atributos morfológicos adicionais, como gênero, número, tempo verbal, entre outros. **HEAD** indica o índice da palavra governante da palavra atual na estrutura de dependência, ou zero se for a raiz da sentença. **DEPREL** especifica o tipo de relação sintática entre a palavra e seu governante (por exemplo, sujeito, objeto, modificador). A coluna **DEPS** pode conter anotações de dependências secundárias ou alternativas, e **MISC** reúne informações adicionais que não se enquadram nas outras categorias, como anotações específicas do corpus ou detalhes adicionais.

O projeto UD definiu o padrão para a forma como as relações entre as unidades de palavras são representadas. Essas relações ocorrem por meio de conexões uni-

direcionais, partindo de uma palavra central chamada raiz (ROOT). Algumas palavras funcionam como núcleos (governadores) e podem ter uma ou mais palavras como dependentes.

Além disso, o projeto UD estabeleceu as classes de dependência utilizadas na área da análise de dependência, bem como a estrutura a ser adotada pelos sistemas de anotação sintática. As classes de dependência estão explanadas no apêndice .8.

As classes de dependências refletem as funções sintáticas (como sujeito, predicado, objeto direto, objeto indireto, adjunto adverbial, etc.) e podem também ser associadas a papéis semânticos (como agente, paciente, instrumento, etc.).

Exemplo: Na frase “A menina brinca no parque.” a análise de dependências pode ser descrita da seguinte forma:

- **brinca** é a raiz (ROOT), desempenha a função sintática de *predicado*, e seu papel semântico é o de *ação*.
- **menina** exerce a função sintática de *sujeito* e tem o papel semântico de *agente* (quem pratica a ação).
- **A** exerce a função sintática de *determinante* e não possui papel semântico autônomo.
- **parque** exerce a função sintática de *objeto oblíquo* (complemento de preposição) e tem o papel semântico de *local*.
- **no** exerce a função sintática de *preposição* (marcador de caso) e não possui papel semântico próprio, mas introduz o constituinte locativo.
- **.** (ponto final) exerce a função sintática de *pontuação* e não possui papel semântico.

As dependências sintáticas podem ser representadas por uma estrutura denominada árvore de dependência sintática, como ilustrado na Figura 7. Cada nó da árvore representa uma palavra da frase, enquanto cada aresta (ou arco) indica uma relação de dependência, apontando do governante (head) para o dependente. Além disso, o nó raiz (ROOT), que normalmente corresponde ao verbo principal da oração.

As dependências sintáticas também podem ser representadas por meio de parênteses, seguindo a estrutura classe (governante, dependente). Essa forma de representação está ilustrada na Figura 8.

A árvore de dependência sintática pode conter informações adicionais, como a classe gramatical da palavra ou seus *features*. Esses *features* incluem gênero, número, tempo verbal, entre outros atributos morfossintáticos. Essa estrutura pode ser observada na Figura 9.

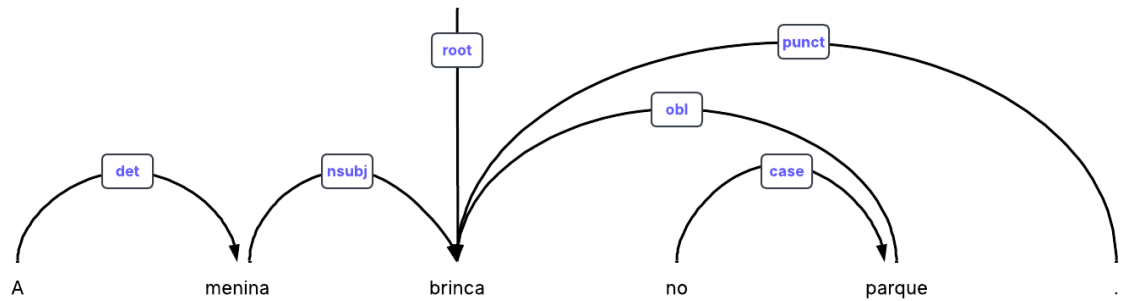


Figura 7 – Exemplo de uma Árvore de Dependência Sintática.

- 1 root(ROOT, brinca)
- 2 nsubj(brinca, menina)
- 3 det(menina, A)
- 4 obl(brinca, parque)
- 5 case(parque, no)
- 6 punct(brinca, .)

Figura 8 – Representação parentética de sentença

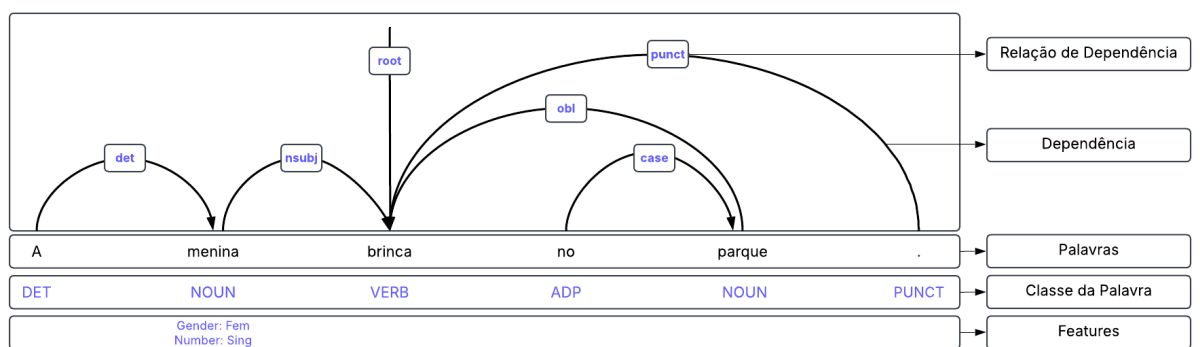


Figura 9 – Exemplo de uma Árvore de Dependência Sintática.

O projeto UD com base no estudo (Petrov; Das; McDonald, 2011) definiu 17 classes gramaticais principais, representadas pelas tags do UPOS. Abaixo estão listadas essas tags:

- **ADJ**: Adjetivo (Exemplo: “grande”, “azul”)
- **ADP**: Adposição (preposições ou pós-posições, Exemplo: “de”, “em”)
- **ADV**: Advérbio (Exemplo: “rapidamente”, “muito”)
- **AUX**: Auxiliar (verbos auxiliares, Exemplo: “ser”, “ter”)
- **CCONJ**: Conjunção coordenativa (Exemplo: “e”, “ou”)
- **DET**: Determinador (artigos, demonstrativos, Exemplo: “o”, “este”)
- **INTJ**: Interjeição (Exemplo: “ah!”, “uau”)
- **NOUN**: Substantivo (Exemplo: “casa”, “livro”)
- **NUM**: Numeral (Exemplo: “dois”, “2023”)
- **PART**: Partícula (Exemplo: infinitival “de”, negação “não”)
- **PRON**: Pronome (Exemplo: “eu”, “isso”)
- **PROPN**: Substantivo próprio (Exemplo: “Brasil”, “João”)
- **PUNCT**: Pontuação (Exemplo: “.”, “,”, “!”)
- **SCONJ**: Conjunção subordinativa (Exemplo: “que”, “se”)
- **SYM**: Símbolo (Exemplo: “\$”, “%”)
- **VERB**: Verbo (Exemplo: “correr”, “ser”)
- **X**: Outros (etiqueta residual para elementos não classificados)

Porém, também existem as tags XPOS. A tag XPOS é específica de cada língua e varia conforme o idioma analisado, mapeando subcategorias do UPOS. Ela não possui uma lista universal fixa, mas é utilizada em paralelo nas anotações do UD. Abaixo, seguem exemplos genéricos para o português, com base no esquema UD-PT:

- **N**: Substantivo comum (mapeia para **NOUN**)
- **PROPN**: Substantivo próprio (mapeia para **PROPN**)
- **VFIN**: Verbo finito (mapeia para **VERB**)

- **ADJ**: Adjetivo (mapeia para **ADJ**)
- **ADV**: Advérbio (mapeia para **ADV**)

De forma geral, o UPOS pertence ao nível universal, interlinguístico, enquanto o XPOS pertence ao nível específico da língua.

2.4.3 Diferença entre Análise Constituinte e Análise de Dependência

As análises de constituintes e de dependências são duas abordagens fundamentais para representar a estrutura sintática das frases na linguística e no processamento de linguagem natural (Gu; Hou; Wang; Duan; Li, 2023).

Ambas as abordagens podem, frequentemente, ser convertidas uma na outra em muitas frases, mas cada uma destaca aspectos diferentes da sintaxe: a análise de constituintes revela a estrutura hierárquica das frases, enquanto a análise de dependências enfatiza as funções sintáticas diretas entre as palavras (Fei; Wu; Ren; Li; Ji, 2021). A abordagem de constituintes é tradicionalmente associada à gramática gerativa, enquanto a de dependências é considerada mais flexível para línguas com ordem livre de palavras.

Dentre as duas abordagens de análise sintática, a análise de dependência destaca-se por possibilitar a compreensão do sentido gramatical expresso na sentença, o que permite aos sistemas de NLP obterem um melhor desempenho na compreensão e na geração de linguagem natural (Reddy; Täckström; Collins; Kwiatkowski; Das; Steedman; Lapata, 2016)

2.4.4 Parser

O processo responsável por implementar a AS, seja de constituinte ou de dependência, é denominada parsing, e os sistemas que executam essa tarefa são chamados de parsers. Esse processo refere-se à previsão da estrutura sintática de uma sentença de entrada sem nenhuma anotação prévia, utilizando um modelo que pode realizar a análise em termos de constituinte ou dependência (Caseli; Nunes, 2023).

Existem características distintas entre os tipos de parsing, seja de constituinte ou de dependência, que podem ser classificados como profundo ou superficial. O parsing profundo realiza uma análise exaustiva, visando obter uma compreensão completa da estrutura sintática da sentença, conforme ilustrado na Figura 9.

Já o parsing superficial oferece uma análise simplificada, com informações mínimas, de acordo com as necessidades da tarefa em questão, conforme ilustrado na Figura 5.

Nos anos 90, os parsers voltados à análise sintática eram desenvolvidos com base em conceitos formais, como as Gramáticas Livres de Contexto (GLC) e as Gramáticas Sensíveis ao Contexto (GSC). Esses modelos representaram um avanço significativo

ao fornecerem estruturas sistemáticas para descrever a sintaxe das línguas. Entretanto, ambos apresentavam limitações no tratamento de características complexas das línguas naturais.

Gramática Livre de Contexto

Conforme descrito por Johnson, o conceito de GLC fornece um modelo estatístico, simples para linguagens naturais, sendo utilizado para descrever a estrutura sintática com base em corpora anotados no formato *treebank*.

Um sistema de análise sintática de ampla cobertura pode ser obtido utilizando um analisador que busca a árvore de análise de máxima probabilidade para a cadeia de entrada, com base na gramática extraída do *treebank*. Esse modelo se caracteriza por sua simplicidade, tornando-se fácil de analisar tanto do ponto de vista teórico quanto empírico. Os modelos mais sofisticados da época eram, em geral, refinamentos do modelo PCFG.

Denomina-se “livre de contexto” porque as regras de substituição são aplicadas independentemente do contexto em que um símbolo aparece.

Os componentes do GLC são definidos como uma quádrupla:

$$G = (V, \Sigma, R, S)$$

Onde:

- V : Conjunto de símbolos não-terminais (variáveis)
Exemplo: S, NP, VP, Det, N, V
- Σ : Conjunto de símbolos terminais (palavras da linguagem)
Exemplo: $o, gato, viu, menino$
- R : Conjunto de regras de produção.

Exemplo:

$$\begin{aligned} S &\rightarrow NP VP \\ NP &\rightarrow Det N \\ VP &\rightarrow V NP \\ Det &\rightarrow o \mid a \\ N &\rightarrow gato \mid menino \\ V &\rightarrow viu \end{aligned}$$

- S : Símbolo inicial (geralmente S , de “sentença”)

Entretanto, esses parsers apresentam dificuldades específicas ao lidar com linguagens naturais, especialmente em casos de ambiguidade sintática, variações de

número ou gênero e dependências de longo alcance, como quando uma palavra no início da sentença está relacionada a outra situada no final.

Alguns dos parsers que utilizaram esse conceito são:

Parser	Tipo	Descrição
CYK (Cocke–Younger–Kasami) (Chappelier; Rajman et al., 1998)	Bottom-up	Verifica se uma cadeia pertence a uma GLC em Forma Normal de Chomsky
Earley Parser (Aycock; Horspool, 2002)	Top-down / Bottom-up (Híbrido)	Funciona para qualquer GLC, incluindo ambíguas e recursivas.

Tabela 3 – Parser GLC para Análise Sintática

Gramática Sensível ao Contexto

Conforme descrito por Dahlhaus; Warmuth, a GSC é definida como um sistema de reescrita nos quais o comprimento do lado direito de cada produção é, no mínimo, igual ao comprimento do lado esquerdo.

Uma GSC é uma gramática formal em que as regras de produção podem depender do contexto em que um símbolo aparece. Esse conceito é capaz de modelar restrições sintáticas que dependem do ambiente ao redor de um símbolo, o que a torna mais adequada para capturar fenômenos da linguagem natural, como a concordância de número e gênero, ou dependências de longo alcance.

Os componentes da Gramática Sensível ao Contexto (GSC) são definidos como uma quádrupla:

$$G = (V, \Sigma, R, S)$$

Onde:

- V : Conjunto de símbolos não-terminais
Ex: S, A, B, C
- Σ : Conjunto de símbolos terminais
Ex: a, b, c
- R : Conjunto de regras de produção sensíveis ao contexto
Exemplo de regras de produção:

$$S \rightarrow aABC$$

$$aA \rightarrow aa$$

$$AB \rightarrow CD$$

$$DC \rightarrow CE$$

$$CE \rightarrow cb$$

- S : Símbolo inicial

Os parsers baseados em GSC apresentam limitações semelhantes às dos parsers baseados em GLC. Os parsers GSC são computacionalmente custosos, têm dificuldades para lidar com múltiplas derivações e apresentam alta complexidade em sua implementação.

Alguns dos parsers que utilizaram esse conceito são:

Parser	Tipo	Descrição
A top-down error-correcting parser for a context-sensitive language (Kobayashi; Ikeda; Tanaka, 1986)	Top-down	Parser top-down com correção de erros para linguagens sensíveis ao contexto
Parser Autômatos com Pilha (Bravo; Neto, 2003)	Bottom-up	Constrói parsers sensíveis ao contexto a partir de gramáticas livres de contexto com linguagem de controle regular.

Tabela 4 – Parser GSC para Análise Sintática

Diversos outros conceitos posteriores às GLC e GSC foram propostos como extensões dessas, com o objetivo de superar as limitações dos parsers baseados nelas, especialmente na área de AS. No entanto, todas essas abordagens compartilhavam uma limitação comum: a dificuldade em lidar com a ambiguidade de certas palavras, dependendo do contexto em que eram empregadas. Por mais que as regras fossem incorporadas aos sistemas de parsing, essa questão ainda persistia.

Outros conceitos e abordagens explorados incluem:

- **Gramáticas Unificadas:** Formalismos que integram informações sintáticas, semânticas e pragmáticas por meio de unificação, facilitando representações linguísticas mais expressivas e precisas. Exemplo: LKB (Linguistic Knowledge Builder) (Copestake; Carroll; Malouf; Oepen, 1999).

- **Modelos Baseados em Regras e Heurísticas:** Abordagens híbridas que combinam regras linguísticas explicitamente definidas com heurísticas aprendidas para melhorar a desambiguação sintática, especialmente em contextos com dados limitados. Exemplo: RASP (Robust Accurate Statistical Parser) (Briscoe, 2006)

Até então, os *parsers* eram todos baseados em regras e funcionavam bem em ambientes controlados, ou seja, quando as sentenças estavam corretamente estruturadas. No entanto, com a quantidade massiva de dados textuais circulando na internet, em sua maioria sem uma estrutura bem definida, tornou-se difícil para esses *parsers* baseados em regras inferirem a estrutura sintática dos textos.

Foi apenas em 2012, com o advento do chamado “Quarto Verão” da AI, que a AS começou a apresentar avanços significativos. Contudo, o verdadeiro ponto de inflexão ocorreu em 2018, com o surgimento do conceito de Grandes Modelos de Linguagem (do inglês *Large Language Models* – LLMs). Esse novo paradigma demonstrou que muitos dos desafios que há tempos assolavam a área poderiam ser superados, permitindo uma compreensão do contexto e da ambiguidade das línguas naturais de forma muito mais robusta e flexível.

2.5 Grandes Modelos de Linguagem

Os LLMs fazem parte da categoria de sistemas e algoritmos de DL. Esses modelos são treinados com grandes volumes de dados textuais e, durante esse processo, conseguem capturar características da linguagem que lhes permitem compreender e gerar texto em linguagem natural. Sua arquitetura é baseada na arquitetura Transformers, introduzidos em 2017 por Vaswani; Shazeer; Parmar; Uszkoreit; Jones; Gomez; Kaiser; Polosukhin.

2.5.1 Transformers

A arquitetura Transformer foi inicialmente desenvolvida para tarefas de tradução automática de textos. Ela é capaz de capturar dependências de longo alcance e relações contextuais nos dados de entrada, o que possibilita uma compreensão mais profunda do conteúdo e a geração de saídas mais precisas e contextualizadas. Ao contrário das arquiteturas anteriores, que processavam os tokens individualmente, o Transformer processa a frase como um todo. Essa arquitetura é composta por dois módulos principais: o codificador e o decodificador. A Figura 10 ilustra todos os componentes da arquitetura Transformer.

Inicialmente, os dados de entrada passam pelo processo de *embedding*; simultaneamente, é adicionada a codificação posicional. Em seguida, esses vetores são processados por camadas de *Multi-Head Attention*. Após a atenção, os dados pas-

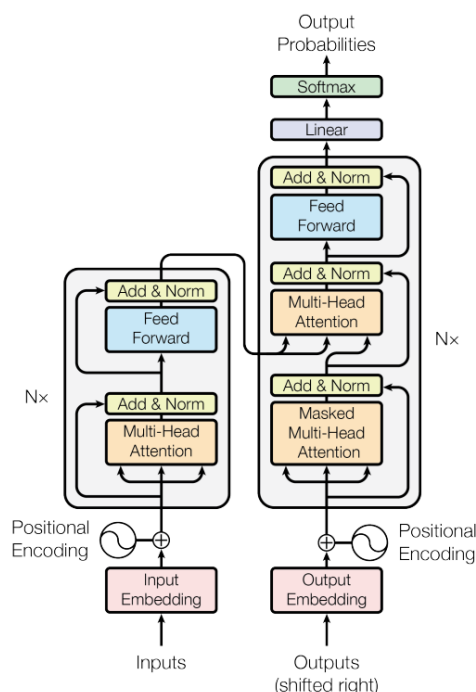


Figura 10 – Arquitetura Transformers (Vaswani; Shazeer; Parmar; Uszkoreit; Jones; Gomez; Kaiser; Polosukhin, 2017a).

sam por uma camada de normalização e uma conexão residual. Depois, seguem para uma camada *Feed-Forward* totalmente conectada, seguida novamente por normalização e conexão residual. Esse processo é repetido por várias camadas empilhadas, formando o codificador. No caso do decodificador, há uma atenção cruzada adicional com a saída do codificador antes de seguir pelas mesmas etapas internas.

Embeddings de Entrada e Saída

Os *embeddings* de entrada correspondem à representação vetorial dos tokens textuais. No contexto do artigo (Vaswani; Shazeer; Parmar; Uszkoreit; Jones; Gomez; Kaiser; Polosukhin, 2017a), esses *embeddings* representam o idioma de origem, ou seja, o idioma que o modelo deve traduzir. De forma semelhante, o idioma de destino também passa por esse mesmo processo, sendo representado por *embeddings* próprios, chamados de *embeddings* de saída. Cada token, que pode ser uma palavra, subpalavra ou morfema, é associado a um identificador único dentro do vocabulário. Esse vocabulário é construído de forma estatística por algoritmos chamados *tokenizers*, que analisam grandes volumes de texto e segmentam as unidades mais frequentes. Esses identificadores são então mapeados para vetores contínuos, que são enviados à arquitetura do modelo para processamento. A Figura 11 mostra como uma palavra é representada em um espaço vetorial contínuo.

Por exemplo, consideremos as seguintes representações vetoriais em um espaço tridimensional para algumas palavras:

$$\begin{aligned}
gato &= [1.0, 2.0, 0.5] \\
cachorro &= [1.2, 1.8, 0.4] \\
pssaro &= [4.0, 2.0, 1.5] \\
peixe &= [3.8, 2.2, 1.7]
\end{aligned}$$

Note que palavras semanticamente próximas, como “gato” e “cachorro”, possuem vetores próximos no espaço vetorial, enquanto palavras de categorias diferentes, como “gato” e “pássaro”, estão mais distantes.

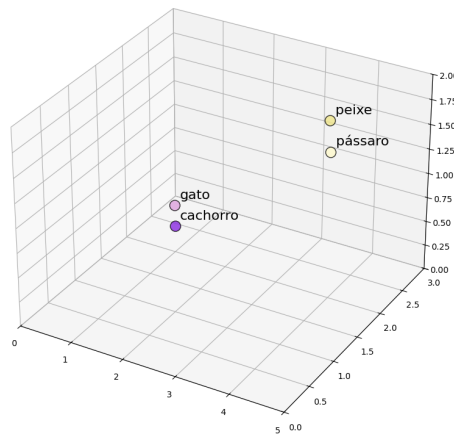


Figura 11 – Exemplo de um vetor de embeddings.

Essa representação vetorial permite capturar o contexto léxico de frases ou palavras. Essa técnica é chamada de *word embedding*, que consiste em uma forma numérica de lidar com o conceito semântico abstrato das palavras (Wang; Zhou; Jiang, 2020).

A arquitetura Transformers não utiliza conceitos de recorrência nem de convolução, o que em tese, faz perder a ordem da sequência, e para que seja possível aproveitar essa ordem, é adicionada as informações sobre a posição relativa ou absoluta de cada token na sequência.

Codificação Posicional

Para isso, foi adicionada a etapa de Codificação Posicional aos *embeddings* de entrada e saída. Essa codificação posicional possui a mesma dimensão dos *embeddings*, permitindo que eles sejam somados. A equação utilizada é dada por:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

- pos : Posição do token na sequência.
- i : Índice da dimensão dentro do vetor de codificação posicional.
- d_{model} : Dimensão total do vetor de *embedding*.
- $PE(pos, 2i)$: Valor da codificação posicional na dimensão par $2i$, calculado pela função seno.
- $PE(pos, 2i + 1)$: Valor da codificação posicional na dimensão ímpar $2i + 1$, calculado pela função cosseno.
- O denominador $10000^{2i/d_{model}}$ controla a frequência do seno e cosseno, variando exponencialmente ao longo das dimensões, criando ondas de diferentes comprimentos.

Após a etapa de Codificação Posicional, os *embeddings* são enviados ao Codificador e ao Decodificador, que então realizam o processo de associação entre os *embeddings* de entrada e os *embeddings* de saída.

Codificador

O codificador é responsável por processar a representação vetorial de entrada, resultante do processo de *embedding* e da codificação posicional. Ele é composto por uma pilha de camadas idênticas, e cada uma dessas camadas contém dois subníveis principais: um mecanismo de autoatenção com múltiplas cabeças (do inglês *multi-head self-attention*) e uma rede *feed-forward*. Entre esses subníveis, aplica-se uma etapa de normalização. As saídas geradas por cada camada mantêm a mesma dimensionalidade.

Decodificador

O decodificador é responsável por processar a representação vetorial de saída, que também é resultado do processo de *embedding* e da codificação posicional. Assim como o codificador, ele é composto por uma pilha de camadas idênticas. Cada uma dessas camadas possui três subníveis. Dois deles são semelhantes aos do codificador: o mecanismo de autoatenção com múltiplas cabeças e a rede *feed-forward*. O terceiro subnível adicional realiza a atenção com múltiplas cabeças sobre a saída do codificador, permitindo que o decodificador integre informações do contexto de entrada.

Mecanismo de Atenção

O mecanismo de atenção é um componente que permite que o modelo foque seletivamente em diferentes partes da entrada ao processar uma sequência, possibilitando compreender como cada palavra está relacionada às demais. O mecanismo de atenção possui dois componentes conforme apresentado na Figura 12 e 13, a função de atenção, *Scaled Dot-Product Attention*, e Atenção Multi-Cabeça.

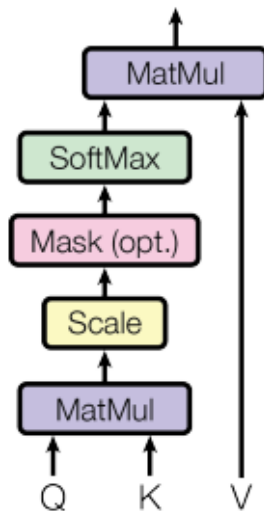


Figura 12 – Scaled Dot-Product Attention (Vaswani; Shazeer; Parmar; Uszkoreit; Jones; Gomez; Kaiser; Polosukhin, 2017a).

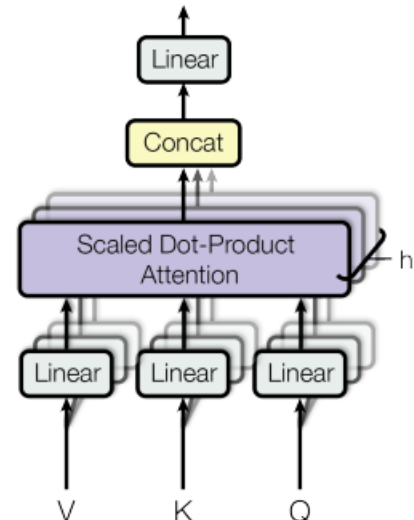


Figura 13 – Multi Head Attention (Vaswani; Shazeer; Parmar; Uszkoreit; Jones; Gomez; Kaiser; Polosukhin, 2017a).

A função de atenção pode ser descrita como um mapeamento de uma consulta (*query*) e um conjunto de pares chave-valor (*key-value*) para uma saída, onde a consulta, as chaves, os valores e as saídas são vetorizados. A saída é calculada como uma soma ponderada dos valores, em que o peso atribuído a cada valor é calculado por uma função de compatibilidade da consulta com a chave correspondente. O mecanismo de atenção pode ser descrito pela equação denominada *Scaled Dot-Product Attention*.

A entrada consiste em consultas e chaves de dimensão d_k , e valores de dimensão d_v . É calculado os produtos escalares da consulta com todas as chaves, dividindo cada um por $\sqrt{d_k}$ e aplicando a função *softmax* para obter os pesos sobre os valores.

Na prática, calcula-se a função de atenção sobre um conjunto de consultas simultaneamente, agrupadas em uma matriz Q . As chaves e os valores também são agrupados em matrizes K e V , respectivamente. A matriz de saídas é então calculada como:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

onde:

- $Q \in R^{n \times d_k}$: matriz de **consultas**.
- $K \in R^{m \times d_k}$: matriz de **chaves**,
- $V \in R^{m \times d_v}$: matriz de **valores**,
- d_k : dimensão dos vetores de chave (e também de consulta),
- QK^T : produto escalar entre as consultas e as chaves,
- $\frac{1}{\sqrt{d_k}}$: fator de escala para evitar valores muito altos na softmax,
- $\text{softmax}()$: função softmax aplicada linha a linha para gerar os pesos de atenção.

A atenção multi-cabeça permite que o modelo atenda, simultaneamente, a informações de diferentes subespaços de representação, em diferentes posições. Com apenas uma cabeça de atenção, a média dos resultados pode inibir essa capacidade.

A atenção multi-cabeça é definida como:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

onde cada cabeça é calculada por:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

As projeções são matrizes de parâmetros aprendidos com dimensões:

$$W_i^Q \in R^{d_{\text{model}} \times d_k}, \quad W_i^K \in R^{d_{\text{model}} \times d_k}, \quad W_i^V \in R^{d_{\text{model}} \times d_v}, \quad W_O \in R^{h \cdot d_v \times d_{\text{model}}}$$

Devido à dimensão reduzida de cada cabeça, o custo computacional total é semelhante ao da atenção de uma única cabeça com a dimensão total.

A atenção multi-cabeça atua de três formas principais: nas camadas de atenção codificador e decodificador, onde as consultas vêm do decodificador e as chaves e valores do codificador, permitindo que o decodificador foque em toda a sequência de entrada; nas camadas de autoatenção do codificador, em que consultas, chaves e valores vêm da mesma camada, possibilitando que cada posição atenda a todas as outras; e nas camadas de autoatenção do decodificador, que também consideram apenas posições anteriores para garantir o caráter auto-regressivo, usando uma máscara na atenção para bloquear conexões futuras.

Feed-Foward

Além das subcamadas de atenção, cada camada do codificador e decodificador contém uma rede *feed-forward* totalmente conectada, que é aplicada separadamente e de forma idêntica a cada posição.

Essa rede consiste em duas transformações lineares com uma ativação ReLU entre elas:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Embora as transformações lineares sejam as mesmas para diferentes posições, elas usam parâmetros diferentes de camada para camada.

A dimensionalidade da entrada e saída é $d_{model} = 512$, e a camada interna tem dimensionalidade $d_{ff} = 2048$. Essas camadas de entrada e saída, além da *feed-forward*, possuem parâmetros que possibilitam associar a entrada à saída.

Parâmetros

Os parâmetros são um componente fundamental das arquiteturas atuais. Atualmente, os modelos são considerados mais eficientes quanto maior o número de parâmetros que possuírem. Esses parâmetros são, normalmente, os neurônios artificiais, e as camadas de um modelo constituem as redes neurais artificiais. Um dos neurônios artificiais mais amplamente utilizados nas arquiteturas atuais é o Perceptron, apresentado por Rosenblatt na Primeira versão da IA.

O **Perceptron** recebe dados de entrada, que são representados como um vetor:

$$\mathbf{x} = [x_1, x_2, \dots, x_n]$$

Cada entrada é associada a um **peso**:

$$\mathbf{w} = [w_1, w_2, \dots, w_n]$$

Esses pesos representam a **força das conexões** entre as entradas e o neurônio, o que é uma analogia com as **sinapses** do cérebro. Esses pesos são ajustados **ao longo do treinamento**.

O perceptron também possui uma **função de soma**, que realiza uma **soma ponderada** das entradas com seus respectivos pesos, além de um **termo de viés** (*bias*):

$$z = \sum_{i=1}^n x_i w_i + b$$

Em seguida, o resultado dessa soma é passado por uma **função de ativação**, que

define a **classe da entrada**. No perceptron clássico, utiliza-se a **função degrau** (*step function*):

$$y = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

Durante o treinamento, a cada erro de previsão, os pesos do neurônio são **reajustados** de acordo com a **taxa de aprendizado** η , conforme a seguinte equação de atualização:

$$w_i := w_i + \eta \cdot (y - \hat{y}) \cdot x_i$$

Onde:

- η : taxa de aprendizado
- y : saída esperada
- $\hat{y}$: saída prevista pelo perceptron
- x_i : valor da entrada correspondente

Todos esses componentes do Perceptron permitem que o modelo compreenda as representações da entrada e as associe a uma determinada saída. O Perceptron pode ser representado pela Figura 14.

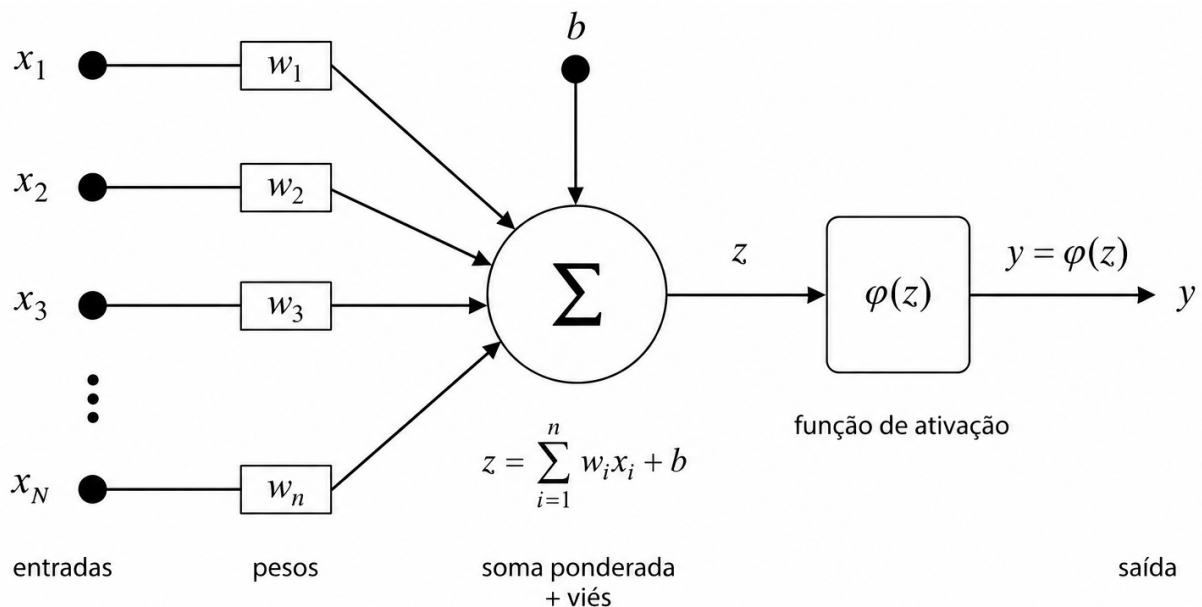


Figura 14 – Perceptron.

O Perceptron Multicamadas (MLP) é um tipo de rede neural artificial composta por múltiplas camadas de perceptrons organizadas em sequência: uma camada de entrada, uma ou mais camadas ocultas e uma camada de saída. Cada camada realiza

transformações lineares seguidas por funções de ativação não lineares (como ReLU ou sigmoid), possibilitando que o modelo compreenda padrões complexos a partir dos dados de entrada. O treinamento ocorre de forma supervisionada, utilizando algoritmos como o backpropagation para ajustar os pesos com base no erro da saída. O MLP é implementado no bloco feed-forward do Transformer, onde cada vetor passa por camadas densas para refinar suas representações. Na Figura 15, é possível visualizar a arquitetura de um MLP.

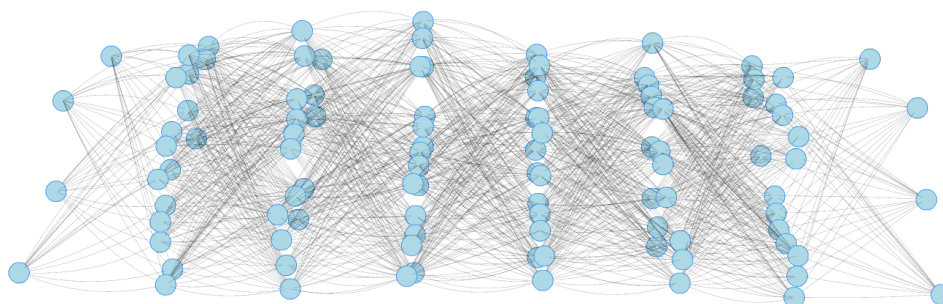


Figura 15 – Perceptron Multicamadas.

Além de todas essas etapas de processamento dos dados, uma fase importante durante a concepção do modelo se destaca, que é a definição dos hiperparâmetros.

Hiperparâmetros

Os hiperparâmetros são variáveis definidas antes do início do processo de treinamento de um modelo de aprendizado de máquina. Diferentemente dos parâmetros do modelo, que são ajustados automaticamente durante o treinamento com base nos dados, os hiperparâmetros precisam ser definidos manualmente ou ajustados por meio de técnicas como busca em grade ou otimização bayesiana.

Eles exercem um papel fundamental na capacidade de aprendizado, generalização e eficiência do modelo. Alguns exemplos comuns de hiperparâmetros em modelos de redes neurais, especialmente na arquitetura Transformers, são:

- **Taxa de aprendizado:** controla o tamanho dos passos dados na atualização dos pesos durante o treinamento.
- **Dimensão do modelo:** número de unidades em cada vetor de representação, impactando a capacidade expressiva da rede.
- **Dimensão da camada feedforward:** tamanho das camadas totalmente conectadas internas ao bloco Transformer.
- **Número de camadas:** quantidade de blocos empilhados no modelo, afetando a profundidade e a capacidade de abstração.

- **Número de cabeças de atenção:** refere-se à quantidade de mecanismos de atenção paralelos utilizados em cada camada de atenção multi-cabeça, influenciando a habilidade do modelo em capturar diferentes padrões de dependência nos dados.

Dentre os LLMs que surgiram baseando-se na arquitetura transformers destaca-se o modelo BERT (do inglês *Bidirectional Encoder Representations from Transformers*).

2.5.2 Bidirectional Encoder Representations from Transformers

O *Bidirectional Encoder Representations from Transformers* (BERT) foi desenvolvido por Devlin; Chang; Lee; Toutanova em 2018. Esse modelo se destaca por conter apenas o componente de codificação da arquitetura Transformers. O BERT possui diversas camadas de codificação, ou seja, o componente de codificação é replicado em múltiplas camadas ocultas, o que difere da arquitetura original dos Transformers, que inclui tanto componentes de codificação quanto de decodificação. Essa representação pode ser visualizada na Figura 16.

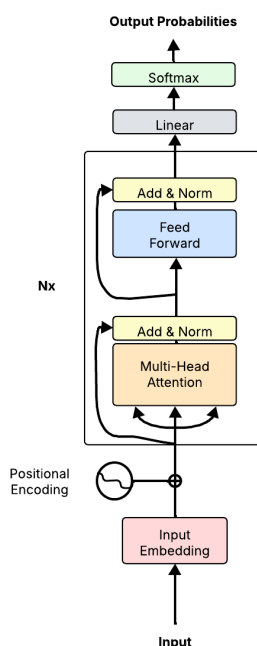


Figura 16 – Arquitetura BERT.

No BERT, os principais hiperparâmetros são: número de camadas codificadoras (L), dimensão do embedding (H) e número de cabeças de autoatenção (A). Os inputs no BERT possuem flexibilidade devido às marcações como [CLS], para tokens de classificação, e [SEP], para separar duas sentenças, caso necessário. Essa flexibilidade é de extrema importância para tarefas como respostas a perguntas, especialmente

em casos de múltipla escolha, permitindo enviar todas as alternativas em conjunto e possibilitando que o modelo as interprete corretamente.

O BERT também permite flexibilidade no tamanho das sentenças de entrada, o que significa que é possível haver sentenças com tamanhos diferentes sem que isso afete o desempenho do modelo.

O BERT recebe uma única matriz com toda a frase, que vai desde o [CLS] até o [SEP]. Para cada [SEP], há uma matriz correspondente a cada sentença. Essa etapa é chamada de *Segment Embedding*. Posteriormente, é adicionada a codificação posicional, como nos Transformers, a cada token que compõe essa matriz.

A saída do BERT é gerada para a tag [CLS], denominada NSP (do inglês *Next Sentence Prediction*), e o modelo também apresenta a representação vetorial para cada token. Essa saída geralmente é utilizada para tarefas de classificação da sentença como um todo, como, por exemplo, inferir o sentimento expresso em uma frase. Na Figura 17, é possível visualizar todo o processo realizado para a classificação de um token utilizando o BERT.

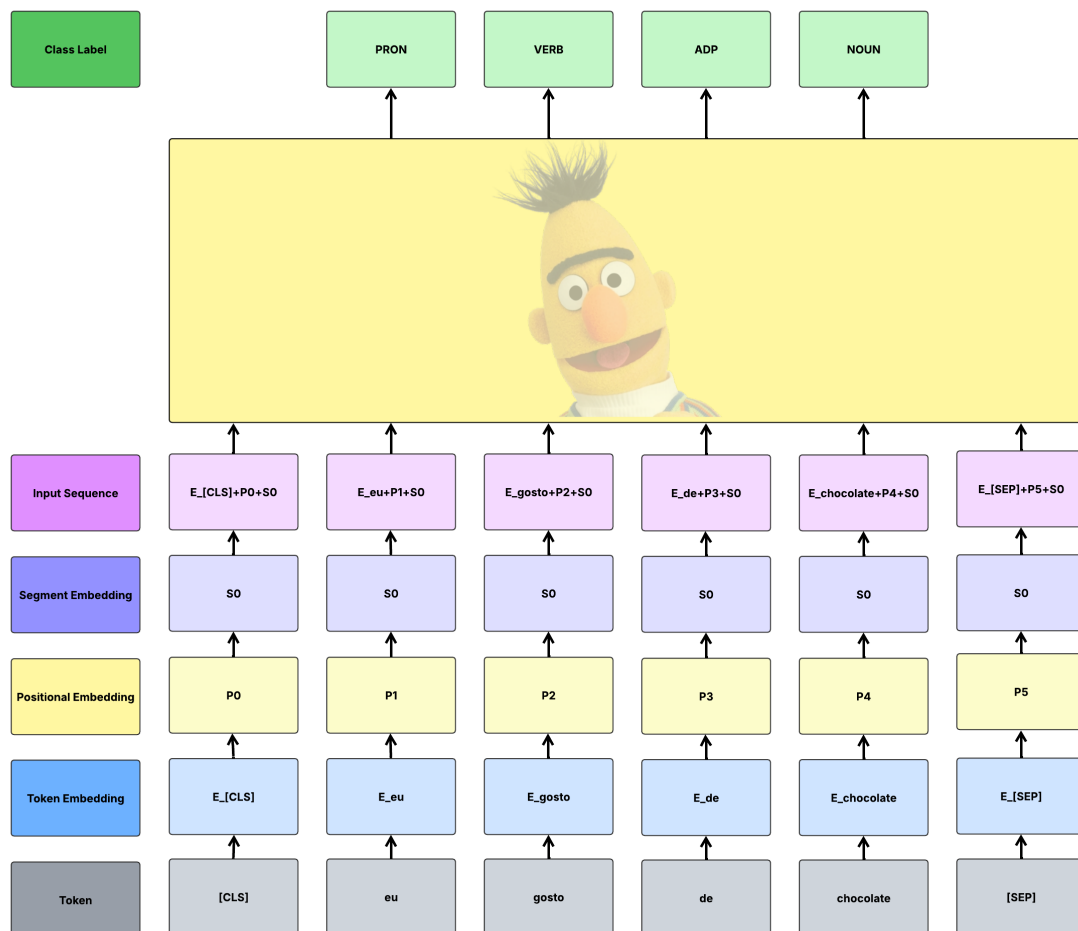


Figura 17 – Processo de classificação de um token com o BERT.

Para seu funcionamento, o BERT utiliza o mascaramento com o objetivo de fazer com que o modelo projete o mesmo valor com base nos tokens à esquerda e à direita. Para cada token que está sendo previsto, o modelo o mascara e, de acordo com os tokens ao redor, tenta prever seu valor correto, que é o próprio token. Esse processo é realizado no componente chamado Masked Language Model. Essa abordagem permite que o modelo compreenda o contexto global da sentença, aprendendo sua estrutura e possibilitando inferir, por exemplo, diferenças entre homônimos, como a palavra “banco”, que pode se referir a uma instituição financeira ou a um assento.

Em média, 15% das palavras são substituídas pelo token [MASK]. Desses 15%, em 80% dos casos aplica-se o mascaramento no token que se deseja prever; em 10% dos casos, são substituídos por tokens aleatórios; e, nos 10% restantes, nenhum token é modificado. Assim, se uma sentença tem 20 palavras, aproximadamente 3 palavras serão substituídas pela tag [MASK].

2.5.3 Ajuste-Fino

O BERT possibilita a realização do processo de Ajuste-Fino (do inglês *fine-tuning*). O *fine-tuning* consiste em utilizar um modelo pré-treinado, que já aprendeu representações gerais da linguagem a partir de grandes corpora, e ajustá-lo para uma tarefa específica, como classificação de sentimentos, reconhecimento de entidades ou análise sintática. Durante esse processo, os pesos do modelo são atualizados com base nos dados da tarefa, permitindo que o BERT adapte seu conhecimento geral da linguagem para resolver problemas concretos com maior precisão. Assim, enquanto o pré-treinamento fornece uma ampla compreensão linguística, o *fine-tuning* refina essas representações para aplicações específicas, aproveitando o aprendizado prévio e evitando a necessidade de treinar o modelo do zero.

2.5.4 Apenas Codificador

As arquiteturas que possuem apenas o codificador se concentram em codificar os dados de entrada. Essas arquiteturas utilizam normalmente autoatenção bidirecional para capturar e representar padrões complexos presentes nos dados de entrada. Elas têm como foco a compreensão profunda da linguagem, possibilitando a inferência sobre os dados processados. Esse comportamento permite que os modelos encoder apresentem bons resultados em tarefas que exigem compreensão profunda, como análise de sentimento, análise sintática, entre outras. A arquitetura que representa os modelos exclusivamente decodificadores pode ser visualizada na Figura 16.

Algumas das desvantagens que diferenciam essa arquitetura das demais são o fato de que os modelos baseados apenas no codificador não conseguem gerar texto, possuem um contexto limitado à entrada e dependem de tarefas supervisionadas, ou seja, que possuem rótulos associados a cada entrada.

2.5.5 Apenas Decodificador

Em contrapartida às arquiteturas que possuem apenas o codificador, existem aquelas compostas somente pela etapa de decodificação dos Transformers. Essas arquiteturas são projetadas para gerar texto semelhante ao produzido por seres humanos, com base na sequência de tokens fornecida. Elas utilizam o mecanismo de autoatenção causal para gerar respostas a partir dos textos de entrada. A arquitetura que representa os modelos exclusivamente decodificadores pode ser visualizada na Figura 18.

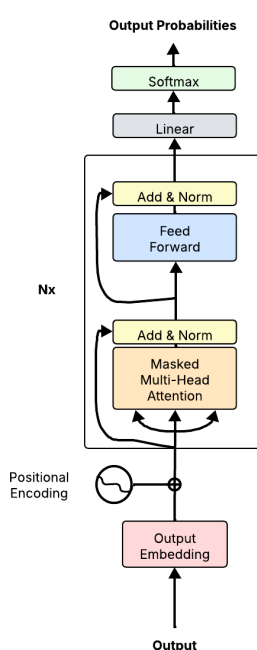


Figura 18 – Arquitetura Apenas Decodificadora.

Os modelos baseados apenas na arquitetura de decodificador foram introduzidos com o objetivo de aprimorar o processamento de sequências longas. Esses modelos apresentam bom desempenho em tarefas de geração de texto.

Entretanto, algumas desvantagens estão associadas ao uso de modelos apenas decodificadores. Entre elas, destaca-se a propensão a alucinações, isto é, a criação de fatos ou eventos plausíveis, porém incorretos, uma vez que o texto é gerado com base em probabilidades e não em uma verificação factual. Além disso, esses modelos tendem a apresentar um custo computacional mais elevado em comparação aos modelos apenas codificadores e desempenho inferior em tarefas estruturadas, como é o caso da análise sintática.

2.6 Aprendizado Multi-Tarefa

E com as limitações das arquiteturas atuais, seja apenas codificadores ou decodificadores, paradigmas que vem ganhando destaque, para superar essas limitações, dentre esses paradigmas se destaca o Aprendizado Multi-Tarefa (do inglês *Multi-Task Learning* - MTL). O MTL foi cunhado por Caruana em 1997, é um paradigma para aproveitar de forma eficaz as informações de tarefas específicas para auxiliar no aprendizado de outras tarefas que estão correlacionadas (Caruana, 1997).

Segundo Caruana, um programa de computador aprende com a experiência E no que diz respeito a alguma classe de tarefas T e medida de desempenho P se seu desempenho nas tarefas em T , conforme medido por P , melhora com a experiência E . Essa ideia pode ser descrita pela equação:

$$E \times T = P$$

Um exemplo de MTL é o ato de andar e correr, logo se alguém aprende a andar, correr é um aprimoramento do andar, então, o conhecimento adquirido para andar pode ser reutilizado para correr caso adaptado corretamente. Esse paradigma explora a ideia de uma generalização dos modelos, não havendo a necessidade de ajustes finos para que aprenda outras tarefas, reduzindo assim o custo computacional para um treinamento.

O MTL pode ser realizado através de duas abordagens. A primeira é o *Hard Parameter Sharing*. Conforme é possível visualizar na figura 19, há o compartilhamento do mesmo modelo entre as tarefas UPOS, DEPREL e HEAD, que são tarefas da AS.

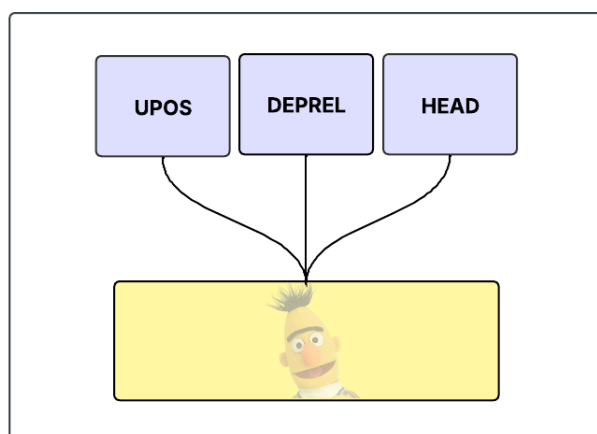


Figura 19 – Arquitetura Hard Parameter Sharing.

Essa abordagem consiste em um único modelo com múltiplas saídas, no qual o compartilhamento ocorre em todo o modelo. Isso significa que uma tarefa influencia

diretamente a outra, possibilitando melhorias quando as tarefas são correlacionadas. Essa estratégia é mais leve, pois utiliza apenas um modelo para diversas tarefas, evitando redundância de parâmetros. Por outro lado, apresenta menor flexibilidade, ou seja, não consegue se adaptar tão bem caso as tarefas não sejam correlacionadas, devido ao compartilhamento integral do modelo.

A segunda abordagem é o *Soft Parameter Sharing*. Conforme é possível visualizar na figura 20, há o compartilhamento dos parâmetros entre diversos modelos para as tarefas UPOS, DEPREL e HEAD, que são tarefas da AS.

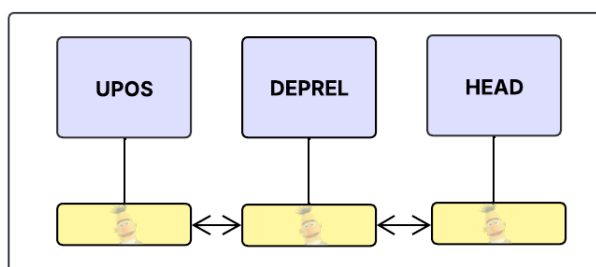


Figura 20 – Arquitetura Soft Parameter Sharing.

Cada tarefa possui seu próprio modelo, mas há um compartilhamento parcial ou “suave” de parâmetros entre eles, geralmente por meio de mecanismos de regularização ou encoders parcialmente comuns. Isso permite que cada tarefa se adapte melhor às suas necessidades específicas, reduzindo interferências negativas entre tarefas não correlacionadas. No entanto, essa abordagem tende a ser mais pesada, pois cada tarefa mantém seus próprios parâmetros, aumentando o uso de memória e a complexidade do treinamento.

3 TRABALHOS RELACIONADOS

Esta seção apresenta uma revisão dos principais trabalhos relacionados ao presente estudo, com foco em parsing de dependências, aprendizado multitarefa MTL, modelos baseados em Transformers e arquiteturas neurais aplicadas ao processamento de linguagem natural para o português e outras línguas morfolologicamente ricas. Também são discutidas abordagens recentes que exploram representações compartilhadas entre diferentes tarefas linguísticas, especialmente em cenários de análise sintática e morfossintática.

Towards PortParser – a Highly Accurate Parsing System for Brazilian Portuguese Following the Universal Dependencies Framework

O trabalho de Lopes; Brisolara; Freitas propõe o PortParser, um sistema de análise sintática e morfossintática para o português brasileiro baseado no padrão Universal Dependencies (UD). O estudo investiga diferentes fatores que influenciam o desempenho do parsing, incluindo arquiteturas neurais, quantidade de épocas de treinamento, tamanho dos embeddings e estratégias de modelagem sintática.

Os autores avaliam diferentes arquiteturas clássicas e neurais, incluindo UDPipe 1.3, UDPipe 2 e Stanza. O UDPipe 1.3 incorpora mecanismos recorrentes baseados em RNN e LSTM, enquanto o UDPipe 2 adiciona mecanismos biaffine para predição de dependências sintáticas. O Stanza, por sua vez, utiliza uma arquitetura pipeline baseada em BiLSTM combinada com mecanismos biaffine.

Além disso, o estudo realiza uma comparação entre modelos multilíngues e monolíngues, utilizando variantes do BERT e do BERTimbau. Os experimentos demonstram que o modelo monolíngue BERTimbau apresenta desempenho superior em tarefas de parsing para o português brasileiro, alcançando aproximadamente 99% de acurácia em PoS tagging e resultados próximos de 96% em UAS e 95% em LAS.

Outro aspecto relevante do trabalho consiste na análise qualitativa dos erros do parser, permitindo identificar padrões sistemáticos associados a construções sintáticas específicas e regiões de baixa representatividade do corpus. Dessa forma, o PortParser constitui uma das principais referências atuais para parsing de dependências

em português brasileiro e representa o principal baseline adotado nesta dissertação.

Universal Grammatical Dependencies for Portuguese with CINTIL Data, LX Processing and CLARIN Support

O trabalho de Branco; Silva; Gomes; António rodrigues apresenta um conjunto abrangente de recursos linguísticos para o português seguindo as diretrizes da Universal Dependencies. O estudo introduz dois recursos principais: o CINTIL-UPos, corpus anotado com etiquetas morfossintáticas e traços morfológicos, e o CINTIL-UDep, treebank anotado com dependências sintáticas segundo o padrão UD.

A construção dos corpora emprega procedimentos rigorosos de anotação, incluindo dupla anotação manual, adjudicação especializada e suporte de gramáticas computacionais profundas, garantindo elevada consistência linguística.

Com base nesses recursos, os autores desenvolveram dois sistemas principais: o LX-UTagger, baseado em BERTimbau para etiquetagem morfossintática, e o LX-UDParser, parser de dependências baseado em estratégias transition-based. Os resultados reportados demonstram desempenho elevado para o português, atingindo aproximadamente 99% de acurácia em PoS e cerca de 90.87% de UAS e 88.01% de LAS.

O trabalho destaca-se principalmente pela disponibilização pública de recursos linguísticos robustos e ferramentas de parsing voltadas ao português europeu e brasileiro, consolidando-se como uma importante referência para estudos envolvendo Universal Dependencies em português.

Etiquetagem Morfossintática Multigênero para o Português do Brasil Segundo o Modelo Universal Dependencies

O estudo de Silva; Pardo; Roman investiga a etiquetagem morfossintática multigênero para o português brasileiro utilizando o padrão Universal Dependencies. Diferentemente de abordagens tradicionais focadas predominantemente em textos jornalísticos, o trabalho avalia modelos em diferentes domínios textuais, incluindo textos acadêmicos e conteúdo gerado por usuários.

Os autores comparam arquiteturas clássicas baseadas em BiLSTM com modelos pré-treinados de linguagem, demonstrando que o BERTimbau apresenta o melhor desempenho entre as abordagens avaliadas, alcançando aproximadamente 99% de acurácia em tarefas de PoS tagging.

O estudo também evidencia que modelos treinados em múltiplos gêneros textuais apresentam maior robustez em cenários de transferência entre domínios, reduzindo significativamente a degradação de desempenho em diferentes estilos linguísticos.

Embora o foco principal do trabalho esteja na etiquetagem morfossintática, os resultados reforçam a importância das representações contextualizadas e das informa-

ções morfossintáticas para tarefas sintáticas mais complexas, como parsing de dependências, aspecto diretamente relacionado à proposta desta dissertação.

Multi-task Learning by Using Contextualized Word Representations for Syntactic Parsing of a Morphologically Rich Language

O trabalho de Ehsan; Butt; Hussain; Alhuzali; Al-laith propõe uma arquitetura multitarefa para parsing sintático aplicada ao idioma Urdu, uma língua morfologicamente rica. O estudo combina parsing de constituintes e parsing de dependências em um único modelo multitarefa baseado em redes BiLSTM, explorando representações compartilhadas entre diferentes estruturas sintáticas.

Os autores investigam tanto cenários de aprendizado de tarefa única quanto aprendizado multitarefa, demonstrando que o compartilhamento de representações sintáticas melhora significativamente os resultados em ambas as tarefas.

A arquitetura proposta utiliza embeddings contextuais baseados em ELMo e mecanismos recorrentes BLSTM para modelagem contextual das sentenças. Os resultados demonstram que o aprendizado multitarefa proporciona ganhos consistentes em LAS, além de reduzir ambiguidades estruturais e erros de classificação entre relações sintáticas semanticamente próximas.

Outro aspecto relevante do trabalho consiste na análise detalhada das matrizes de erro e das ambiguidades sintáticas, evidenciando que representações compartilhadas entre diferentes tarefas linguísticas contribuem para a redução de confusões entre relações sintáticas complexas.

Embora o trabalho utilize arquiteturas recorrentes baseadas em BLSTM e ELMo, diferentemente da presente dissertação, que emprega Transformers e mecanismos biaffine, os resultados apresentados fornecem fortes evidências empíricas de que estratégias multitarefa são particularmente eficazes para parsing sintático em línguas morfologicamente ricas.

Além disso, o estudo reforça a hipótese central desta dissertação de que informações morfossintáticas e sintáticas compartilham dependências estruturais relevantes, permitindo que representações compartilhadas contribuam positivamente para tarefas de parsing de dependências.

Esses trabalhos reforçam a relevância do aprendizado multitarefa para tarefas de parsing e fornecem embasamento teórico para a proposta desta dissertação, especialmente no que se refere ao compartilhamento de representações entre tarefas linguísticas correlacionadas.

Tabela 5 – Comparação entre os trabalhos relacionados e a proposta desta dissertação.

Trabalho	Arquitetura	MTL	UD	Relação com esta dissertação
PortParser (Lopes; Brisolara; Freitas, 2024)	BERTimbau + Biaffine	Parcial	Sim	Baseline principal e principal referência em parsing UD para o português brasileiro.
CINTIL/LX-UDParser (Branco; Silva; Gomes; Antônio rodrigues, 2022)	BERTimbau + Transition-based	Não	Sim	Referência em recursos linguísticos e parsing UD para o português.
Stil/PetroGold (Silva; Pardo; Roman, 2023)	BiLSTM e BERTimbau	Não	Parcial	Evidencia a importância de representações morfossintáticas robustas para tarefas sintáticas.
(Ehsan; Butt; Hussain; Alhuzali; Al-laith, 2025)	BLSTM + ELMo	Sim	Sim	Demonstra empiricamente os benefícios do aprendizado multitarefa em parsing sintático.
Proposta desta dissertação	Modelo + Linear/Biaffine	Sim	Sim	Propõe um parser multitarefa conjunto para UPOS, DEPREL e HEAD em português, comparando arquiteturas lineares e biaffine sobre modelos Transformer pré-treinados.

4 METODOLOGIA

Esta seção tem como objetivo apresentar e descrever a metodologia adotada no presente trabalho. Inicialmente, realizamos o processo de seleção do conjunto de dados, seguido da escolha das tarefas e dos modelos. Em seguida, efetuamos a otimização dos hiperparâmetros e definimos o modelo final selecionado. Avaliamos, por meio do *cross-validation*, a presença de *overfitting* ou *underfitting* no modelo, e posteriormente definimos as métricas e os *baselines* utilizados para comparação com o trabalho proposto. O fluxo desse processo pode ser visualizado na Figura 21.

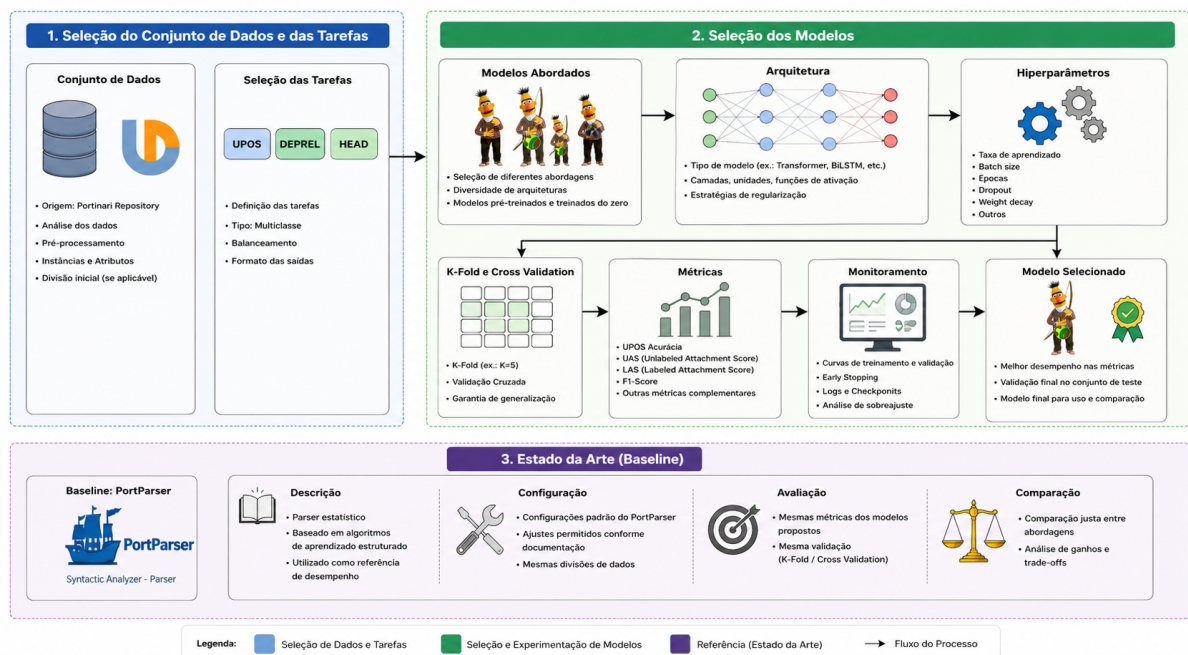


Figura 21 – Pipeline Metodologia.

4.1 Seleção do Conjunto de Dados e das Tarefas

Conjunto de Dados

O conjunto de dados adotado para o processo de concepção do *parser* é o Portinari (Duran; Lopes; Graças nunes; Pardo, 2023). Esse conjunto foi criado em 2023

com o objetivo de ser um recurso multigênero para a área de PLN. O processo de anotação durou quase três anos e envolveu diversos anotadores treinados.

Existem conjuntos de dados que possuem até cerca de 4 milhões de sentenças anotadas de acordo com as normas da UD. Esses conjuntos continuam recebendo atualizações e estão disponíveis no repositório do GitHub¹.

Há, porém, algumas variações desse conjunto de dados: Porttinari-base, Porttinari-check e Porttinari-automatic. O **Porttinari-base** é um corpus revisado manualmente em detalhe para servir como padrão; ele apresenta alta concordância entre os anotadores, medida pela métrica Kappa, de 97,8% e 96,2%. O Porttinari-check é um pequeno corpus estruturalmente semelhante ao Porttinari-base, concebido para servir como banco de testes para avaliações adicionais e diversificadas. Já o Porttinari-automatic é um corpus anotado automaticamente por um *parser* de última geração, baseado no Porttinari-base.

Para o presente trabalho, selecionou-se o Porttinari-base, devido à sua robustez, comprovada por meio da métrica Kappa. A versão utilizada, disponível no repositório do GitHub, contém 168.080 tokens, totalizando 8.418 sentenças, sendo 117.972 (70%) tokens destinados ao conjunto de treino, 16.528 (10%) ao conjunto de desenvolvimento e 33.580 (20%) ao conjunto de teste, conforme disposto na Tabela 6.

Tabela 6 – Distribuição dos dados no conjunto Porttinari-base.

Conjunto	Tokens	Percentual	Sentenças
Treino	117.972	70%	5.893
Desenvolvimento	16.528	10%	842
Teste	33.580	20%	1.683
Total	168.080	100%	8.418

Seleção das Tarefas

As tarefas abordadas neste trabalho são UPOS, DEPREL e HEAD. A tarefa de UPOS apresenta, nos três conjuntos, distribuição desbalanceada, o que reflete a distribuição natural das categorias gramaticais em textos, nos quais aparecem muitos substantivos e preposições e poucos símbolos e interjeições. Esse desbalanceamento implica uma possível dificuldade para o modelo aprender a identificar as categorias de menor frequência. A distribuição nos três conjuntos pode ser observada nas Figuras 22, 23 e 24. O mesmo ocorre para as tags relacionadas à tarefa DEPREL, conforme mostrado nas Figuras 25, 26 e 27.

O conjunto de treino possui comprimento médio de 20,02 tokens por sentença e mediana de 18 tokens. Nas Figuras 28, 29 e 30 observam-se *outliers* nos três conjuntos. A maior sentença presente no corpus possui 79 tokens.

¹https://github.com/UniversalDependencies/UD_Portuguese-Porttinari

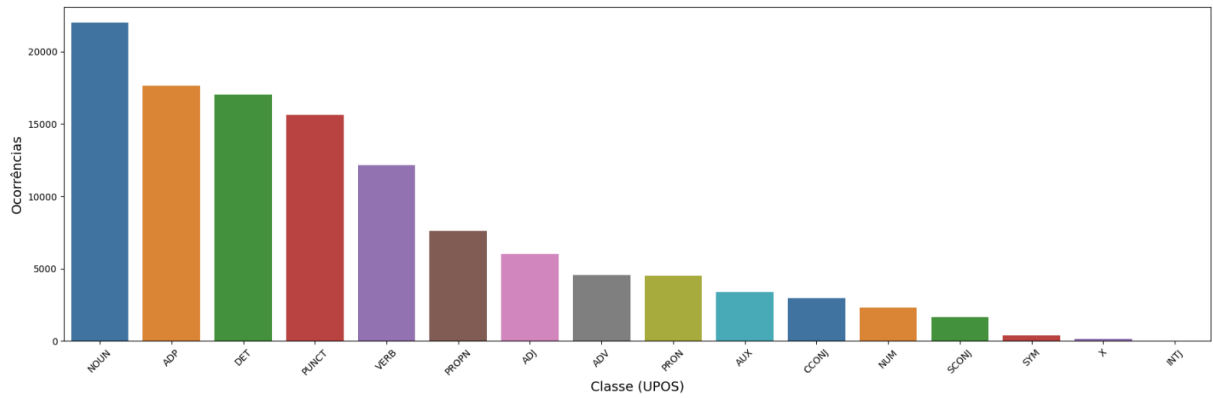


Figura 22 – Frequência UPOS conjunto de Treino.

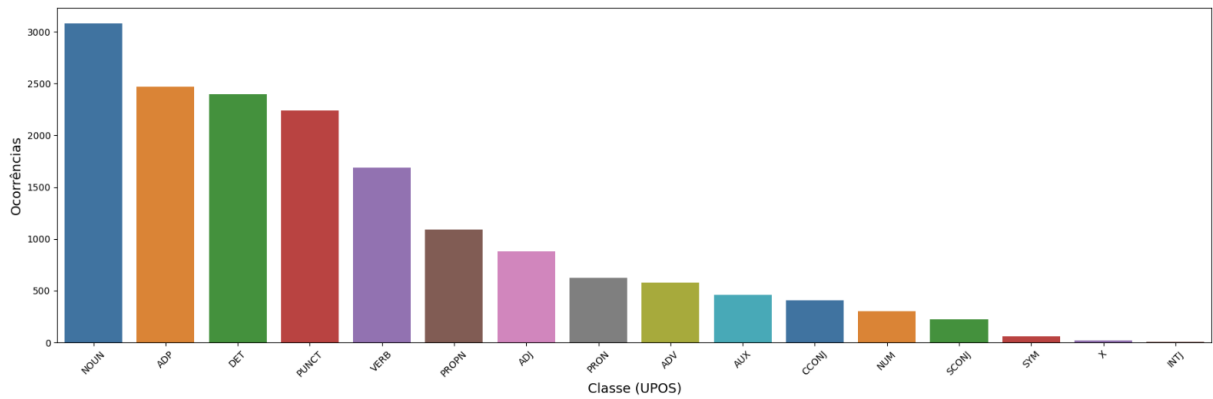


Figura 23 – Frequência UPOS conjunto de Desenvolvimento.

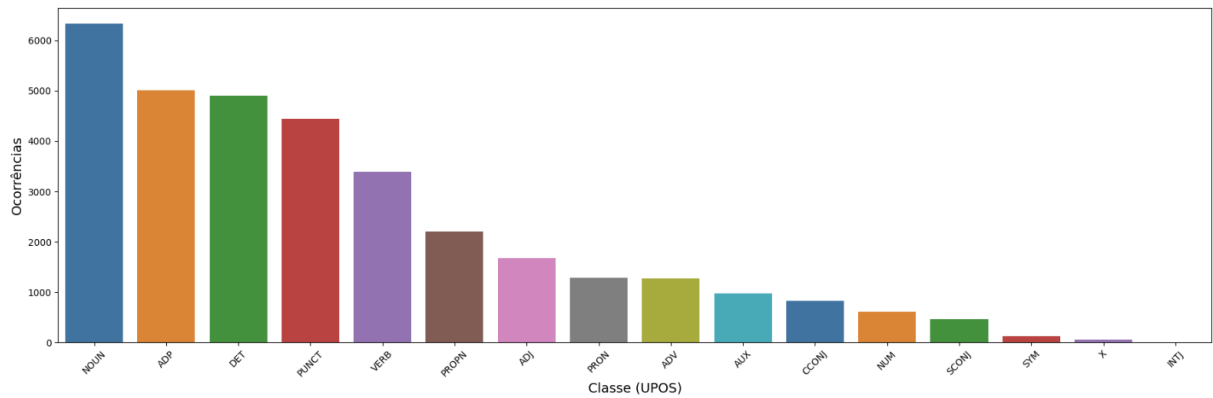


Figura 24 – Frequência UPOS conjunto de Teste.

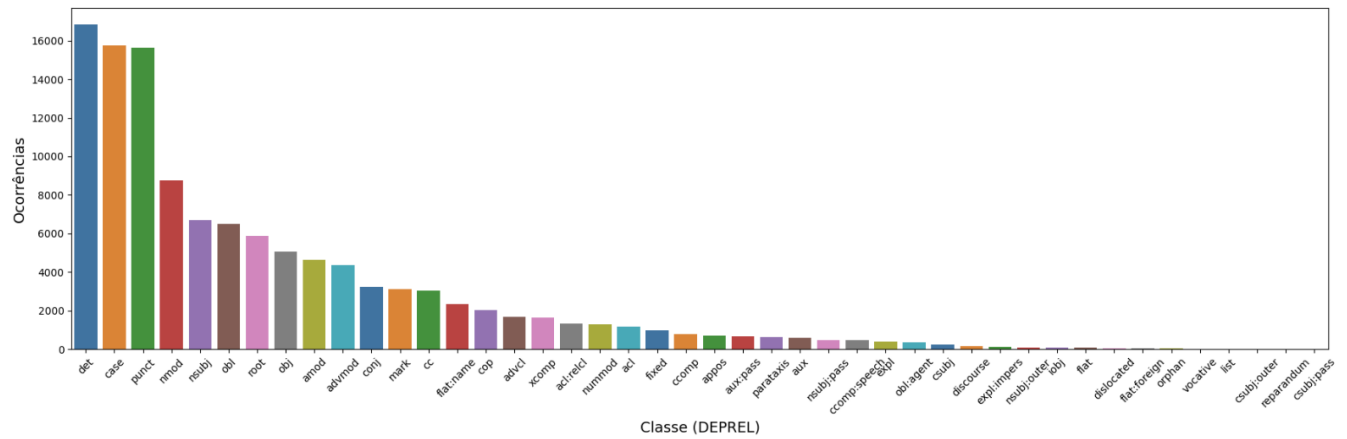


Figura 25 – Frequência DEPREL conjunto de Treino.

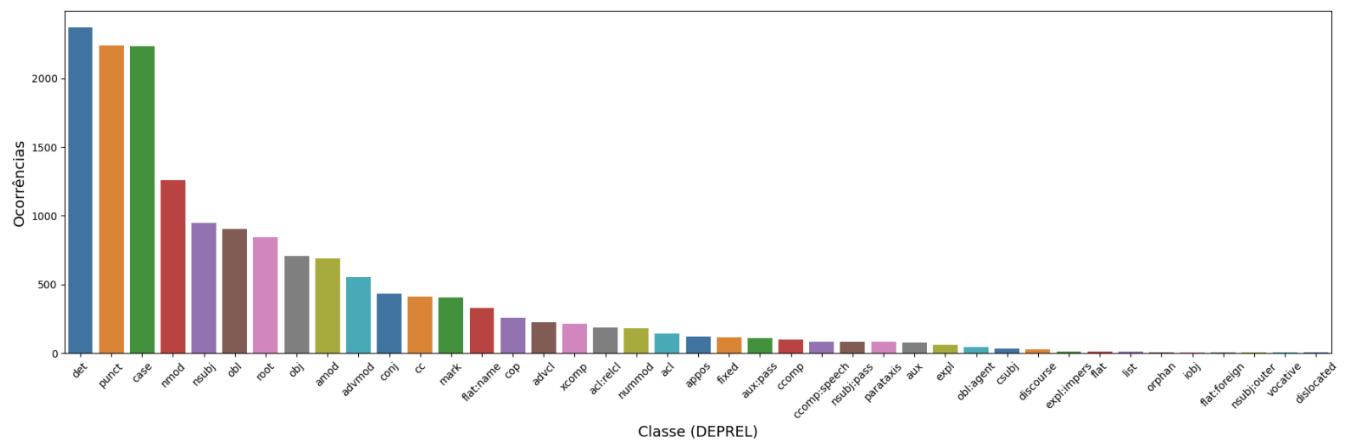


Figura 26 – Frequência DEPREL conjunto de Desenvolvimento.

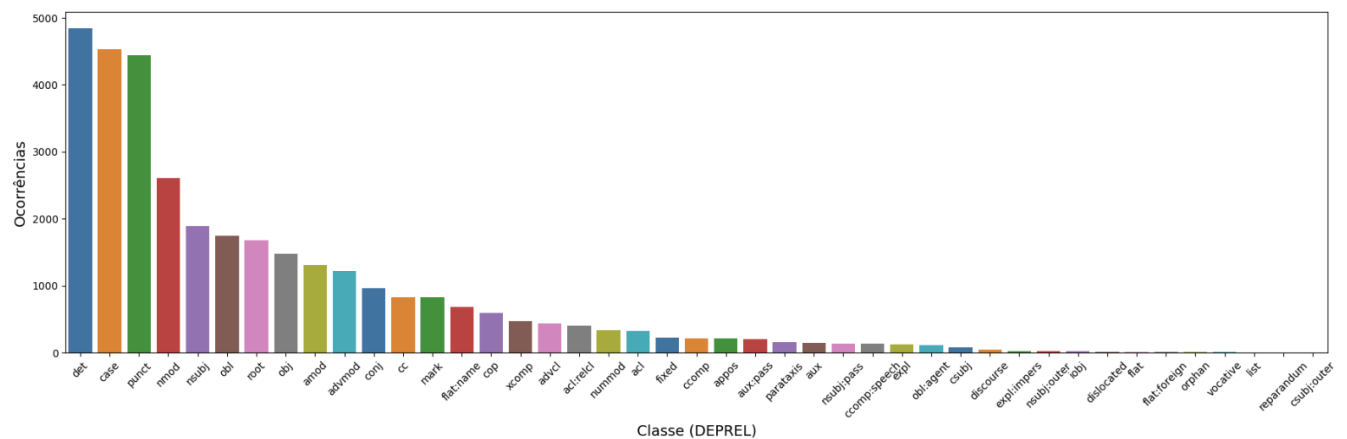


Figura 27 – Frequência DEPREL conjunto de Teste.

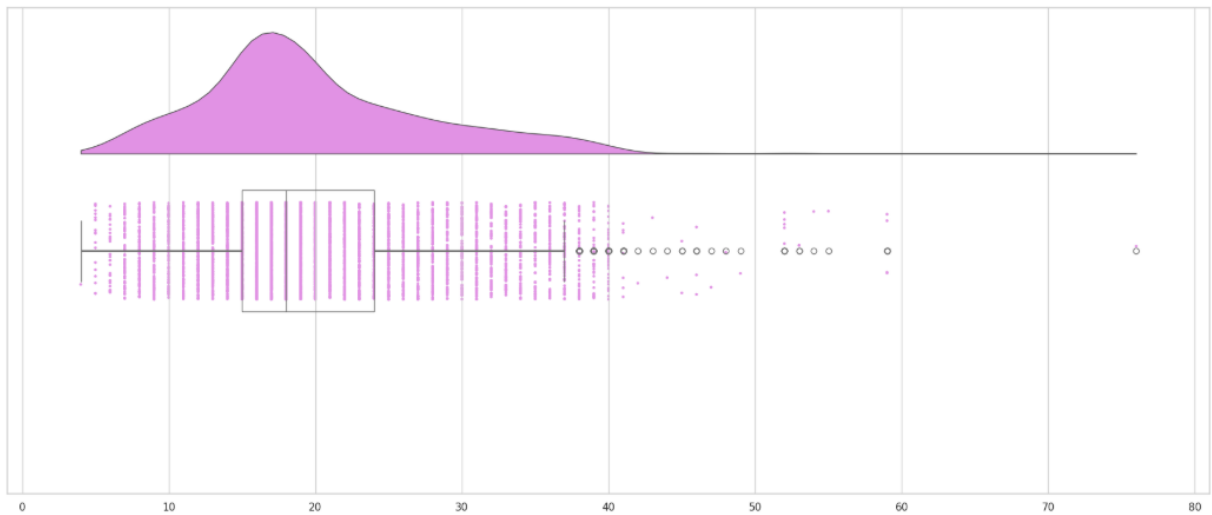


Figura 28 – Box plot do conjunto de treino, mostrando a distribuição do tamanho das sentenças em tokens, incluindo outliers.

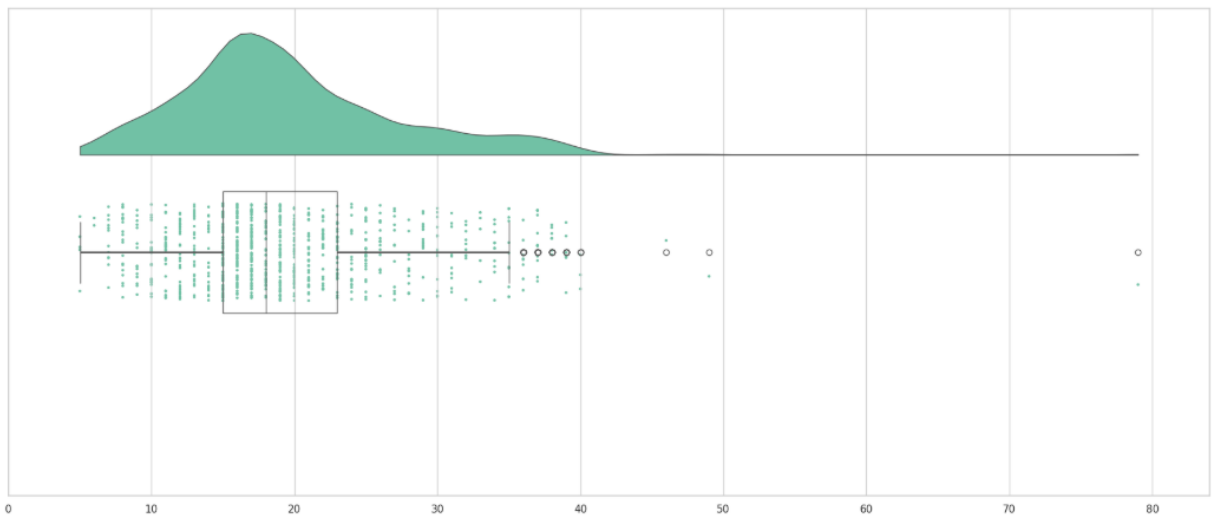


Figura 29 – Box plot do conjunto de desenvolvimento, mostrando a distribuição do tamanho das sentenças em tokens, incluindo outliers.

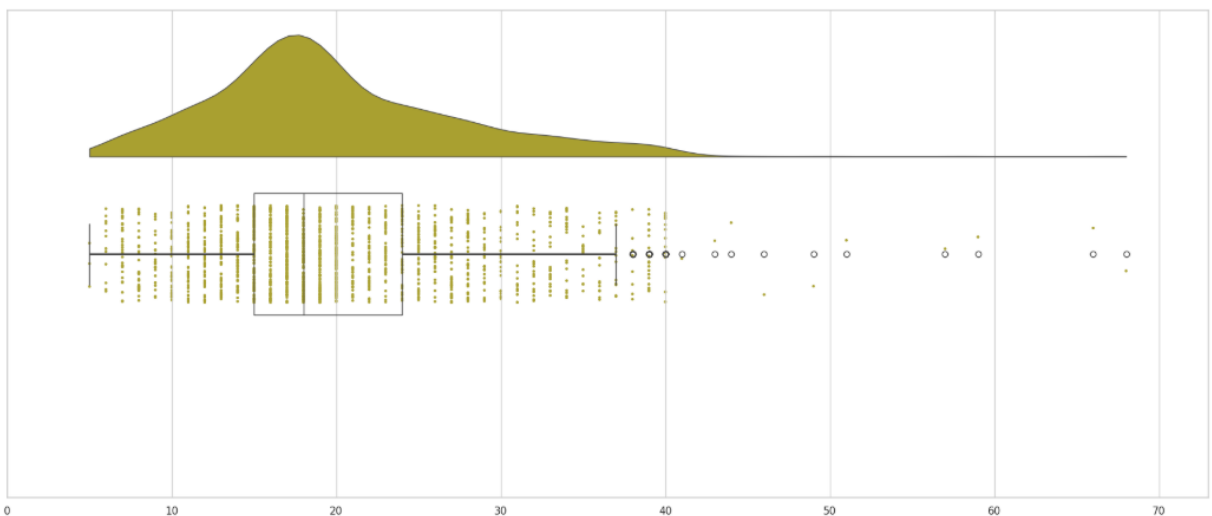


Figura 30 – Box plot do conjunto de teste, mostrando a distribuição do tamanho das sentenças em tokens, incluindo outliers.

4.2 Seleção dos Modelos

Modelos Abordados

Para a escolha do modelo, foram abordados tipos de arquiteturas existentes e atualmente disponíveis — apenas encoders, apenas decoders e encoder-decoder — mesclando abordagens monolíngues e multilíngues de diversos tamanhos. Devido a limitações de hardware, selecionamos modelos com menos de 1 bilhão de parâmetros. Os modelos escolhidos foram: mBERT (multilíngue)², BERTimbau Base (monolíngue)³, BERTimbau Large (monolíngue)⁴ e ModernJabuticaBERT-Base-1k (monolíngue)⁵.

Modelos monolíngues são especializados em um idioma específico, úteis em línguas com morfologia rica e nuances complexas. Tendem a apresentar melhor desempenho em idiomas com poucos recursos disponíveis, pois o pré-treinamento focado permite extrair mais informações mesmo de corpora limitados (Lima; Lopes; Pereira; Silveira; Freitas; Corrêa, 2025). Modelos multilíngues, por outro lado, são pré-treinados em várias línguas simultaneamente, o que os torna eficazes em idiomas com abundância de recursos, embora seu desempenho possa ser inferior ao de modelos monolíngues em idiomas com morfologia complexa ou poucos dados (Silveira; Lima; Júnior; Freitas; Corrêa, 2025).

mBERT

O mBERT é um modelo multilíngue baseado na arquitetura BERT (Devlin; Chang; Lee; Toutanova, 2019), pré-treinado com textos de 104 línguas, incluindo o português. O BERT é um modelo *Transformer* que aprende representações bidirecionais do texto por meio de dois objetivos de pré-treinamento: MLM (*Masked Language Modeling*), no qual o modelo prediz tokens mascarados aleatoriamente no texto de entrada, e NSP (*Next Sentence Prediction*), que determina se duas sentenças são consecutivas no corpus original. O modelo opera com contexto de até 512 tokens e conta com 110 milhões de parâmetros.

BERTimbau Base e Large

O BERTimbau (Souza; Nogueira; Lotufo, 2020) é um modelo treinado especificamente para o português, a partir do corpus BrWaC por 1.000.000 de passos de otimização com estratégia de *whole-word masking*. Esse corpus possui 3,53 milhões de documentos; para o pré-treinamento, o BERTimbau utilizou apenas o corpo dos documentos. O modelo possui contexto de 512 tokens e está disponível em duas versões:

²<https://huggingface.co/google-bert/bert-base-multilingual-cased>

³<https://huggingface.co/neuralmind/bert-base-portuguese-cased>

⁴<https://huggingface.co/neuralmind/bert-large-portuguese-cased>

⁵<https://huggingface.co/amadeusai/modernJabuticaBERT-Base-1k>

Base, com 110 milhões de parâmetros, e Large, com 330 milhões de parâmetros.

JabuticaBERT

O JabuticaBERT (Pires; Egger; Lhoneux, 2024) é um modelo monolíngue baseado na arquitetura BERT, desenvolvido com foco específico no português brasileiro e em cenários de maior escala de dados. O modelo foi pré-treinado em um conjunto de dados ampliado, combinando múltiplas fontes textuais de alta qualidade, o que contribui para representações linguísticas mais robustas. O treinamento segue o paradigma de MLM, com adaptações modernas nas estratégias de tokenização e otimização alinhadas com práticas recentes da literatura. O modelo mantém suporte a sequências de até 512 tokens e utiliza o vocabulário expandido da versão *Modern-BERT*, posicionando-se como alternativa entre modelos base e large em termos de capacidade de representação.

Pré-processamento e Tokenização

Carregamento e Estrutura dos Dados

O corpus Porttinari-base foi armazenado em arquivos CSV com uma sentença por linha. Cada linha contém listas serializadas com os tokens, rótulos UPOS, DEPREL e os índices inteiros de HEAD. Esses arquivos foram carregados com a biblioteca *Pandas* e convertidos para o formato *Dataset* da biblioteca *HuggingFace Datasets* (Wolf; Debut; Sanh; Chaumond; Delangue; Moi; Cistac; Rault; Louf; Funtowicz et al., 2020), criando um *DatasetDict* com as três partições: treino (`train.csv`), desenvolvimento (`val.csv`) e teste (`test.csv`).

Cada exemplo no conjunto de dados contém os seguintes campos:

- `tokens`: lista de strings com os tokens da sentença;
- `head_tags`: lista de inteiros (0-indexados) indicando o índice do token regente (HEAD) para cada posição. Para os modelos com cabeçote linear, o HEAD é predito como classificação sobre 100 posições; para o cabeçote biaffine, o espaço de predição abrange 200 posições, cobrindo todas as sentenças do corpus;
- `deprel_tags`: lista de inteiros mapeando cada token ao índice de seu rótulo de relação de dependência (DEPREL) em um vocabulário fechado de 44 rótulos;
- `upos_tags`: lista de inteiros mapeando cada token ao índice de sua categoria morfofossintática universal (UPOS).

Tokenização por Subpalavras

A tokenização foi realizada pelo tokenizador nativo de cada encoder pré-treinado, carregado via `AutoTokenizer.from_pretrained` da biblioteca *HuggingFace Transfor-*

mers (Wolf; Debut; Sanh; Chaumond; Delangue; Moi; Cistac; Rault; Louf; Funtowicz et al., 2020). Todos os encoders avaliados utilizam a estratégia WordPiece (Wu; Schuster; Chen; Le; Norouzi; Macherey; Krikun; Cao; Gao; Macherey et al., 2016) para segmentação de palavras em subpalavras.

Como as sentenças do Porttinari já estão pré-segmentadas ao nível de palavra, a tokenização foi realizada com o parâmetro `is_split_into_words=True`, que instrui o tokenizador a processar cada elemento da lista de entrada como uma palavra previamente delimitada, aplicando a segmentação em subpalavras internamente. Os demais parâmetros foram configurados da seguinte forma:

- **Comprimento máximo** (`max_length=512`): reflete o limite de contexto dos encoders baseados em BERT. Como a maior sentença do corpus possui 79 tokens, e a relação de expansão de palavras para subpalavras é tipicamente pequena para o português, nenhuma sentença do Porttinari-base exigiu truncção após a tokenização;
- **Truncção** (`truncation=True`): ativada como salvaguarda para sentenças que eventualmente ultrapassem 512 subpalavras após expansão;
- **Preenchimento** (`padding="max_length"`): todas as sequências são completadas com o token de preenchimento (`[PAD]`) até 512 posições, garantindo lotes de tamanho uniforme na fase de pré-processamento.

Alinhamento de Rótulos com Subpalavras

A segmentação em subpalavras introduz um desalinhamento entre os índices dos tokens originais e os das subpalavras geradas. Por exemplo, a palavra “*realização*” pode ser segmentada em duas subpalavras: “*realiza*” e “*##ção*”, resultando em dois vetores de saída do encoder para uma única entrada anotada. Para que os rótulos de DEPREL, HEAD e UPOS sejam corretamente associados a cada subpalavra, foi aplicada a seguinte estratégia de alinhamento:

1. O método `word_ids(batch_index)` da saída do tokenizador retorna, para cada posição da sequência tokenizada, o índice da palavra original correspondente, ou `None` para tokens especiais (`[CLS]`, `[SEP]`) e posições de preenchimento;
2. Posições com `word_id = None` recebem o rótulo sentinela `-100`, que é ignorado pela função de perda `CrossEntropyLoss` durante o treinamento;
3. Para cada subpalavra resultante da segmentação de uma palavra w_i , o rótulo de w_i é atribuído a **todas** as subpalavras geradas por aquela palavra — incluindo fragmentos intermediários como “*##ção*” — que recebem o mesmo índice de rótulo que a subpalavra inicial.

Essa estratégia garante que todas as subpalavras de uma palavra contribuam igualmente para o cálculo da perda durante o treinamento. O Trecho de Código 4.1 apresenta a implementação da função de alinhamento utilizada:

Listing 4.1 – Função de alinhamento de rótulos com subpalavras.

```
def align_labels_with_tokens(labels , word_ids):
    new_labels , current_word = [] , None
    for word_id in word_ids:
        if word_id != current_word:
            current_word = word_id
            label = -100 if word_id is None else labels[word_id]
        elif word_id is None:
            label = -100
        else :
            label = labels[word_id] # mesma etiqueta para subpalavras
        new_labels.append(label)
    return new_labels
```

Estratégia de Predição na Inferência

Durante o treinamento, todas as subpalavras de uma palavra contribuem para o gradiente com o mesmo rótulo da palavra original. Durante a inferência, entretanto, apenas a **primeira subpalavra** de cada token original é utilizada para extrair a predição final:

$$\hat{y}_i = \arg \max f_{\theta}(\mathbf{h}_{\text{first}(i)}),$$

em que $\mathbf{h}_{\text{first}(i)}$ é a representação da primeira subpalavra do token i na saída do encoder e f_{θ} é o cabeçote de predição. Essa assimetria treino/inferência é uma escolha deliberada: ao treinar com todas as subpalavras, o modelo recebe mais exemplos supervisionados por sentença; ao predizer com apenas a primeira, evita-se ambiguidade sobre qual subpalavra deve representar a palavra inteira. Essa prática é amplamente adotada em tarefas de anotação ao nível de token com encoders pré-treinados (Devlin; Chang; Lee; Toutanova, 2019).

Colator de Dados (Data Collator)

Os exemplos pré-processados foram agrupados em lotes por meio de um colator de dados personalizado implementado em PyTorch. O colator recebe uma lista de exemplos já tokenizados e alinhados, e os combina em tensores, aplicando preenchimento dinâmico ao comprimento máximo do lote. As convenções de preenchimento

são:

- `input_ids`: preenchidos com `pad_token_id` do tokenizador;
- `attention_mask`: preenchidos com 0 (posições de *padding* são mascaradas);
- `deprel_label`, `head_label` e `upos_label`: preenchidos com `-100`, garantindo que posições de preenchimento não influenciem o cálculo da perda.

Arquitetura

Correlação entre as Tarefas de UPOS e DEPREL

Inicialmente, foi avaliado o grau de dependência entre as tarefas de rotulagem morfo-sintática (UPOS) e de rotulagem sintática (DEPREL) no corpus utilizado. O objetivo é quantificar o nível de interdependência entre essas tarefas — ou seja, em que medida o conhecimento da classe gramatical de um token fornece informações relevantes sobre sua função sintática.

4.2.0.0.1 Métricas de correlação.

Como UPOS e DEPREL são variáveis categóricas nominais, foram empregadas métricas de associação estatística adequadas para esse tipo de dado.

4.2.0.0.2 Informação Mútua (*Mutual Information* — MI).

A Informação Mútua mede a quantidade de informação compartilhada entre duas variáveis aleatórias. No contexto deste trabalho, a MI quantifica a redução de incerteza sobre o rótulo DEPREL ao se conhecer o rótulo UPOS de um token:

$$MI(U, D) = \sum_{u \in U} \sum_{d \in D} p(u, d) \log_2 \frac{p(u, d)}{p(u)p(d)},$$

em que U e D representam os conjuntos de rótulos de UPOS e DEPREL, respectivamente, enquanto $p(u, d)$, $p(u)$ e $p(d)$ correspondem às probabilidades conjunta e marginais estimadas a partir do corpus.

4.2.0.0.3 Informação Mútua Normalizada (*Normalized Mutual Information* — NMI).

Para permitir comparação entre corpora distintos, a MI foi normalizada para o intervalo $[0, 1]$:

$$NMI(U, D) = \frac{MI(U, D)}{\frac{1}{2}(H(U) + H(D))},$$

em que $H(\cdot)$ representa a entropia de Shannon.

4.2.0.0.4 V de Cramér.

O V de Cramér é uma medida de tamanho de efeito baseada no teste qui-quadrado:

$$V = \sqrt{\frac{\chi^2}{N \cdot (\min(r, c) - 1)}},$$

em que χ^2 é a estatística do teste, N é o número total de observações, e r e c indicam o número de categorias de UPOS e DEPREL. O valor de V varia entre 0 (ausência de associação) e 1 (associação máxima).

4.2.0.0.5 Resultados.

A análise foi realizada sobre a totalidade do corpus Porttinari-base (168.080 tokens):

- **MI:** 2,0306
- **NMI:** 0,7600
- **V de Cramér:** 0,7962
- **Qui-quadrado** (χ^2): 1.812.777,35 ($p\text{-value} < 10^{-10}$)

Os resultados indicam forte associação estatística entre UPOS e DEPREL. O valor de NMI igual a 0,76 evidencia elevado compartilhamento informacional entre as variáveis analisadas, e o V de Cramér de 0,7962 confirma que a associação não é marginal. O teste qui-quadrado também rejeita a hipótese de independência estatística com $p\text{-value}$ praticamente nulo.

Esses resultados fornecem evidências favoráveis ao uso de aprendizado multitarefa, pois a classe gramatical de um token fornece informações relevantes sobre sua função sintática. Entretanto, embora as métricas indiquem forte dependência estatística entre UPOS e DEPREL, tais medidas não implicam causalidade. A predição do HEAD, por envolver a identificação do índice do token regente, continua dependendo de informações estruturais globais da sentença, especialmente em construções sintáticas ambíguas ou dependências de longa distância.

Arquitetura Linear Multitarefa

Com base na forte dependência estatística entre UPOS e DEPREL, foi proposta uma arquitetura multitarefa composta exclusivamente por camadas lineares específicas para cada tarefa, mantendo o compartilhamento dos parâmetros do encoder.

O encoder pré-treinado recebe como entrada a sequência tokenizada e produz um tensor de representações contextuais $\mathbf{H} \in \mathbb{R}^{B \times L \times d}$, em que B é o tamanho do

lote, L é o comprimento máximo da sequência e d é a dimensão oculta do encoder ($d = 768$ para modelos Base e $d = 1024$ para modelos Large). Após a aplicação de *dropout* (com taxa determinada pela configuração `hidden_dropout_prob` ou `classifier_dropout` de cada encoder), o tensor é projetado por três camadas `nn.Linear` independentes:

- **HEAD:** $W_h \in R^{100 \times d}$, projetando para um espaço de 100 posições candidatas a token regente;
- **DEPREL:** $W_d \in R^{44 \times d}$, projetando para os 44 rótulos de dependência do Porttiani;
- **UPOS:** $W_u \in R^{N_u \times d}$, projetando para os N_u rótulos morfossintáticos do corpus.

A função de perda total é a soma das perdas `CrossEntropyLoss` para cada tarefa, calculadas apenas sobre posições com rótulo $\neq -100$:

$$\mathcal{L} = \mathcal{L}_{HEAD} + \mathcal{L}_{DEPREL} + \mathcal{L}_{UPOS}.$$

Diferentemente de abordagens amplamente utilizadas na literatura, como UDify (Kondratyuk; Straka, 2019), UDPipe 2.0 (Straka; Straková, 2017) e Stanza (Qi; Zhang; Zhang; Bolton; Manning, 2020), a arquitetura proposta não incorpora mecanismos explícitos de interação entre pares de tokens para a predição de dependências sintáticas. Em vez disso, investiga-se até que ponto as representações compartilhadas produzidas pelo encoder são capazes de capturar informações suficientes para a predição conjunta de UPOS, HEAD e DEPREL por projeção linear direta. A Figura 31 apresenta a arquitetura multitarefa linear.

4.2.0.0.6 Comparação com o UDify.

O UDify (Kondratyuk; Straka, 2019) propõe uma arquitetura multitarefa e multilíngue baseada no mBERT para análise linguística segundo o framework UD, realizando simultaneamente etiquetagem morfossintática, predição de características morfológicas, lematização e análise de dependências. Para a tarefa de *parsing*, o modelo utiliza classificadores biaffine que modelam explicitamente relações estruturais entre palavras. Embora ambas as abordagens explorem aprendizado multitarefa, seus objetivos são distintos: o UDify foi concebido como solução universal para múltiplos idiomas e tarefas, enquanto o ParseH2IA concentra-se especificamente na análise sintática do português com arquiteturas mais simples e sem encoder multilíngue.

4.2.0.0.7 Comparação com o UDPipe 2.0.

O UDPipe 2.0 (Straka; Straková, 2017) é um sistema modular em *pipeline* que contempla tokenização, lematização, etiquetagem morfossintática e *parsing*. Cada etapa

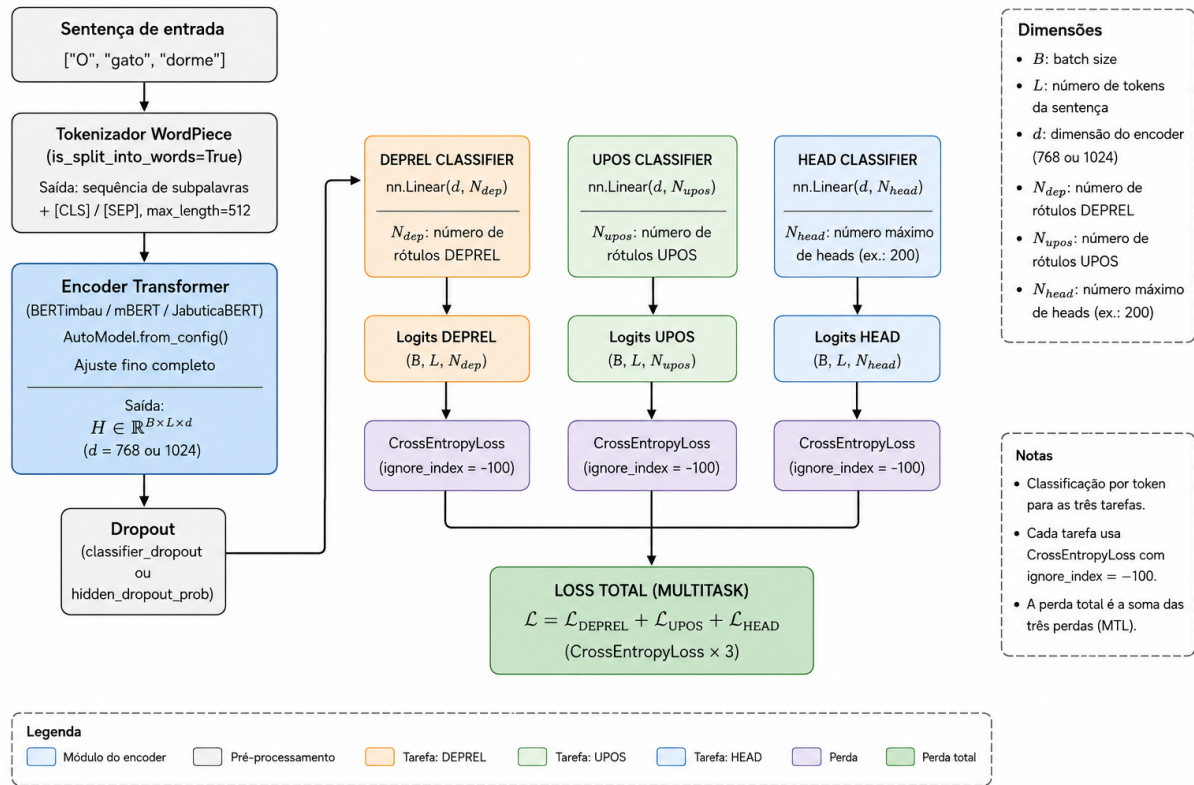


Figura 31 – Arquitetura multitarefa baseada em camadas lineares.

é tratada por componentes especializados sequencialmente. A presente abordagem diferencia-se por adotar aprendizado multitarefa com compartilhamento de parâmetros, permitindo que UPOS, HEAD e DEPREL sejam aprendidas simultaneamente a partir de um único encoder contextualizado.

4.2.0.0.8 Comparação com o Stanza.

O Stanza (Qi; Zhang; Zhang; Bolton; Manning, 2020) é um *toolkit* neural completo para PLN multilíngue, com módulos especializados para cada tarefa. Esta dissertação difere por investigar especificamente o impacto do aprendizado multitarefa sobre HEAD e DEPREL com arquiteturas simples, em vez de priorizar cobertura de múltiplas tarefas e idiomas.

Arquitetura Biaffine Multitarefa

Além da arquitetura linear, foi avaliada uma segunda abordagem multitarefa que incorpora o mecanismo biaffine (Dozat; Manning, 2017) para as tarefas de predição de HEAD e DEPREL. O objetivo é investigar se a modelagem explícita das relações entre pares de tokens proporciona ganhos em relação à abordagem linear.

Nesta variante, a tarefa de UPOS continua sendo realizada por uma camada linear sobre a saída do encoder. Para as tarefas de HEAD e DEPREL, o tensor de saída do encoder é projetado por quatro MLPs especializadas (Li-

near + ELU + LayerNorm + Dropout):

- *arc_head_mlp*: projeta cada token para o papel de *cabeça de arco*, com dimensão oculta $d_{arc} = 500$;
- *arc_dep_mlp*: projeta cada token para o papel de *dependente de arco*, com dimensão oculta $d_{arc} = 500$;
- *rel_head_mlp*: projeta cada token para o papel de *cabeça de relação*, com dimensão oculta $d_{rel} = 100$;
- *rel_dep_mlp*: projeta cada token para o papel de *dependente de relação*, com dimensão oculta $d_{rel} = 100$.

Em seguida, dois mecanismos de atenção biaffine calculam escores entre pares de representações:

$$\text{score}_{arc}(i, j) = \mathbf{h}_i^{(arc-dep)} \mathbf{W}_{arc} \mathbf{h}_j^{(arc-head)},$$

$$\text{score}_{rel}(i, j) = \mathbf{h}_i^{(rel-dep)} \mathbf{W}_{rel} \mathbf{h}_j^{(rel-head)},$$

em que $\mathbf{W}_{arc} \in R^{1 \times (d_{arc}+1) \times (d_{arc}+1)}$ e $\mathbf{W}_{rel} \in R^{44 \times (d_{rel}+1) \times (d_{rel}+1)}$ são parâmetros aprendidos, e os termos de viés são incorporados ao vetor antes da multiplicação (bias_x=True para ambos; bias_y=False para o arco e bias_y=True para a relação, conforme Dozat; Manning (2017)).

A função de perda total do modelo biaffine é a soma das três perdas CrossEntropyLoss:

$$\mathcal{L} = \mathcal{L}_{arc} + \mathcal{L}_{rel} + \mathcal{L}_{UPOS}.$$

A Figura 32 apresenta a arquitetura multitarefa biaffine.

A Tabela 7 apresenta um resumo comparativo das camadas utilizadas em cada arquitetura para as três tarefas.

Tabela 7 – Comparação entre as arquiteturas avaliadas para as tarefas de UPOS, DEPREL e HEAD.

Abordagem	UPOS	DEPREL	HEAD
Linear MTL	Linear	Linear	Linear
Biaffine MTL	Linear	Biaffine	Biaffine

As duas arquiteturas caracterizam aprendizado multitarefa com compartilhamento de parâmetros: um único encoder produz representações utilizadas simultaneamente nas três tarefas. A diferença reside exclusivamente no cabeçote de predição para HEAD e DEPREL.

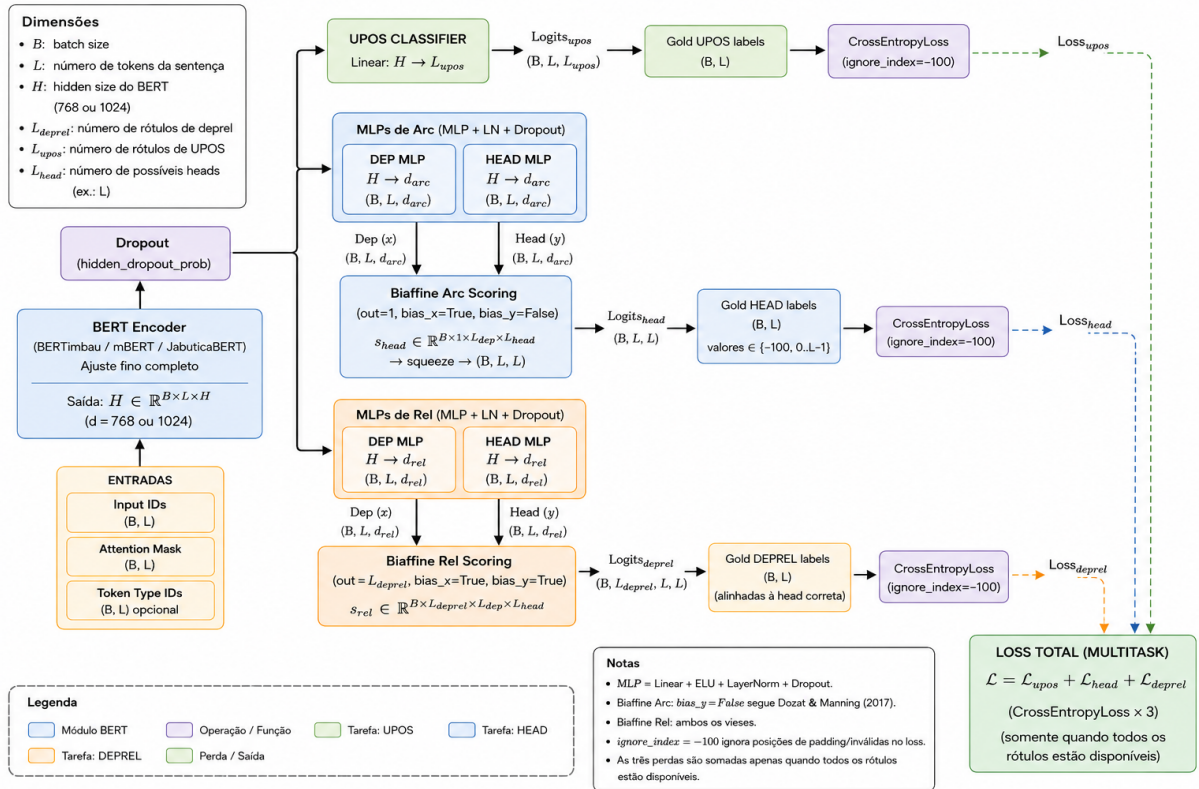


Figura 32 – Arquitetura multitarefa com mecanismo biaffine para predição de HEAD e DEPREL.

Estudo de Ablação: Regime sem UPOS

Para avaliar a contribuição isolada da tarefa auxiliar de UPOS sobre o desempenho do *parsing*, foram treinadas versões ablativas dos modelos em que a cabeça de predição de UPOS é removida. Nesse regime, o encoder é ajustado apenas pelos gradientes provenientes das tarefas de HEAD e DEPREL, e a função de perda se reduz a:

$$\mathcal{L}_{ablao} = \mathcal{L}_{HEAD} + \mathcal{L}_{DEPREL}.$$

Os hiperparâmetros do regime de ablação foram os mesmos obtidos na busca bayesiana para cada combinação encoder $\times$ cabeçote, garantindo que a comparação entre MTL e ablação não seja confundida por diferenças de otimização. Ao todo, foram treinados e avaliados **16 modelos**: 4 encoders $\times$ 2 arquiteturas (linear e biaffine) $\times$ 2 regimes (MTL com UPOS e ablação sem UPOS).

Métricas

A avaliação do desempenho do *parser* foi definida conforme a adoção na literatura, com as métricas LAS, UAS e acurácia UPOS.

4.2.0.0.9 Labeled Attachment Score (LAS).

O LAS avalia simultaneamente a precisão da predição do HEAD e do rótulo sintático associado à dependência (Nivre; Hall; Nilsson; Eryiğit; Marinov, 2006):

$$LAS = \frac{\sum_{i=1}^N 1[(\hat{h}_i, \hat{r}_i) = (h_i, r_i)]}{N},$$

em que $\hat{h}_i$ e $\hat{r}_i$ representam o HEAD e o rótulo sintático preditos para o token i , enquanto h_i e r_i correspondem aos valores de referência anotados no corpus.

4.2.0.0.10 Unlabeled Attachment Score (UAS).

O UAS mede a precisão da estrutura sintática predita, considerando apenas a identificação correta do HEAD, independentemente do rótulo (Chen; Kawahara; Uchimoto; Zhang; Isahara, 2008):

$$UAS = \frac{\sum_{i=1}^N 1[\hat{h}_i = h_i]}{N}.$$

4.2.0.0.11 Acurácia de etiquetas UPOS.

A acurácia UPOS avalia a precisão da classificação morfossintática (Paul; Islam; Rahman et al., 2023):

$$UPOS = \frac{\sum_{i=1}^N 1[\hat{y}_i = y_i]}{N},$$

em que $\hat{y}_i$ é a etiqueta morfossintática predita e y_i é a anotação de referência.

Hiperparâmetros

Os hiperparâmetros dos modelos foram definidos por meio do framework Optuna (Akiba; Sano; Yanase; Ohta; Koyama, 2019). O Optuna é uma ferramenta de otimização automática de hiperparâmetros voltada para aprendizado de máquina, caracterizada pela alta modularidade e pela possibilidade de construção dinâmica dos espaços de busca. O framework combina otimização bayesiana, amostragem probabilística e parada antecipada de *trials* não promissores (*pruning*).

Para a seleção dos hiperparâmetros candidatos, o Optuna utiliza o algoritmo TPE (*Tree-structured Parzen Estimator*) (Bergstra; Bardenet; Bengio; Kégl, 2011), adotado como padrão pelo framework. O TPE é um método de otimização bayesiana que modela separadamente a distribuição de hiperparâmetros associados a bons resultados e àqueles associados a resultados ruins, guiando a busca para regiões do espaço com maior probabilidade de bom desempenho.

Os intervalos de busca foram baseados em recomendações da literatura recente

sobre ajuste fino de modelos de linguagem pré-treinados (Halfon; Gretz; Arviv; Spector; Toledo-ronen; Katz; Ein-dor; Shmueli-scheuer; Slonim, 2024; Gupta; Thérien; Ibrahim; Richter; Anthony; Belilovsky; Rish; Lesort, 2023; Ibrahim; Thérien; Gupta; Richter; Anthony; Lesort; Belilovsky; Rish, 2024; Pareja; Nayak; Wang; Killamsetty; Sudalairaj; Zhao; Han; Bhandwaldar; Xu; Xu; Han; Inglis; Srivastava, 2024).

O processo de otimização teve como objetivo **maximizar o LAS** no conjunto de validação. Foram realizados 10 *trials* por combinação de encoder e arquitetura (8 combinações no total), cada *trial* executando a validação cruzada de 5 *folds*. As variáveis de otimização foram:

- taxa de aprendizado (*learning_rate*): 1×10^{-5} a 5×10^{-5} , escala log-uniforme;
- decaimento de peso (*weight_decay*): 0,1 a 0,3, escala uniforme;
- fator de *warmup* (*warmup_ratio*): 0,3 a 0,5, escala uniforme.

As Tabelas 8 e 9 apresentam, respectivamente, os hiperparâmetros mantidos fixos e o espaço de busca do Optuna.

Tabela 8 – Hiperparâmetros fixos utilizados durante o treinamento.

Hiperparâmetro	Valor
Épocas de treinamento	40
Estratégia de avaliação	por época
Acumulação de gradiente	1
Norma máxima do gradiente	1,0
Tipo de agendador de LR	linear (<i>warmup</i> + decaimento)
Métrica para melhor modelo	LAS
<i>Label smoothing</i>	0,0
Otimizador	AdamW
<i>Batch size</i> — treino	16 amostras/dispositivo
<i>Batch size</i> — avaliação	32 amostras/dispositivo
Carregadores de dados (<i>workers</i>)	4
fp16 / bf16	desabilitado
<i>Gradient checkpointing</i>	desabilitado

Tabela 9 – Espaço de busca utilizado na otimização de hiperparâmetros com Optuna.

Hiperparâmetro	Intervalo	Distribuição
<i>learning_rate</i>	$1 \times 10^{-5} - 5 \times 10^{-5}$	log-uniforme
<i>weight_decay</i>	0,1 – 0,3	uniforme
<i>warmup_ratio</i>	0,3 – 0,5	uniforme

K-Fold Cross-Validation

Para avaliar a estabilidade do modelo e reduzir a dependência de uma única partição dos dados, foi empregada a validação cruzada k -fold com $k = 5$. Essa abordagem divide os dados (união das partições de treino e desenvolvimento) em cinco subconjuntos de tamanhos semelhantes. A cada iteração, um subconjunto é utilizado para avaliação e os demais para treinamento, alternando-se sucessivamente, conforme ilustrado na Figura 33. A divisão foi realizada com embaralhamento aleatório (`shuffle=True`) e semente fixada em 42 (`random_state=42`) para reprodutibilidade.

Ao final das cinco iterações, foi reportada a **média do LAS** como estimativa do desempenho esperado de cada configuração, e o *trial* com maior média foi selecionado como conjunto ótimo de hiperparâmetros para o treinamento final.

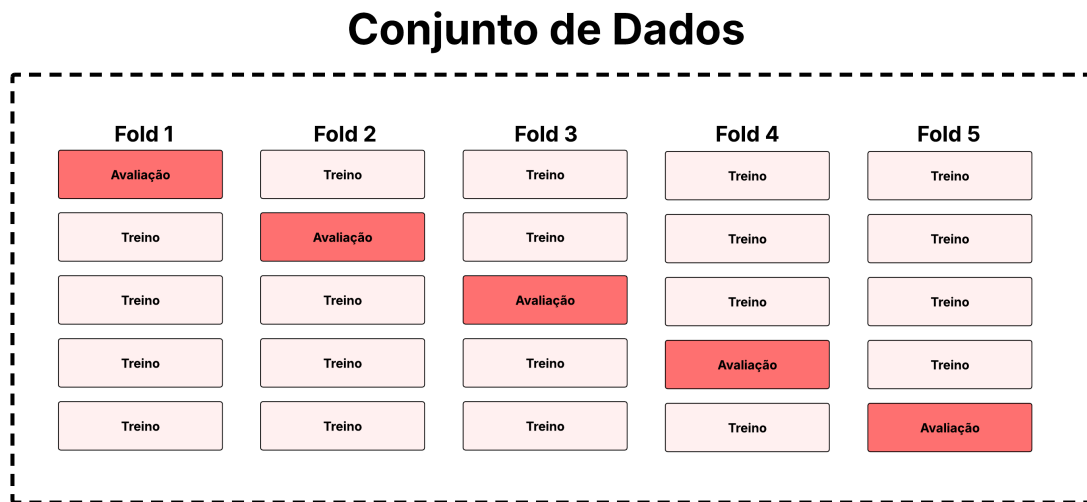


Figura 33 – Exemplo da aplicação da técnica de validação cruzada com $k = 5$.

Monitoramento

Para o monitoramento dos experimentos, foi utilizada a plataforma WandB⁶ (Weights & Biases, 2020). O WandB é uma ferramenta de MLOps (*Machine Learning Operations*) voltada ao rastreamento, visualização e gerenciamento de experimentos. A ferramenta permite acompanhar métricas de treinamento, funções de perda, consumo de memória, utilização de hardware e comportamento do processo de otimização ao longo das épocas, auxiliando na análise da estabilidade do treinamento e na identificação de problemas de convergência. A Figura 34 apresenta um exemplo da interface de monitoramento utilizada durante os experimentos.

Para garantir a reprodutibilidade dos experimentos, a semente aleatória foi fixada em 42 para todos os frameworks e bibliotecas utilizados, conforme apresentado na

⁶<https://wandb.ai>

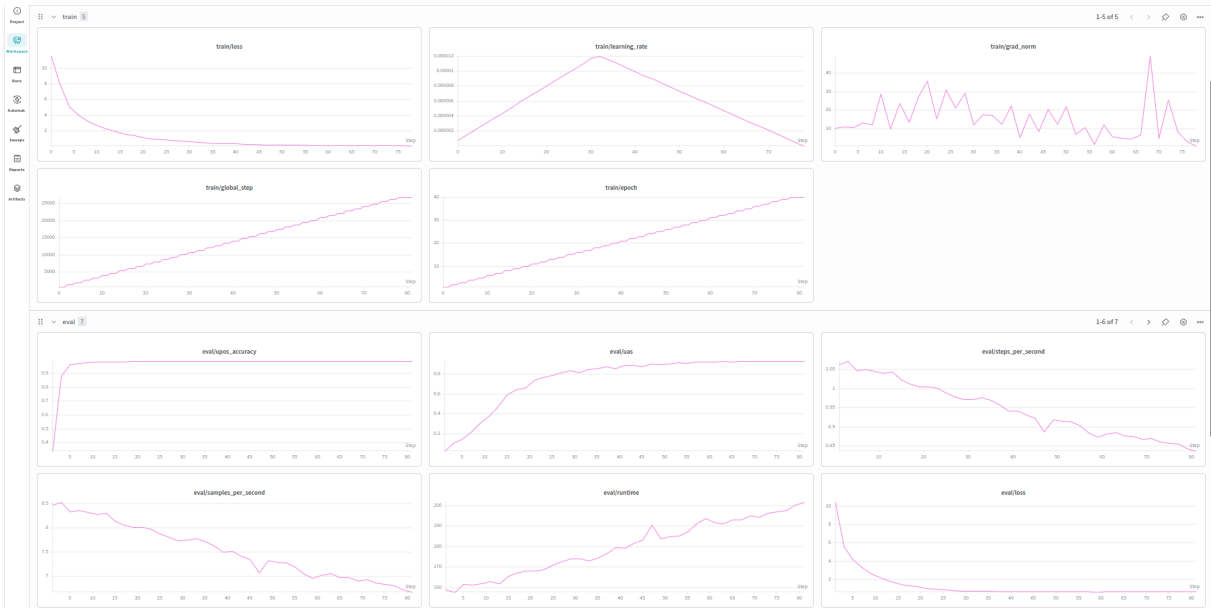


Figura 34 – Interface de monitoramento dos experimentos utilizando WandB.

Tabela 10 – Configurações de reprodutibilidade aplicadas a todos os experimentos.

Componente	Configuração
Semente (<i>seed</i>)	42
Python random	semente fixada
NumPy random	semente fixada
PyTorch (CPU)	semente fixada
PyTorch (CUDA)	semente fixada
CuDNN determinístico	habilitado
CuDNN <i>benchmark</i>	desabilitado
PYTHONHASHSEED	42
TOKENIZERS_PARALLELISM	false

Tabela 10.

Modelo Selecionado

A otimização de hiperparâmetros resultou nos valores apresentados na Tabela 11. Os valores correspondem ao *trial* com maior média de LAS no *k-fold* para cada combinação de encoder e arquitetura.

A Tabela 12 apresenta as configurações de hardware utilizadas durante a execução dos experimentos de busca de hiperparâmetros e treinamento final.

Entre as arquiteturas avaliadas, o modelo BERTimbau-Large com abordagem Linear MTL apresentou o melhor desempenho geral, considerando conjuntamente as métricas UPOS, UAS e LAS. Esse resultado sugere que o compartilhamento de representações aliado a camadas lineares específicas para cada tarefa foi suficiente para capturar padrões relevantes das estruturas sintáticas do corpus analisado.

Tabela 11 – Hiperparâmetros selecionados para cada modelo após otimização com Optuna.

Modelo	Learning Rate	Weight Decay	Warmup Ratio
mBERT (Linear)	$3,63 \times 10^{-5}$	0,197	0,421
mBERT (Biaffine)	$2,96 \times 10^{-5}$	0,238	0,328
BERTimbau-Base (Linear)	$4,42 \times 10^{-5}$	0,214	0,367
BERTimbau-Base (Biaffine)	$3,12 \times 10^{-5}$	0,129	0,410
BERTimbau-Large (Linear)	$3,76 \times 10^{-5}$	0,126	0,404
BERTimbau-Large (Biaffine)	$3,00 \times 10^{-5}$	0,116	0,417
JabuticaBERT (Linear)	$4,93 \times 10^{-5}$	0,150	0,435
JabuticaBERT (Biaffine)	$3,50 \times 10^{-5}$	0,158	0,303

Tabela 12 – Configuração de hardware utilizada nos experimentos.

Componente	Especificação
CPU	AMD Ryzen 9 5900X (12 núcleos, 24 threads)
RAM	128 GB
GPU	NVIDIA GeForce RTX 4090
VRAM	24 GB

4.3 Estado da Arte

Baseline: PortParser

Para fins de comparação experimental, foi selecionado como *baseline* o modelo PortParser (Lopes; Brisolara; Freitas, 2024), considerado uma das principais referências para *parsing* de dependências em português. O modelo foi avaliado no mesmo corpus Porttinari-base adotado neste trabalho, permitindo comparação direta. De acordo com os resultados reportados na literatura, o PortParser alcança 99,09% de acurácia em UPOS, 96,08% de UAS e 94,61% de LAS, demonstrando elevado desempenho em análise sintática para o português.

5 RESULTADOS E DISCUSSÃO

Este capítulo apresenta e discute os resultados obtidos pelos modelos de *parsing* de dependências desenvolvidos neste trabalho, avaliados no conjunto de teste do corpus Porttinari. A análise está organizada em cinco eixos principais: desempenho geral comparado ao PortParser; impacto da arquitetura do cabeçote de predição; efeito do MTL; análise de desempenho por rótulo de dependência; e dinâmica de instabilidade durante o treinamento. As definições completas dos rótulos de dependência citados ao longo do capítulo encontram-se no Apêndice .8. Os resultados detalhados da busca de hiperparâmetros e as curvas de treinamento por época de cada modelo estão disponíveis, respectivamente, nos Apêndices .9 e .10.

5.1 Desempenho Geral

A Tabela 13 e a Figura 35 apresentam os resultados de todos os modelos avaliados em termos de UAS, LAS e acurácia de UPOS. O melhor resultado obtido neste trabalho foi alcançado pelo modelo **BERTimbau Large com cabeçote linear e MTL** (*BtL-Lin-MTL*), com UAS de 94,99% e LAS de 93,84%.

Em relação ao **PortParser** (Lopes; Brisolara; Freitas, 2024), modelo de referência para o português brasileiro, o melhor resultado deste trabalho apresenta uma diferença de **1,09 pontos percentuais** em UAS e **0,77 pontos percentuais** em LAS. A principal diferença arquitetural que explica essa lacuna é a **ausência de uma camada BiLSTM bidirecional** no ParseH2IA: enquanto o PortParser insere uma camada BiLSTM entre o encoder e os cabeçotes de predição para contextualização adicional e realiza ajuste fino completo, o ParseH2IA conecta o encoder diretamente aos cabeçotes com ajuste fino completo de todos os parâmetros, sem camada intermediária. Essa arquitetura mais simples permite uma análise controlada da contribuição do cabeçote de predição isolado da influência do BiLSTM.

Nota-se que a lacuna de desempenho é consideravelmente pequena dada a supressão do BiLSTM: o BtL-Lin-MTL aproxima-se do PortParser a menos de 1,1 pontos percentuais em UAS, sugerindo que o BERTimbau Large com ajuste fino completo pro-

Tabela 13 – Comparação Baseline

Parser	UPOS	UAS	LAS
PortParser	99.09%	96.08%	94.61%
Nosso Parser-Bertimbau-Large (Linear MTL)	99.28%	94.99%	93.84%
Nosso Parser-Bertimbau-Large (Biaffine MTL)	99.34%	91.52%	90.56%
Nosso Parser-Bertimbau-Large (Biaffine Sem UPOS)	–	91.96%	90.85%
Nosso Parser-Bertimbau-Large (Linear Sem UPOS)	–	94.73%	93.52%
Nosso Parser-Bertimbau-Base (Linear MTL)	99.22%	93.74%	92.49%
Nosso Parser-Bertimbau-Base (Biaffine MTL)	99.27%	89.84%	88.71%
Nosso Parser-Bertimbau-Base (Linear Sem UPOS)	–	93.52%	92.23%
Nosso Parser-Bertimbau-Base (Biaffine Sem Upos)	–	90.81%	89.71%
Nosso Parser-mBert (Linear MTL)	98.88%	92.70%	91.04%
Nosso Parser-mBert (Biaffine MTL)	98.93%	86.92%	85.42%
Nosso Parser-mBert (Linear Sem UPOS)	–	92.44%	90.81%
Nosso Parser-mBert (Biaffine Sem UPOS)	–	89.11%	87.61%
Nosso Parser-JabuticaBERT (Linear MTL)	98.81%	87.78%	86.25%
Nosso Parser-JabuticaBERT (Biaffine MTL)	98.92%	88.94%	87.44%
Nosso Parser-JabuticaBERT (Linear Sem UPOS)	–	87.00%	85.29%
Nosso Parser-JabuticaBERT (Biaffine Sem UPOS)	–	90.04%	88.45%

Tabela 14 – Melhores resultados por modelo considerando as métricas UPOS, UAS e LAS.

duz representações suficientemente ricas para suportar o *parsing* diretamente, sem a camada de contextualização adicional.

Quanto à **acurácia de UPOS**, todos os modelos MTL superam o PortParser (99,09%), com valores entre 98,81% (JabuticaBERT Linear) e 99,34% (BERTimbau Large Biaffine). Esse resultado indica que a tarefa de classificação morfológica está próxima do limite superior de desempenho para o Porttinari, independentemente da arquitetura do cabeçote.

5.2 Comparação entre Encoders

A Figura 36 mostra as curvas de aprendizado (UAS no conjunto de teste por época) para os modelos com cabeçote linear e MTL de cada encoder. A hierarquia de desempenho final é consistente ao longo de praticamente todo o treinamento:

$$BERTimbauLarge > BERTimbauBase > mBERT > JabuticaBERT$$

O **BERTimbau Large** lidera em todas as configurações, confirmando que modelos pré-treinados em português com maior capacidade oferecem representações mais adequadas para a tarefa. O **BERTimbau Base**, treinado no mesmo corpus mas com menor capacidade, apresenta desempenho próximo ao Large, com diferença de apenas 1,25 pontos percentuais em UAS no melhor caso (94,99% vs. 93,74%).

O **mBERT**, apesar de ser multilíngue, supera o JabuticaBERT em todas as métri-

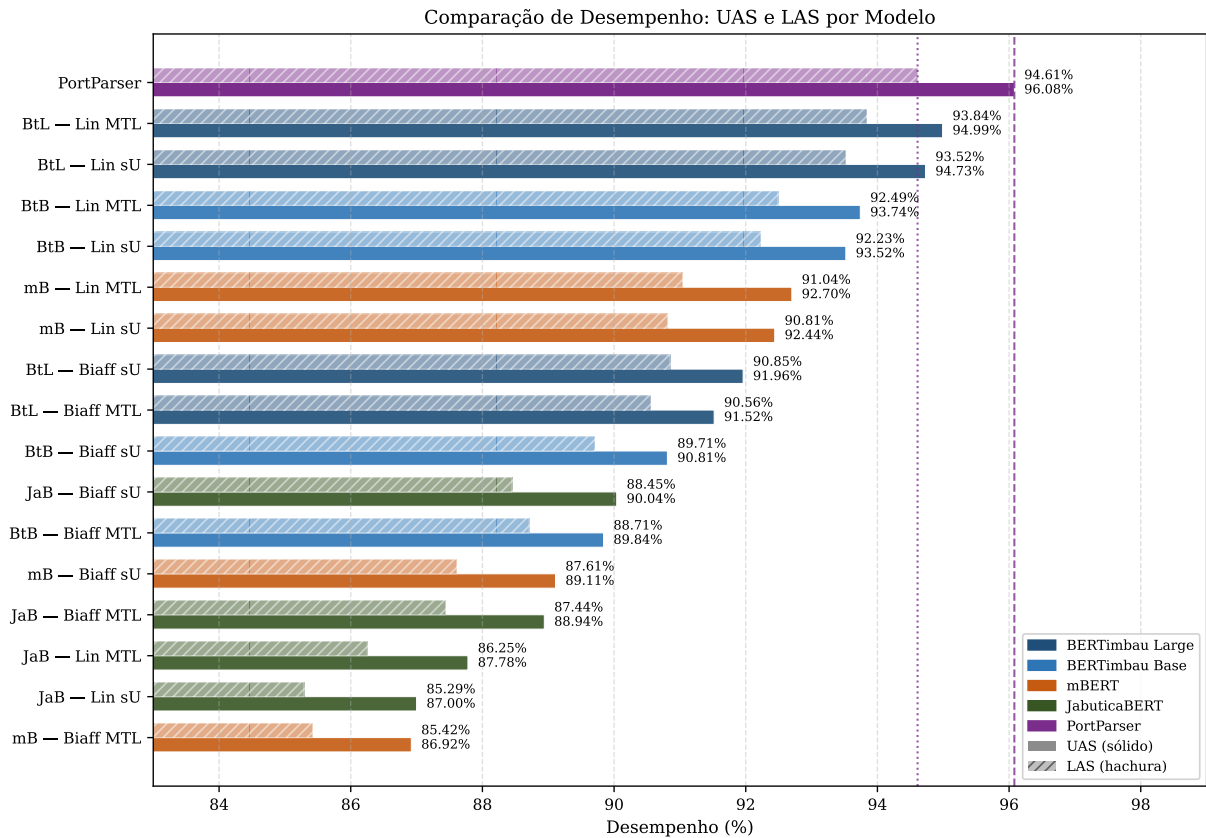


Figura 35 – Comparação de desempenho (UAS e LAS) de todos os modelos avaliados, ordenados por UAS decrescente. As linhas tracejadas indicam o desempenho do PortParser como referência (**UAS = 96,08%**, **LAS = 94,61%**). cU = com UPOS (MTL); sU = sem UPOS (ablação).

cas, exceto na variante Biaffine sem UPOS. Isso é notável, pois o JabuticaBERT é um modelo monolíngue para o português. A explicação mais provável está no tamanho e arquitetura do JabuticaBERT: trata-se de um modelo *Base* treinado com técnicas de aprendizado modernas (*continual pretraining*, vocabulário expandido), o que pode gerar uma distribuição de representações subótima para a interface direta com os cabeçotes de predição. Além disso, o JabuticaBERT utiliza um vocabulário maior e tokenização diferente do BERTimbau, gerando potencialmente mais subpalavras por token e introduzindo ruído na etapa de extração do primeiro subtoken por palavra.

As curvas de aprendizado revelam ainda um padrão de **convergência em duas fases** para os modelos com mais épocas: um crescimento rápido nas primeiras 20 épocas (fase de alta taxa de aprendizado, $\eta = 10^{-3}$), seguido de uma fase de refinamento mais lenta. As curvas completas por época, incluindo *training loss* e *validation loss*, estão disponíveis no Apêndice .10 e corroboram que todos os modelos convergiram sem sinais de sobreajuste grave na fase inicial, com a *validation loss* estabilizando antes do fim do treinamento. Esse comportamento é coerente com o esquema de taxa de aprendizado em dois estágios adotado durante o treinamento, cujos hiperparâmetros ótimos por encoder foram determinados pela busca apresentada no Apêndice .9.

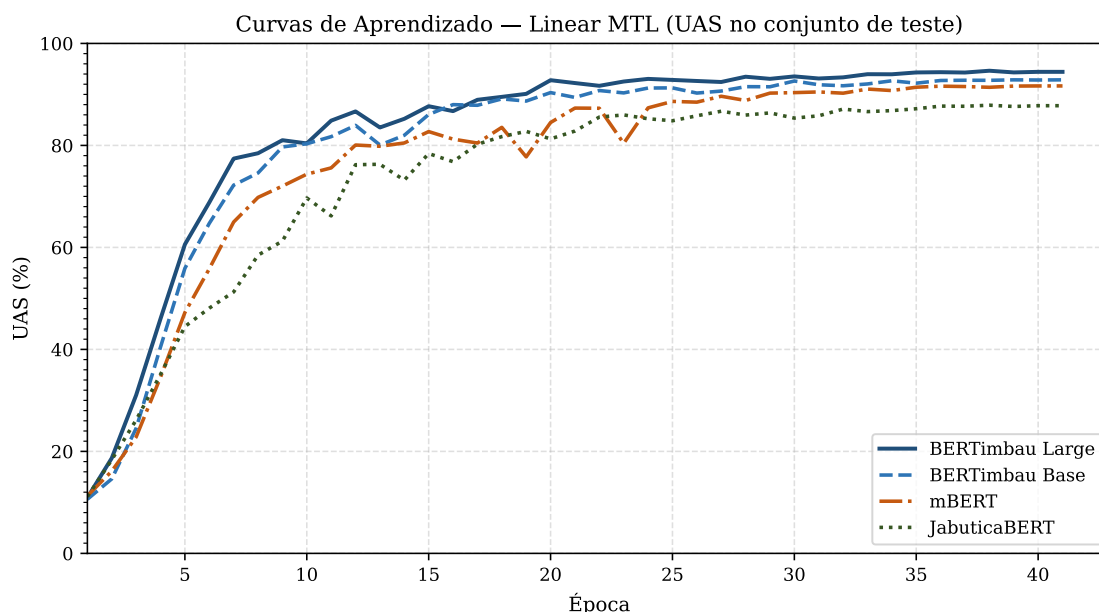


Figura 36 – Curvas de UAS por época para os modelos Linear MTL de cada encoder. O eixo vertical indica o UAS (%) no conjunto de teste ao final de cada época.

5.3 Impacto da Arquitetura do Cabeçote

5.3.1 Linear versus Biaffine

O resultado mais surpreendente deste trabalho é a **superioridade consistente dos cabeçotes lineares sobre os biaffines** em UAS e LAS (Figura 37). Em três dos quatro encoders (BtL, BtB e mBERT), o cabeçote linear supera o biaffine em UAS por uma margem substancial: 3,47 pontos para BERTimbau Large (94,99% vs. 91,52%), 3,90 pontos para BERTimbau Base (93,74% vs. 89,84%) e 5,78 pontos para mBERT (92,70% vs. 86,92%). A única exceção é o JabuticabERT, onde o Biaffine supera levemente o Linear (88,94% vs. 87,78% em UAS para a variante MTL, e 90,04% vs. 87,00% sem UPOS).

Vale observar que essa diferença não é artefato da escolha de hiperparâmetros: como mostra o Apêndice .9, a busca bayesiana de hiperparâmetros foi conduzida **independentemente** para cada combinação de encoder e arquitetura, e os hiperparâmetros ótimos do biaffine foram aplicados no treinamento final. Para o BERTimbau Large, por exemplo, os melhores hiperparâmetros do biaffine (Trial 0: $LR = 3,0 \times 10^{-5}$, $weight\ decay = 0,116$, $warmup\ ratio = 0,417$) produziram LAS médio de 0,887 na validação cruzada, substancialmente inferior ao LAS médio do linear (0,926, Trial 2). A diferença persiste no conjunto de teste final, confirmando que a superioridade do linear é sistemática e não decorre de sub-otimização do biaffine.

Na literatura, o cabeçote biaffine é amplamente reconhecido como superior para o *parsing* de dependências (Dozat; Manning, 2017), o que torna este resultado inesperado. Quatro hipóteses, complementares entre si, explicam esse comportamento no

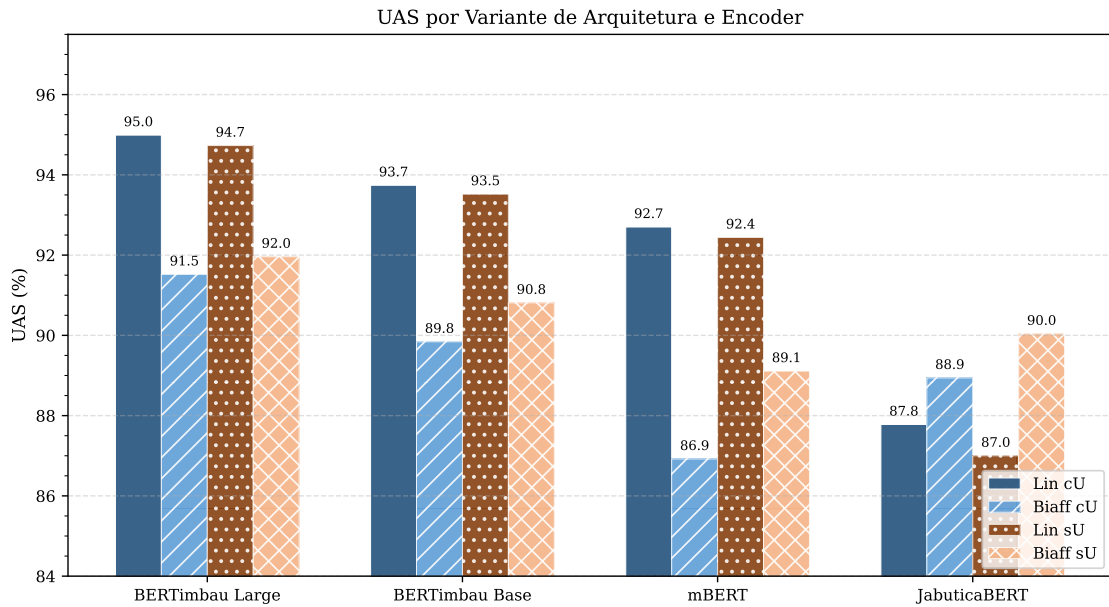


Figura 37 – UAS (%) por variante de arquitetura (Lin/Biaff × cU/sU) e encoder. A superioridade do cabeçote linear sobre o biaffine é consistente em três dos quatro encoders.

contexto específico deste trabalho:

1. **Codificação sintática implícita nas representações BERT:** Estudos de *probing* (Tenney; Das; Pavlick, 2019; Hewitt; Manning, 2019; Jawahar; Sagot; Seddah, 2019) demonstraram que modelos de linguagem pré-treinados como o BERT codificam, de forma implícita, informação sintática substancial em seus vetores de saída, incluindo relações de dependência, distâncias na árvore sintática e papéis gramaticais. Mesmo após o ajuste fino completo, essa organização sintática do espaço de representações tende a ser preservada (Hewitt; Manning, 2019). Nesse cenário, um cabeçote linear atua como uma *sonda linear* (*linear probe*) direta sobre a estrutura já organizada pelo encoder, aprendendo de forma eficiente e estável. O cabeçote biaffine, ao introduzir transformações MLP e bilineares adicionais, adiciona graus de liberdade que, em corpus pequeno, podem não convergir para uma solução melhor do que a leitura direta pelo linear. Essa hipótese é corroborada pelo fato de que a vantagem do linear é mais pronunciada nos encoders de maior capacidade (BERTimbau Large e Base), que tendem a codificar mais ricamente a estrutura sintática em suas representações.
2. **Corpus limitado e sobreajuste:** O cabeçote biaffine introduz substancialmente mais parâmetros treináveis: quatro MLPs (Linear + ELU + LayerNorm + Dropout) com $d_{arc} = 500$ e $d_{rel} = 100$, mais duas matrizes bilineares. Com apenas 5.893 sentenças de treinamento no Porttinari, essa capacidade adicional aumenta o risco de sobreajuste às particularidades do corpus, enquanto o cabeçote linear, com suas três projeções independentes, atua como regularização implícita.

3. Instabilidade de treinamento: Como será discutido na Seção 5.5, o cabeçote biaffine apresenta maior número de eventos de *forgetting* durante o treinamento. As curvas detalhadas no Apêndice .10 confirmam que os modelos biaffines exibem oscilações mais acentuadas entre *validation loss* e UAS ao longo das épocas, sugerindo que sua otimização é mais instável em corpus pequeno, o que prejudica o desempenho final.

A hipótese da codificação sintática implícita é particularmente relevante porque inverte a lógica da escolha arquitetural usual: ao invés de projetar um cabeçote mais expressivo para capturar relações complexas, o encoder com ajuste fino já capturou essas relações, e o papel do cabeçote é simplesmente lê-las, tarefa para a qual a simplicidade do linear é uma vantagem, não uma limitação, especialmente com corpus de treinamento restrito.

5.3.2 Impacto do Multi-Task Learning

A Figura 38 mostra a diferença de UAS e LAS entre os modelos com e sem a tarefa auxiliar de UPOS (MTL versus ablação). O impacto geral é **pequeno e assimétrico**: o MTL beneficia consistentemente os modelos lineares (ganho médio de +0,33 pp em UAS para o BERTimbau Large), mas prejudica os modelos biaffines (perda média de -1,10 pp em UAS), com a exceção do JabuticaBERT Biaffine.

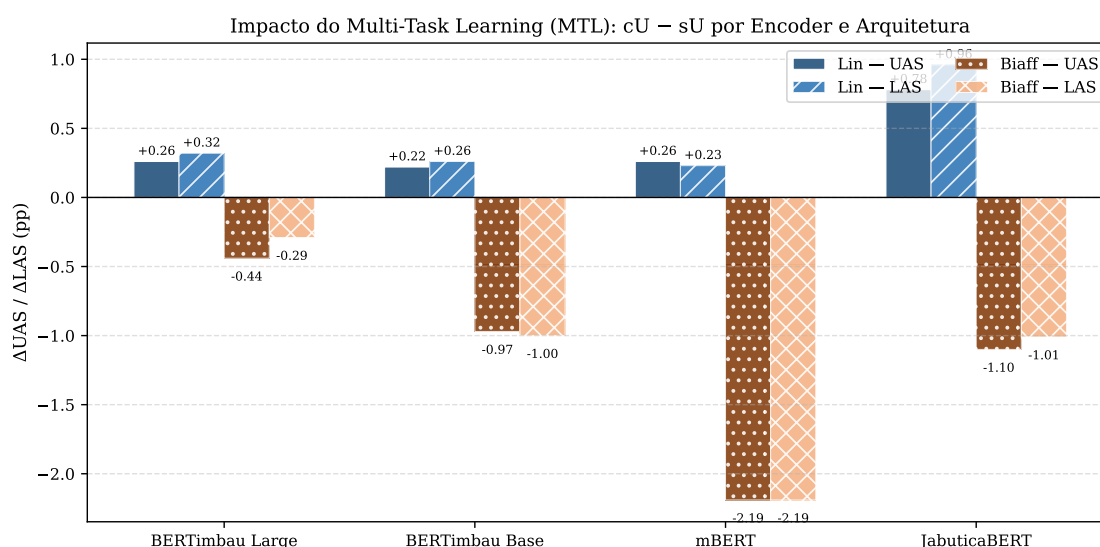


Figura 38 – Diferença de UAS e LAS (em pontos percentuais) entre modelos MTL (com UPOS) e de ablação (sem UPOS), por variante e encoder. Valores positivos indicam que o MTL beneficia o *parsing*; negativos indicam prejuízo.

Esse resultado sugere que a tarefa de UPOS, já próxima do teto de desempenho (>99%), adiciona pouca informação de gradiente útil ao ajuste fino do encoder sobre o corpus Portinari. O ganho marginal nos modelos lineares pode decorrer de uma leve regularização implícita introduzida pela tarefa auxiliar, enquanto o prejuízo nos

modelos biaffines pode resultar de um conflito de gradientes: as tarefas de HEAD e UPOS competem pelos parâmetros das MLPs com sinais de grandezas muito diferentes, prejudicando a convergência do mecanismo biaffine.

5.4 Análise por Rótulo de Dependência

A análise por rótulo DEPREL (Tabelas 58–61 no Apêndice .12) revela uma variação expressiva de desempenho entre rótulos. As definições de cada rótulo citado nesta seção encontram-se no Apêndice .8.

5.4.1 Rótulos de Alta Acurácia

Os rótulos *punct* (0,9982–0,9991), *case* (0,9956–0,9982) e *cop* (0,9849–0,9983) apresentam alta acurácia consistente em todos os modelos. Esses rótulos correspondem a palavras com forte restrição gramatical: pontuação, preposições/posposições e verbos de ligação têm pouca ambiguidade de dependência e são frequentes no Portinari (para suas definições, ver Apêndice .8). A alta frequência de ocorrência fornece mais exemplos de treinamento, enquanto a baixa ambiguidade resulta em padrões altamente previsíveis. Esses são também os rótulos mais facilmente extraíveis por um cabeçote linear, pois a informação sintática correspondente tende a estar fortemente codificada nas representações do BERT, reforçando a hipótese de leitura direta discutida anteriormente.

5.4.2 Rótulos Difíceis

A Figura 39 apresenta os rótulos com menor acurácia nos quatro melhores modelos. Os rótulos *orphan*, *dislocated*, *vocative* e *parataxis* são consistentemente os mais desafiadores (ver Apêndice .8 para definições detalhadas).

5.4.2.0.1 *orphan* e *dislocated*.

Com acurácia abaixo de 50% na maioria dos modelos, esses rótulos são raros no Portinari (14 e 17 ocorrências no conjunto de teste, respectivamente) e semanticamente complexos: *orphan* marca dependentes de um núcleo elidido em estruturas de elipse, enquanto *dislocated* é usado em deslocamentos sintáticos (ver Apêndice .8). A raridade extrema impede o aprendizado de padrões robustos, e a natureza não-local dessas construções dificulta sua codificação implícita pelo BERT, que opera com viés de proximidade na atenção.

5.4.2.0.2 *parataxis* e *discourse*.

Esses rótulos codificam relações semanticamente difusas, com acurácia variando de 62,75% a 78,12% para *parataxis* e de 56,86% a 74,51% para *discourse*. Ambos

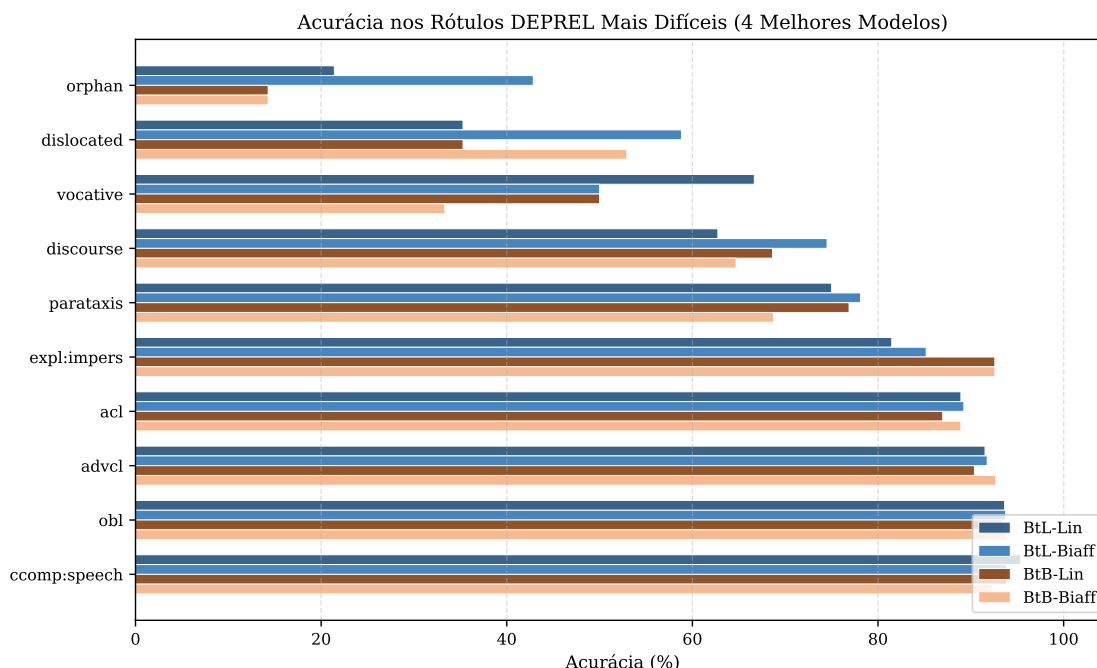


Figura 39 – Acurácia (%) nos rótulos DEPREL mais difíceis para os quatro melhores modelos. *orphan* e *dislocated* apresentam desempenho inferior a 50% na maioria dos modelos.

são frequentemente confundidos entre si e com outros rótulos de alta abstração (e.g., *advcl*, *ccomp*). Trata-se de relações que dependem de conhecimento discursivo e pragmático que o BERT capta apenas parcialmente via *masked language modeling*, e que não são recuperáveis linearmente das representações de tokens isolados.

5.4.2.0.3 *acl* e *advcl*.

Com acurácia em torno de 89–93%, as cláusulas adverbiais e adnominais apresentam dificuldade moderada, refletindo a ambiguidade de adjunção em português: uma mesma construção pode ser analisada como modificador nominal (*acl*) ou adverbial (*advcl*) dependendo do contexto discursivo. Seus contornos de decisão envolvem informação semântica que transcende o nível local da sentença.

5.4.2.0.4 *nmod* e *obl*.

Os rótulos nominais modificadores apresentam acurácia entre 93–96%, mas são os que geram mais eventos de *forgetting* durante o treinamento (ver Seção 5.5), o que sugere que o modelo aprende padrões para esses rótulos, mas os perde em ajustes posteriores de gradiente. A confusão frequente entre *nmod* e *obl*, ambos expressando nominais associados a predicados ou outros nominais via preposição, com distinção dependente de papéis temáticos, é coerente com a dificuldade de codificação linear desse contraste (ver definições no Apêndice .8).

A análise por rótulo revela ainda que a diferença entre modelos é mais pronunciada nos rótulos de dificuldade moderada (90–95%) do que nos extremos (rótulos fáceis

>99% ou muito difíceis <50%), onde a tarefa é inerentemente trivial ou insolúvel sem mais dados de treinamento.

5.5 Análise de Instabilidade do Treinamento

A análise de instabilidade, realizada com base nas previsões por época sobre o conjunto de teste, quantifica dois fenômenos distintos: **forgetting** (transição de acerto para erro em épocas consecutivas) e **recovery** (transição inversa). Esses fenômenos indicam que o modelo não converge monotonicamente: certos tokens são classificados corretamente em épocas intermediárias mas perdem essa classificação em ajustes posteriores de gradiente.

5.5.1 Curvas de Aprendizado do BERTimbau Large

A Figura 40 mostra a progressão do UAS por época para as quatro variantes do BERTimbau Large. Observa-se que o **cabeçote linear** converge de forma mais suave e estável, enquanto o cabeçote **biaffine** apresenta oscilações mais pronunciadas nas primeiras épocas, compatível com a maior instabilidade quantificada na análise de *forgetting*. As tabelas de treinamento completas por época, com UAS, LAS, UPOS, *training loss* e *validation loss* para cada modelo, estão disponíveis no Apêndice .10, permitindo verificar em detalhe os pontos de inflexão e as épocas de maior volatilidade de cada variante.

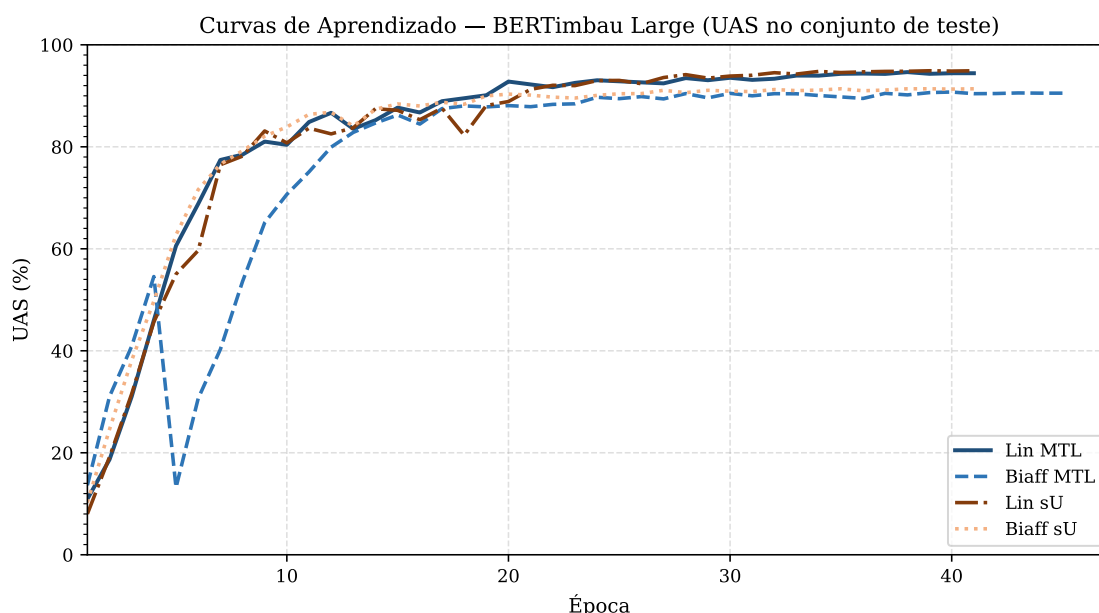


Figura 40 – Curvas de UAS por época para as quatro variantes do BERTimbau Large. A variante Linear MTL converge de forma mais estável que a Biaffine MTL, que registra mais oscilações nas primeiras 20 épocas.

Nota-se ainda que a variante Linear sem UPOS (sU) apresenta UAS final ligeiramente inferior ao Linear MTL (+0,26 pp), confirmando o benefício marginal da tarefa

auxiliar para o cabeçote linear. Para o Biaffine, o comportamento é inverso: a variante sem UPOS alcança UAS maior que a MTL, sugerindo que a tarefa de UPOS introduz conflito de gradientes nessa configuração.

5.5.2 Instabilidade por Rótulo DEPREL

A Figura 41 apresenta o número de eventos de *forgetting* para os 15 rótulos DEPREL mais instáveis do BERTimbau Large. Os resultados complementam a análise de acurácia: enquanto rótulos como *parataxis* e *discourse* têm baixa acurácia por dificuldade intrínseca, rótulos como *nmod* e *obl* têm acurácia razoável (93–96%) mas alta instabilidade, o que indica que o modelo os aprende e desaprende repetidamente durante o treinamento. Isso sugere que as fronteiras de decisão para esses rótulos são próximas de outros rótulos de alta frequência no espaço de representações, o que os torna vulneráveis a deslocamentos de gradiente em épocas subsequentes.

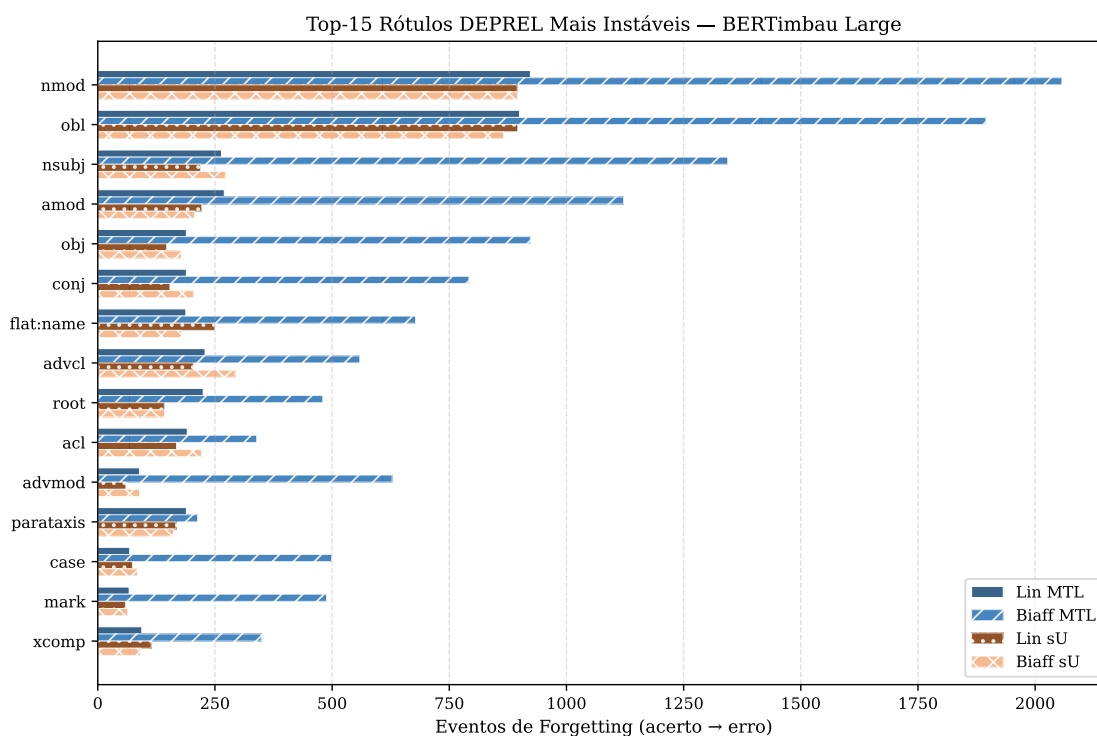


Figura 41 – Número de eventos de *forgetting* (acerto → erro) por rótulo DEPREL para as quatro variantes do BERTimbau Large. Rótulos de alta frequência com fronteiras de decisão difusas (*nmod*, *obl*) são os mais instáveis.

O rótulo *nmod* (modificador nominal) registra o maior número de *forgettings* em praticamente todas as variantes, com até 2.057 eventos na variante BtL-Biaff-MTL. Isso se deve à alta ambiguidade de adjunção em português: sintagmas preposicionados podem ser analisados como *nmod*, *obl* ou *advcl*, e a decisão correta depende frequentemente de informação semântica não captada pelo cabeçote sintático. O rótulo *obl* (oblíquo nominal) é o segundo mais instável, com padrão semelhante (para as definições formais desses rótulos, ver Apêndice .8).

A comparação entre variantes revela que a **variante Biaffine MTL é consistentemente a mais instável**, com totais de *forgetting* superiores às variantes Linear em todos os rótulos analisados, exceto *conj* e *root*. Isso é coerente com a hipótese de que o cabeçote biaffine, com maior capacidade e dependência de geometria das representações, oscila mais ao longo do treinamento em corpus de tamanho limitado, e também com a hipótese da codificação implícita: o cabeçote linear, ao operar como uma sonda (*probe*) direta sobre representações já organizadas sintaticamente pelo encoder, aprende uma fronteira de decisão mais estável e menos suscetível a oscilações de gradiente.

A análise de instabilidade das Tabelas ??-?? no Apêndice .12 mostra ainda que a diferença de instabilidade entre variantes com e sem UPOS é pequena para o cabeçote linear (o MTL não aumenta nem reduz significativamente o *forgetting*), mas é mais expressiva para o biaffine (o MTL aumenta a instabilidade em alguns rótulos).

5.6 Discussão Geral

Os resultados apresentados permitem extrair as seguintes conclusões:

5.6.0.0.1 1. A ausência do BiLSTM é o principal fator diferencial em relação ao PortParser.

A lacuna de 1,09 pp em UAS em relação ao PortParser não é explicada pela diferença de arquitetura do cabeçote, mas pela ausência da camada BiLSTM bidirecional que o PortParser utiliza entre o encoder e o cabeçote biaffine. As curvas de treinamento no Apêndice .10 mostram que os modelos convergem sem sinais de sobreajuste grave, indicando que o teto de desempenho observado é imposto pela ausência da contextualização adicional fornecida pelo BiLSTM, não pela capacidade de generalização do cabeçote. Trabalhos futuros com a adição de um BiLSTM ao ParseH2IA devem reduzir substancialmente essa lacuna.

5.6.0.0.2 2. O cabeçote linear é mais eficaz em corpus pequeno com ajuste fino completo, possivelmente por explorar a codificação sintática implícita do BERT.

A superioridade consistente do cabeçote linear sobre o biaffine em três dos quatro encoders, mesmo em regime de ajuste fino completo, constitui o principal achado empírico deste trabalho. A interpretação mais plausível, à luz dos resultados de instabilidade e da literatura de *probing*, combina dois mecanismos. Primeiro, estudos demonstram que modelos de linguagem pré-treinados codificam implicitamente estrutura sintática em suas representações (Tenney; Das; Pavlick, 2019; Hewitt; Manning, 2019): relações de dependência, distâncias na árvore sintática e papéis gramaticais

estão linearmente acessíveis no espaço vetorial produzido pelo encoder. Um cabeçote linear funciona essencialmente como uma sonda direta (*linear probe*) sobre essa informação codificada, aprendendo de forma eficiente e estável. O cabeçote biaffine, com suas quatro MLPs e operações bilineares, adiciona graus de liberdade que não produzem ganho quando a informação relevante já está linearmente acessível. Segundo, com apenas 5.893 sentenças de treino no Porttinari, a capacidade paramétrica adicional do biaffine se manifesta como maior risco de sobreajuste em vez de melhor generalização. A conclusão prática é que, ao trabalhar em corpus pequeno e sem BiLSTM, arquiteturas de cabeçote mais simples não apenas bastam, elas são preferíveis. Essa conclusão é reforçada pela consistência dos resultados da busca de hiperparâmetros (Apêndice .9): mesmo com configurações otimizadas independentemente, o biaffine não alcança o linear em três dos quatro encoders.

5.6.0.0.3 3. O aprendizado multi-tarefa tem impacto marginal no *parsing*.

A tarefa auxiliar de UPOS, já próxima do teto de desempenho, não fornece gradiente suficiente para melhorar substancialmente o desempenho em HEAD e DEPREL no Porttinari. O ganho marginalmente positivo para os modelos lineares (+0,33 pp em UAS em média) e o impacto ligeiramente negativo para os biaffines (−1,10 pp em média) sugerem que o sinal UPOS interage de forma distinta com as duas arquiteturas de cabeçote: para o linear, atua como leve regularizador; para o biaffine, pode introduzir conflito de gradiente entre as MLPs da tarefa principal e o classificador de UPOS.

5.6.0.0.4 4. JabuticaBERT apresenta desempenho abaixo do esperado.

A suposição de que um modelo pré-treinado exclusivamente em português deveria superar o BERTimbau em tarefas para o português não se confirma. Diferenças de tokenização e arquitetura do vocabulário podem estar influenciando negativamente a extração de representações por subtoken nesta configuração. Adicionalmente, a hipótese da codificação implícita sugere que a qualidade da codificação sintática, e não apenas a língua do pré-treinamento, é determinante; nesse sentido, o BERTimbau Large, por sua maior capacidade e treinamento, codificaria sintaxe de forma mais rica que o JabuticaBERT Base. Nota-se também que os resultados da busca de hiperparâmetros (Apêndice .9) mostram maior variância (*std LAS*) para o JabuticaBERT em comparação aos demais encoders, o que indica maior sensibilidade aos hiperparâmetros, possivelmente reflexo da mesma instabilidade de representações observada.

5.6.0.0.5 5. Rótulos de alta ambiguidade semântica permanecem desafiadores.

parataxis, *discourse* e *orphan* apresentam acurácia baixa e alta variabilidade entre modelos, sugerindo que esses rótulos requerem informação discursiva e contextual que vai além do que um cabeçote sintático treinado sobre o nível de sentença

pode capturar, e que, diferentemente dos rótulos gramaticalmente determinísticos, não está linearmente codificada nas representações do encoder (ver definições formais no Apêndice .8).

5.7 Limitações

Este trabalho apresenta as seguintes limitações que devem ser consideradas na interpretação dos resultados:

- **Corpus único:** Os modelos foram treinados e avaliados exclusivamente no corpus Porttinari, o que restringe a generalização dos resultados a outros domínios e gêneros textuais. Avaliação em corpora de notícias, redes sociais ou textos jurídicos pode produzir resultados distintos.
- **Ausência de BiLSTM:** O ParseH2IA suprime a camada BiLSTM bidirecional presente no PortParser, o que impede comparação direta em condições arquiteturais idênticas. Os resultados refletem a escolha deliberada de avaliar o cabeçote de predição sem a influência da contextualização adicional fornecida pelo BiLSTM. Trabalhos futuros que adicionem o BiLSTM ao ParseH2IA permitirão isolar sua contribuição.
- **Ausência de decodificação estrutural:** Os modelos implementados utilizam decodificação gulosa (*greedy*), sem garantir que a estrutura de árvore seja válida. O PortParser e outros modelos de estado da arte utilizam algoritmos de decodificação exata (e.g., algoritmo de Eisner) para garantir arborescências projetivas válidas, o que pode explicar parte da diferença de desempenho.
- **Avaliação de uma única execução:** Os resultados reportados correspondem a uma única execução de treinamento por configuração. Experimentos com múltiplas sementes aleatórias permitiriam estimar a variância dos resultados e tornar as comparações estatisticamente mais robustas.
- **Hiperparâmetros fixos por arquitetura:** A busca de hiperparâmetros foi conduzida individualmente para cada combinação de encoder e arquitetura (Apêndice .9), replicando os intervalos de busca do PortParser original. Uma busca com espaço ampliado poderia beneficiar especialmente o cabeçote biaffine, que demonstrou maior sensibilidade aos hiperparâmetros.

Apesar dessas limitações, os resultados demonstram que é possível construir modelos de *parsing* competitivos para o português brasileiro com arquiteturas relativamente simples, desde que se utilize um encoder pré-treinado de alta qualidade como o BERTimbau Large. A hipótese de codificação sintática implícita, em particular, oferece

uma contribuição interpretativa relevante ao sugerir que a superioridade do cabeçote linear não é um artefato experimental, mas reflexo de uma propriedade fundamental dos encoders BERT: a organização sintática latente de suas representações.

6 CONCLUSÃO

Este trabalho teve como objetivo desenvolver e avaliar o sistema **ParseH2IA**, um *parser* de dependências sintáticas para o português brasileiro baseado em encoders BERT com ajuste fino completo, investigando o impacto de diferentes encoders pré-treinados e de duas arquiteturas de cabeçote de predição no desempenho das tarefas de HEAD, DEPREL e UPOS. Foram treinados e avaliados 16 modelos resultantes da combinação de quatro encoders, BERTimbau Large, BERTimbau Base, mBERT e JabuticabERT, com duas arquiteturas de cabeçote (linear e biaffine) e dois regimes de treinamento (multi-tarefa com UPOS e ablação sem UPOS), todos avaliados no corpus Porttinari.

6.1 Sobre a Arquitetura Proposta

É importante destacar a diferença entre a arquitetura do PortParser e a proposta pelo ParseH2IA. O PortParser (Lopes; Brisolara; Freitas, 2024) utiliza um encoder BERTimbau Large seguido de uma camada BiLSTM bidirecional e cabeçotes biaffine para HEAD e DEPREL, com ajuste fino completo de todos os parâmetros. O ParseH2IA, por sua vez, **suprime o BiLSTM** e conecta o encoder diretamente às cabeças de predição, adotando duas variantes:

- **Cabeçote Linear:** o tensor de saída do encoder (após dropout) é projetado diretamente por três camadas `nn.Linear` independentes para HEAD (100 classes), DEPREL (44 rótulos) e UPOS (N classes). A predição do arco HEAD é, portanto, tratada como classificação multiclasse sobre posições relativas da sentença.
- **Cabeçote Biaffine:** o tensor de saída do encoder é processado por quatro MLPs especializadas (Linear + ELU + LayerNorm + Dropout, $d_{arc} = 500$, $d_{rel} = 100$), seguidas por dois mecanismos de atenção biaffine, um para arc/HEAD ($out = 1$) e outro para rel/DEPREL ($out = 44$), conforme proposto por Dozat; Manning (2017). A tarefa de UPOS utiliza uma camada linear direta sobre a saída do encoder, sem passar pelas MLPs.

Em ambas as variantes, **todos os parâmetros do encoder são ajustados durante o treinamento** (*full fine-tuning*), incluindo os pesos do BERT. A diferença arquitetural está inteiramente no cabeçote de predição.

6.2 Sobre o Conjunto de Dados

Todos os modelos foram treinados e avaliados no corpus **Porttinari** (Duran; Lopes; Graças Nunes; Pardo, 2023), um *treebank* de dependências para o português brasileiro desenvolvido no padrão Universal Dependencies (UD). O corpus foi dividido nas partições padrão utilizadas ao longo do trabalho: **5.893 sentenças** de treinamento, **842 de validação** e **1.683 de teste** (33.580 tokens no conjunto de teste).

O tamanho do corpus de treinamento é um fator relevante para a interpretação dos resultados. Com menos de seis mil sentenças de treino, o Porttinari representa um cenário de dados limitados (*low-resource*) mesmo para modelos com ajuste fino completo. Esse contexto favorece modelos com menor número de parâmetros treináveis no cabeçote, pois a capacidade adicional do mecanismo biaffine, que inclui quatro MLPs e duas matrizes bilineares, introduz um risco maior de sobreajuste ao vocabulário de construções do corpus em comparação ao cabeçote linear, que projeta diretamente a saída do encoder com três camadas simples.

O Porttinari apresenta ainda uma distribuição altamente assimétrica de rótulos de dependência: rótulos frequentes como *det*, *nsubj* e *punct* possuem milhares de ocorrências no treino, enquanto rótulos raros como *orphan*, *dislocated* e *vocative* contam com menos de 50 exemplos no conjunto de teste, o que explica o baixo desempenho consistente do sistema nesses rótulos independentemente da arquitetura do cabeçote. Ampliar o Porttinari com dados adicionais de outros corpora ou domínios textuais representaria ganho direto na cobertura dos rótulos minoritários.

6.3 Custo Computacional

Todos os modelos foram treinados em uma única GPU **NVIDIA GeForce RTX 4090** (24 GB de VRAM), com *batch size* de 16 amostras por dispositivo no treino e 32 na avaliação. O monitoramento dos experimentos foi realizado via **Weights & Biases (WandB)**, no projeto *hf-optuna* da entidade *gdlima-universidade-federal-de-pelotas*, disponível para consulta dos dados de treinamento de todos os modelos desenvolvidos neste trabalho.

A Tabela 15 apresenta um resumo do custo computacional por fase do experimento, com os dados extraídos do WandB. Os valores de tempo por época referem-se à média das últimas 10 épocas de treinamento, excluindo a fase inicial de *warmup* onde a variância é maior.

Tabela 15 – Custo computacional estimado por fase experimental (GPU: RTX 4090). Valores extraídos do WandB (hf-optuna).

Fase	Modelos	Tempo médio/época	Épocas
Busca Optuna (linear)	4 encoders × 10 trials	<i>ver WandB</i>	40
Busca Optuna (biaffine)	4 encoders × 10 trials	<i>ver WandB</i>	40
Treino final (linear)	8 modelos	<i>ver WandB</i>	40
Treino final (biaffine)	8 modelos	<i>ver WandB</i>	40

Em termos relativos, os modelos com cabeçote biaffine apresentam maior custo de inferência por *forward pass*, pois as quatro MLPs e as operações bilineares introduzem multiplicações matriciais adicionais em comparação ao cabeçote linear. Contudo, dado que o gargalo computacional no ajuste fino de modelos BERT concentra-se no encoder (auto-atenção com complexidade $\mathcal{O}(n^2)$ no comprimento da sequência), a diferença prática de tempo de treinamento entre os dois cabeçotes é pequena e se torna mais relevante apenas na inferência em produção.

A busca de hiperparâmetros com 10 *trials* por encoder e por arquitetura via Optuna, com validação cruzada em 5 partições, representou a etapa de maior custo computacional total: cada *trial* exige 5 treinamentos completos, resultando em 50 execuções de 40 épocas por combinação encoder × cabeçote. O custo total da fase de busca, portanto, supera em ordem de magnitude o custo dos treinamentos finais.

6.4 Síntese dos Resultados

O melhor desempenho foi alcançado pela configuração **BERTimbau Large com cabeçote linear e aprendizado multi-tarefa** (BtL-Lin-MTL), com UAS de 94,99%, LAS de 93,84% e acurácia de UPOS de 99,28%, representando uma diferença de 1,09 pontos percentuais em UAS em relação ao PortParser (96,08%). Essa lacuna é atribuível às diferenças arquiteturais entre os dois sistemas: o PortParser utiliza BiLSTM sobre o encoder como camada intermediária de contextualização adicional e cabeçote biaffine otimizado para essa representação, enquanto o ParseH2IA opera sem o BiLSTM, com cabeçote linear diretamente sobre a saída do encoder.

Os principais achados são:

1. **Cabeçote linear supera o biaffine em três dos quatro encoders:** com margens de até 5,78 pp em UAS (mBERT), o cabeçote linear demonstrou maior eficácia no cenário de ajuste fino completo sobre o Porttinari. A hipótese central é que, com corpus pequeno (5.893 sentenças de treino), o cabeçote linear generaliza melhor por introduzir menos parâmetros e menos oportunidades de sobreajuste, enquanto o biaffine, apesar de mais expressivo, não consegue aproveitar essa expressividade por insuficiência de dados supervisionados.

2. **BERTimbau Large produz as melhores representações:** a hierarquia de desempenho $BtL > BtB > mBERT > JaB$ foi consistente ao longo de todo o treinamento, confirmando que encoders maiores e pré-treinados especificamente em português oferecem representações mais adequadas para o *parsing*.
3. **MTL tem impacto marginal:** o ganho médio do aprendizado multi-tarefa com UPOS sobre a ablação sem UPOS foi de apenas +0,33 pp em UAS para os modelos lineares e negativo ($-1,10$ pp em média) para os biaffines. O UPOS, próximo ao teto de desempenho, adiciona pouco sinal discriminativo ao ajuste fino.
4. **JabuticaBERT ficou abaixo do esperado:** o modelo monolíngue para o português não superou o mBERT multilíngue em três das quatro variantes, possivelmente em razão de diferenças de tokenização e de volume de dados de pré-treinamento disponíveis para o corpus Porttinari especificamente.
5. **A análise de instabilidade revela dinâmicas ocultas:** o cabeçote biaffine apresenta sistematicamente mais eventos de *forgetting* por época, e rótulos de alta frequência com fronteiras difusas (*nmod*, *obl*) concentram a maior parte da instabilidade.

6.5 Contribuições

6.5.0.0.1 Contribuição empírica.

Este trabalho fornece a primeira comparação sistemática entre cabeçotes linear e biaffine, sem BiLSTM intermediário, com ajuste fino completo do encoder, para quatro encoders distintos, no corpus Porttinari. Os resultados documentam de forma controlada que a vantagem do biaffine sobre o linear, amplamente assumida na literatura, não se mantém em corpora de tamanho limitado quando o BiLSTM é suprimido. A comparação independente de hiperparâmetros por configuração (Apêndice .9) garante que as conclusões não são artefato de sub-otimização.

6.5.0.0.2 Contribuição metodológica.

A metodologia de análise de instabilidade proposta — que rastreia previsões por época e quantifica eventos de *forgetting* e *recovery* por rótulo e por token, oferece uma perspectiva complementar à avaliação por métricas finais. Ela permite distinguir rótulos difíceis por raridade ou ambiguidade intrínseca daqueles difíceis por instabilidade de otimização, informação diretamente útil para orientar decisões arquiteturais e de treinamento.

6.6 Trabalhos Futuros

6.6.0.0.1 Inclusão do BiLSTM no ParseH2IA.

A principal diferença arquitetural entre o ParseH2IA e o PortParser é a ausência do BiLSTM. Avaliar sistematicamente o impacto da adição de uma camada BiLSTM bidirecional entre o encoder e os cabeçotes, na mesma estrutura de comparação linear/biaffine, permitiria isolar a contribuição dessa camada de contextualização adicional sobre o desempenho.

6.6.0.0.2 Ampliação e diversificação do corpus.

Treinar e avaliar o ParseH2IA em outros *treebanks* disponíveis para o português, como o GSD ou corpora de domínios específicos (jurídico, jornalístico, redes sociais), permitiria avaliar a robustez das conclusões para além do Porttinari e verificar se a vantagem do cabeçote linear se mantém em distribuições de dados distintas ou com volumes maiores de treinamento.

6.6.0.0.3 Decodificação estrutural.

A substituição da decodificação gulosa (*greedy*) por algoritmos de decodificação exata, como o algoritmo de Eisner para árvores projetivas (Eisner, 1996), garantiria que as predições constituem arborescências válidas, eliminando uma fonte de diferença em relação ao PortParser e potencialmente melhorando o LAS em construções ambíguas.

6.6.0.0.4 Validação estatística.

Experimentos com múltiplas sementes aleatórias permitiriam calcular intervalos de confiança para as métricas reportadas, tornando as comparações entre modelos estatisticamente mais rigorosas, especialmente relevante para as diferenças pequenas observadas entre variantes MTL e ablação.

6.6.0.0.5 Exploração do custo computacional via WandB.

Os dados de treinamento registrados no WandB (*hf-optuna*) permitem uma análise quantitativa detalhada de custo por configuração, tempo de GPU por época, memória utilizada, throughput de amostras, que pode orientar a escolha de arquitetura em cenários com restrições de hardware.

6.7 Considerações Finais

Esta dissertação demonstrou que é possível construir sistemas de *parsing* de dependências competitivos para o português brasileiro sem o BiLSTM intermediário pre-

sente no PortParser, utilizando cabeçotes de predição conectados diretamente ao encoder com ajuste fino completo. O ParseH2IA atingiu 94,99% de UAS com a configuração BERTimbau Large + cabeçote linear + MTL, a menos de 1,1 pontos percentuais do estado da arte.

O achado mais significativo, a superioridade do cabeçote linear sobre o biaffine em corpus de tamanho limitado, sugere que a escolha arquitetural não deve ser guiada apenas pelos resultados da literatura em cenários com corpora maiores. No Porttinari, com menos de seis mil sentenças de treino, a expressividade adicional do biaffine não se traduz em ganho de desempenho: a linearidade da projeção direta sobre o encoder é mais eficaz e mais estável.

A análise de instabilidade introduzida neste trabalho também contribui com uma lente analítica nova para o estudo de *parsers* neurais: o desempenho final esconde trajetórias de aprendizado muito distintas entre arquiteturas, e a estabilidade de convergência, medida pelos eventos de *forgetting* e *recovery* por rótulo, é um indicador complementar valioso para avaliar e comparar modelos além das métricas de acurácia final.

Por fim, o projeto registrado no WandB e os 16 modelos treinados constituem um recurso empírico para a comunidade de PLN em português, permitindo reprodução dos experimentos, análise de custo computacional e ponto de partida para extensões futuras do ParseH2IA.

REFERÊNCIAS

- AKIBA, T.; SANO, S.; YANASE, T.; OHTA, T.; KOYAMA, M. Optuna: A Next-Generation Hyperparameter Optimization Framework. In: ACM SIGKDD INTERNATIONAL CONFERENCE ON KNOWLEDGE DISCOVERY AND DATA MINING, 25., 2019. **Proceedings...** ACM, 2019. p.2623–2631.
- ALKOHLANI, F. A.; ALGUZO, N. I. S.; SAYED, H.; SHENOUDA, E. The Arabic sentence: towards a clear view. **Journal Archive**, [S.l.], v.53, 2025.
- AYCOCK, J.; HORSPOOL, R. N. Practical earley parsing. **The Computer Journal**, [S.l.], v.45, n.6, p.620–630, 2002.
- BERGSTRA, J.; BARDENET, R.; BENGIO, Y.; KÉGL, B. Algorithms for Hyper-Parameter Optimization. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 2011. **Anais...** Curran Associates: Inc., 2011. v.24, p.2546–2554.
- BRANCO, A.; SILVA, J. R.; GOMES, L.; ANTÓNIO RODRIGUES, J. Universal Grammatical Dependencies for Portuguese with CINTIL Data, LX Processing and CLARIN support. In: THIRTEENTH LANGUAGE RESOURCES AND EVALUATION CONFERENCE, 2022, Marseille, France. **Proceedings...** European Language Resources Association, 2022. p.5617–5626.
- BRAVO, C.; NETO, J. J. Building context-sensitive parsers from CF grammars with regular control language. In: INTERNATIONAL CONFERENCE ON IMPLEMENTATION AND APPLICATION OF AUTOMATA, 2003. **Anais...** [S.l.: s.n.], 2003. p.306–308.
- BRISCOE, T. **An introduction to tag sequence grammars and the RASP system parser**. [S.l.]: University of Cambridge, Computer Laboratory, 2006.
- CARUANA, R. Multitask learning. **Machine learning**, [S.l.], v.28, n.1, p.41–75, 1997.
- CASELI, H.; NUNES, M. Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português. BPLN, 2 edn.(2024). , [S.l.], 2023.

CHAPPELIER, J.-C.; RAJMAN, M. et al. A Generalized CYK Algorithm for Parsing Stochastic CFG. **TAPD**, [S.l.], v.98, n.133-137, p.5, 1998.

CHEN, W.; KAWAHARA, D.; UCHIMOTO, K.; ZHANG, Y.; ISAHARA, H. Dependency parsing with short dependency relations in unlabeled data. In: THIRD INTERNATIONAL JOINT CONFERENCE ON NATURAL LANGUAGE PROCESSING: VOLUME-I, 2008. **Proceedings...** [S.l.: s.n.], 2008.

CHOMSKY, N. **Syntactic structures**. [S.l.]: Mouton de Gruyter, 2002.

COPESTAKE, A.; CARROLL, J.; MALOUF, R.; OEPEN, S. **The (new) LKB system**. 1999. [S.l.]: CSLI, 1999.

DAHLHAUS, E.; WARMUTH, M. K. Membership for growing context-sensitive grammars is polynomial. **Journal of Computer and System Sciences**, [S.l.], v.33, n.3, p.456–472, 1986.

DE MARNEFFE, M.-C.; DOZAT, T.; SILVEIRA, N.; HAVERINEN, K.; GINTER, F.; NIVRE, J.; MANNING, C. D. Universal Stanford dependencies: A cross-linguistic typology. In: LREC, 2014. **Anais...** [S.l.: s.n.], 2014. v.14, p.4585–4592.

DEPENDENCIES, U. **Universal Dependencies**. 2025. Acesso em: 03 de outubro de 2025, <https://universaldependencies.org/>.

DEVLIN, J.; CHANG, M.-W.; LEE, K.; TOUTANOVA, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In: CONFERENCE OF THE NORTH AMERICAN CHAPTER OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS: HUMAN LANGUAGE TECHNOLOGIES, 2019., 2019, Minneapolis, Minnesota. **Proceedings...** Association for Computational Linguistics, 2019. v.1, p.4171–4186.

DOBREV, D. A definition of artificial intelligence. **arXiv preprint arXiv:1210.1568**, [S.l.], 2012.

DOZAT, T.; MANNING, C. D. Deep Biaffine Attention for Neural Dependency Parsing. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS (ICLR 2017), 5., 2017, Toulon, France. **Proceedings...** [S.l.: s.n.], 2017. arXiv preprint arXiv:1611.01734.

DURAN, M.; LOPES, L.; GRAÇAS NUNES, M. das; PARDO, T. The Dawn of the Porttiani Multigenre Treebank: Introducing its Journalistic Portion. In: XIV SIMPÓSIO BRASILEIRO DE TECNOLOGIA DA INFORMAÇÃO E DA LINGUAGEM HUMANA, 2023, Porto Alegre, RS, Brasil. **Anais...** SBC, 2023. p.115–124.

EHSAN, T.; BUTT, M.; HUSSAIN, S.; ALHUZALI, H.; AL-LAITH, A. Multi-task learning by using contextualized word representations for syntactic parsing of a morphologically rich language. **Plos one**, [S.l.], v.20, n.9, p.e0332580, 2025.

EISNER, J. Three New Probabilistic Models for Dependency Parsing: An Exploration. In: INTERNATIONAL CONFERENCE ON COMPUTATIONAL LINGUISTICS (COLING 1996), 16., 1996, Copenhagen, Denmark. **Proceedings...** Association for Computational Linguistics, 1996. p.340–345.

ENFIELD, N. J. Scale in language. **Cognitive Science**, [S.l.], v.47, n.10, p.e13341, 2023.

FEI, H.; WU, S.; REN, Y.; LI, F.; JI, D. Better Combine Them Together! Integrating Syntactic Constituency and Dependency Representations for Semantic Role Labeling. , [S.l.], p.549–559, 2021.

FERNÁNDEZ-GONZÁLEZ, D.; GÓMEZ-RODRÍGUEZ, C. Multitask pointer network for multi-representational parsing. **Knowledge-Based Systems**, [S.l.], v.236, p.107760, 2022.

FRIEDBERG, R. M.; DUNHAM, B.; NORTH, J. H. A learning machine: Part II. **IBM Journal of Research and Development**, [S.l.], v.3, n.3, p.282–287, 1959.

GU, Y.; HOU, Y.; WANG, Z.; DUAN, X.; LI, Z. High-order Joint Constituency and Dependency Parsing. , [S.l.], p.8144–8154, 2023.

GUPTA, K.; THÉRIEN, B.; IBRAHIM, A.; RICHTER, M. L.; ANTHONY, Q.; BELILOVSKY, E.; RISH, I.; LESORT, T. **Continual Pre-Training of Large Language Models**: How to (re)warm your model? 2023. Disponível em: <<https://arxiv.org/abs/2308.04014>>.

HALFON, A.; GRETZ, S.; ARVIV, O.; SPECTOR, A.; TOLEDO-RONEN, O.; KATZ, Y.; EIN-DOR, L.; SHMUELI-SCHEUER, M.; SLONIM, N. **Stay Tuned**: An Empirical Study of the Impact of Hyperparameters on LLM Tuning in Real-World Applications. 2024. Disponível em: <<https://arxiv.org/abs/2407.18990>>.

HEWITT, J.; MANNING, C. D. A Structural Probe for Finding Syntax in Word Representations. In: CONFERENCE OF THE NORTH AMERICAN CHAPTER OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS: HUMAN LANGUAGE TECHNOLOGIES, 2019., 2019, Minneapolis, Minnesota. **Proceedings...** Association for Computational Linguistics, 2019. v.1, p.4129–4138.

IBRAHIM, A.; THÉRIEN, B.; GUPTA, K.; RICHTER, M. L.; ANTHONY, Q.; LESORT, T.; BELILOVSKY, E.; RISH, I. **Simple and Scalable Strategies to Continually Pre-train Large Language Models**. 2024. Disponível em: <<https://arxiv.org/abs/2403.08763>>.

JAWAHAR, G.; SAGOT, B.; SEDDAH, D. What Does BERT Learn about the Structure of Language? In: ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS, 57., 2019, Florence, Italy. **Proceedings...** Association for Computational Linguistics, 2019. p.3651–3657.

JOHNSON, M. PCFG models of linguistic tree representations. **Computational Linguistics**, [S.l.], v.24, n.4, p.613–632, 1998.

JOSHI, A.; DABRE, R.; KANOJIA, D.; LI, Z.; ZHAN, H.; HAFFARI, G.; DIPPOLD, D. Natural language processing for dialects of a language: A survey. **ACM Computing Surveys**, [S.l.], v.57, n.6, p.1–37, 2025.

JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3.ed. [S.l.: s.n.], 2023. Draft of January 7, 2025. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>.

KOBAYASHI, H.; IKEDA, M.; TANAKA, E. A top-down error-correcting parser for a context-sensitive language. **Systems and Computers in Japan**, [S.l.], v.17, n.5, p.84–91, 1986.

KONDRATYUK, D.; STRAKA, M. 75 languages, 1 model: Parsing universal dependencies universally. In: EMNLP-IJCNLP), 2019., 2019. **Proceedings...** [S.l.: s.n.], 2019. p.2779–2795.

LEDERBERG, J. **DENDRAL-64-A system for computer construction, enumeration and notation of organic molecules as tree structures and cyclic graphs. Part II-Topology of cyclic graphs Interim report**. [S.l.: s.n.], 1965.

LIGHTHILL, J. Artificial intelligence: A general survey, 1974. URL http://www.chilton-computing.org.uk/inf/literature/reports/lighthill_report/p001.htm, [S.l.], 2020.

LIMA, G. D.; LOPES, E.; PEREIRA, H.; SILVEIRA, M.; FREITAS, L.; CORRÊA, U. Generative Models for Multiple-Choice Question Answering in Portuguese: A Monolingual and Multilingual Experimental Study. In: THE INTERNATIONAL FLAIRS CONFERENCE PROCEEDINGS, 2025. **Anais...** [S.l.: s.n.], 2025. v.38.

LIMA, G.; OLIVEIRA, A.; SILVA, F.; PINHEIRO, L.; LUZ, E.; FREITAS, L. Desafios sobre a Detecção de Discurso de Ódio em Português Brasileiro: Construção de um Novo Corpus e Reflexões sobre o Processo. , [S.l.], 2024.

LOPES, L.; BRISOLARA, U.; FREITAS, L. A. d. Towards a New Portuguese Dependency Parser. In: INTERNATIONAL CONFERENCE ON COMPUTATIONAL PROCESSING OF PORTUGUESE (PROPOR 2024), 16., 2024, Santiago de Compostela, Spain. **Proceedings...** Association for Computational Linguistics, 2024.

MARTINET, A. et al. *Lingüística sincrónica.* , [S.l.], 1978.

MARVIN, M.; SEYMOUR, A. P. Perceptrons. **Cambridge, MA: MIT Press**, [S.l.], v.6, n.318-362, p.7, 1969.

MCCARTHY, J. History of LISP. In: **History of programming languages.** [S.l.: s.n.], 1978. p.173–185.

MCCARTHY, J.; MINSKY, M. L.; ROCHESTER, N.; SHANNON, C. E. A proposal for the dartmouth summer research project on artificial intelligence, august 31, 1955. **AI magazine**, [S.l.], v.27, n.4, p.12–12, 2006.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. **The bulletin of mathematical biophysics**, [S.l.], v.5, n.4, p.115–133, 1943.

MEL'CUK, I. A. et al. **Dependency syntax: theory and practice.** [S.l.]: SUNY press, 1988.

MILLER, P. H. **Clitics and constituents in phrase structure grammar.** [S.l.]: Philip Miller, 1992.

MORALES, E. F.; ESCALANTE, H. J. A brief introduction to supervised, unsupervised, and reinforcement learning. In: **Biosignal processing and classification using computational learning and intelligence.** [S.l.]: Elsevier, 2022. p.111–129.

N, T. R.; GUPTA, R. A Survey on Machine Learning Approaches and Its Techniques:. In: IEEE INTERNATIONAL STUDENTS' CONFERENCE ON ELECTRICAL,ELECTRONICS AND COMPUTER SCIENCE (SCEECS), 2020., 2020. **Anais...** [S.l.: s.n.], 2020. p.1–6.

NATURAL LANGUAGE PROCESSING, U. o. S. Institute for. **Project Textual Corpora and Tools for Their Exploration.** 2025. Acesso em: 03 de outubro de 2025, <https://www.ims.uni-stuttgart.de/en/research/projects/textkorpora-werkzeuge/>.

NEWELL, A.; SIMON, H. The logic theory machine—A complex information processing system. **IRE Transactions on information theory**, [S.l.], v.2, n.3, p.61–79, 1956.

NILSSON, N. J.; HILPISCH, Y.; YAO, M.; ZHOU, A.; JIA, M.; BAESSEN, B.; VLASSELAER, V.; VERBEKE, W. The quest for ai: A history of ideas and achievements. **Erişim adresi: <http://ai.stanford.edu/~nilsson/>**(Özgün eser 2009 tarihlidir), [S.l.], 2010.

NIVRE, J.; HALL, J.; NILSSON, J.; ERYİĞİT, G.; MARINOV, S. Labeled pseudo-projective dependency parsing with support vector machines. In: TENTH CONFERENCE ON COMPUTATIONAL NATURAL LANGUAGE LEARNING (CONLL-X), 2006. **Proceedings...** [S.l.: s.n.], 2006. p.221–225.

OTERO, P. G. The meaning of syntactic dependencies. **Linguistik online**, [S.l.], v.35, n.3, 2008.

PACHECO, W. E. A.; GUARANHA, M. F. Parser sintático para o português brasileiro: desafios e soluções. **Texto Livre**, [S.l.], v.15, p.e37569, 2022.

PAREJA, A.; NAYAK, N. S.; WANG, H.; KILLAMSETTY, K.; SUDALAIRAJ, S.; ZHAO, W.; HAN, S.; BHANDWALDAR, A.; XU, G.; XU, K.; HAN, L.; INGLIS, L.; SRIVASTAVA, A. **Unveiling the Secret Recipe: A Guide For Supervised Fine-Tuning Small LLMs**. 2024. Disponível em: <<https://arxiv.org/abs/2412.13337>>.

PAUL, R. R.; ISLAM, M. R.; RAHMAN, M. et al. Bengali UPOS-Tag: A Systematic Approach to Universal Dependency-Based Dataset Creation for Enhanced NLP Research. **Bengali UPOS-Tag: A Systematic Approach to Universal Dependency-Based Dataset Creation for Enhanced NLP Research (December 06, 2023)**, [S.l.], 2023.

PETROV, S.; DAS, D.; MCDONALD, R. A universal part-of-speech tagset. **arXiv preprint arXiv:1104.2086**, [S.l.], 2011.

PIRES, T.; EGGER, J.; LHONEUX, M. d. **ModernJabuticaBERT: A Modern BERT for Brazilian Portuguese**. 2024. Disponível em: <https://huggingface.co/amadeusai/modernJabuticaBERT-Base-1k>. TODO: verificar referência definitiva do artigo/relatório técnico.

POUYANFAR, S.; SADIQ, S.; YAN, Y.; TIAN, H.; TAO, Y.; REYES, M. P.; SHYU, M.-L.; CHEN, S.-C.; IYENGAR, S. S. A survey on deep learning: Algorithms, techniques, and applications. **ACM computing surveys (CSUR)**, [S.l.], v.51, n.5, p.1–36, 2018.

QI, P.; ZHANG, Y.; ZHANG, Y.; BOLTON, J.; MANNING, C. D. Stanza: A Python Natural Language Processing Toolkit for Many Human Languages. In: ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS: SYSTEM DEMONSTRATIONS, 58., 2020, Online. **Proceedings...** Association for Computational Linguistics, 2020. p.101–108.

RAI, D. H. Artificial Intelligence Through Time: A Comprehensive Historical Review. , [S.l.], 2024.

REAL, L.; FONSECA, E.; PON, M.; CRAVALHO, H.; BALAGE FILHO, P. P.; RAMISCH, C. ASSIN 2: A Full-fledged Dataset for Semantic Textual Similarity and Textual Entailment for Portuguese. In: COMPUTATIONAL PROCESSING OF THE Portuguese LANGUAGE (PROPOR 2020), 2020. **Anais...** Springer: Cham, 2020. p.406–412. (Lecture Notes in Computer Science, v.12037).

REDDY, S.; TÄCKSTRÖM, O.; COLLINS, M.; KWIATKOWSKI, T.; DAS, D.; STEEDMAN, M.; LAPATA, M. Transforming dependency structures to logical forms for semantic parsing. **Transactions of the Association for Computational Linguistics**, [S.l.], v.4, p.127–140, 2016.

ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. **Psychological review**, [S.l.], v.65, n.6, p.386, 1958.

RUSSELL, S.; NORVIG, P. Artificial Intelligence: a modern approach, 4th US ed. **aima: . URL: <https://aima.cs.berkeley.edu/> (: 26.02. 2023)**, [S.l.], 2021.

SAMUEL, A. L. Some studies in machine learning using the game of checkers. **IBM Journal of research and development**, [S.l.], v.3, n.3, p.210–229, 1959.

SAPIR, E. The status of linguistics as a science. **language**, [S.l.], p.207–214, 1929.

SHORTLIFFE, E. H.; DAVIS, R.; AXLINE, S. G.; BUCHANAN, B. G.; GREEN, C. C.; COHEN, S. N. Computer-based consultations in clinical therapeutics: explanation and rule acquisition capabilities of the MYCIN system. **Computers and biomedical research**, [S.l.], v.8, n.4, p.303–320, 1975.

SILVA, E.; PARDO, T.; ROMAN, N. Etiquetagem morfossintática multigênero para o português do Brasil segundo o modelo Universal Dependencies: In: XIV SIMPÓSIO BRASILEIRO DE TECNOLOGIA DA INFORMAÇÃO E DA LINGUAGEM HUMANA, 2023, Porto Alegre, RS, Brasil. **Anais...** SBC, 2023. p.63–73.

SILVEIRA, M. R.; LIMA, G. D.; JÚNIOR, C. A. C. S.; FREITAS, L. A.; CORRÊA, U. B. Small vs. Large Language Models: A Comparative Study on Multiple-Choice Question Answering in Portuguese. In: BRAZILIAN SYMPOSIUM ON MULTIMEDIA AND THE WEB (WEBMEDIA), 2025. **Anais...** [S.l.: s.n.], 2025. p.420–427.

SOUZA, F.; NOGUEIRA, R.; LOTUFO, R. BERTimbau: Pre-trained BERT Models for Brazilian Portuguese. In: INTELLIGENT SYSTEMS (BRACIS 2020), 2020. **Anais...** Springer: Cham, 2020. p.403–417. (Lecture Notes in Computer Science, v.12319).

Statista. **Digital population worldwide as of April 2024**. 2024. Acesso em: 29 set. 2025. Disponível em: <<https://www.statista.com/statistics/617136/digital-population-worldwide/>>.

STRAKA, M.; STRAKOVÁ, J. Tokenizing, pos tagging, lemmatizing and parsing ud 2.0 with udpipe. In: CONLL 2017 SHARED TASK: MULTILINGUAL PARSING FROM RAW TEXT TO UNIVERSAL DEPENDENCIES, 2017. **Proceedings...** [S.l.: s.n.], 2017. p.88–99.

TechJury. **How Much Data Is Created Every Day?** 2023. Acesso em: 29 set. 2025. Disponível em: <<https://techjury.net/industry-analysis/how-much-data-is-created-every-day/>>.

TENNEY, I.; DAS, D.; PAVLICK, E. BERT Rediscovered the Classical NLP Pipeline. In: ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS, 57., 2019, Florence, Italy. **Proceedings...** Association for Computational Linguistics, 2019. p.4593–4601.

TOOSI, A.; BOTTINO, A. G.; SABOURY, B.; SIEGEL, E.; RAHMIM, A. A brief history of AI: how to prevent another winter (a critical review). **PET clinics**, [S.l.], v.16, n.4, p.449–469, 2021.

TURING, A. M. Computing machinery and intelligence. In: **Parsing the Turing test:** Philosophical and methodological issues in the quest for the thinking computer. [S.l.]: Springer, 2007. p.23–65.

VAN ENGELEN, J. E.; HOOS, H. H. A survey on semi-supervised learning. **Machine learning**, [S.l.], v.109, n.2, p.373–440, 2020.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention Is All You Need. **CoRR**, [S.l.], v.abs/1706.03762, 2017.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention Is All You Need. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 2017. **Anais...** Curran Associates: Inc., 2017. v.30, p.5998–6008.

WANG, S.; ZHOU, W.; JIANG, C. A survey of word embeddings based on deep learning. **Computing**, [S.l.], v.102, n.3, p.717–740, 2020.

Weights & Biases. **W&B:** A Tool for Experiment Tracking, Dataset Versioning, and Model Management. 2020. Disponível em: <https://wandb.ai>.

WINOGRAD, T. **Procedures as a representation for data in a computer program for understanding natural language.** [S.l.: s.n.], 1971.

WOLF, T.; DEBUT, L.; SANH, V.; CHAUMOND, J.; DELANGUE, C.; MOI, A.; CISTAC, P.; RAULT, T.; LOUF, R.; FUNTOWICZ, M. et al. Transformers: State-of-the-Art Natural Language Processing. In: CONFERENCE ON EMPIRICAL METHODS IN NATURAL LANGUAGE PROCESSING: SYSTEM DEMONSTRATIONS, 2020., 2020, On-line. **Proceedings...** Association for Computational Linguistics, 2020. p.38–45.

WU, Y.; SCHUSTER, M.; CHEN, Z.; LE, Q. V.; NOROUZI, M.; MACHEREY, W.; KRISKUN, M.; CAO, Y.; GAO, Q.; MACHEREY, K. et al. Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation. **arXiv preprint arXiv:1609.08144**, [S.I.], 2016.

YUAN, L.; WANG, J.; YU, L.-C.; ZHANG, X. Encoding syntactic information into transformers for aspect-based sentiment triplet extraction. **IEEE Transactions on Affective Computing**, [S.I.], v.15, n.2, p.722–735, 2023.

ZHOU, Z.-H. **Machine learning**. [S.I.]: Springer nature, 2021.

Apêndices

.8 Classes de Dependencia

- **acl**: Uma oração finita ou não finita que modifica um substantivo. **Exemplo**: as questões como ele as vê. `acl(questões, vê)`. Variação de **acl** é **acl:relcl**, modificador de oração relativa.
- **advcl**: Uma cláusula que modifica um verbo ou outro predicado como modificador, não como complemento central. **Exemplo**: O acidente aconteceu enquanto a noite caía. `advcl(aconteceu, caía)`. Variação de **advcl** é **advcl:relcl**, modificador de oração adverbial relativa.
- **advmod**: Um advérbio ou frase adverbial que modifica um predicado ou modificador. **Exemplo**: Alimento geneticamente modificado. `advmod(modificado, geneticamente)`. Variações de **advmod** são, **advmod:emph**, palavra enfatizadora, intensificador e **advmod:lmod** modificador de adverbial locativo.
- **amod**: Uma frase adjetiva que modifica um substantivo. **Exemplo**: Pedro come cachorros-quentes grandes. `amod(cachorros-quentes, grandes)`.
- **appos**: Um nominal que define ou descreve outro nominal imediatamente após. **Exemplo**: Tiago, meu irmão, chegou. `appos(Tiago, irmão)`.
- **aux**: Uma palavra funcional associada a um verbo léxico que expressa categorias gramaticais como tempo ou modo. **Exemplo**: Pedro já chegou. `aux(chegou, já)`. Variação de **aux** é **aux:pass**, auxiliar passivo, Tiago foi embora. `aux:pass(embora, foi)`.
- **case**: Uma palavra funcional que indica o caso gramatical de um nominal. **Exemplo**: O escritório do presidente. `case(presidente, do)`.
- **cc**: Uma palavra que liga palavras ou constituintes maiores. **Exemplo**: Pedro é grande e honesto. `cc(é, e)`. Variação de **cc** é **cc:preconj**, um elemento antes de uma conjunção coordenativa. **Exemplo**: Tanto os meninos quanto as meninas estão aqui. `cc:preconj(meninos, tanto)`
- **ccomp**: Uma oração dependente que completa o significado de um verbo. **Exemplo**: Ele diz que você gosta de nadar. `ccomp(diz, gosta)`
- **clf**: Uma palavra que categoriza um substantivo, usada com numerais. **Exemplo**: Três unidades de lápis. `clf(lápis, unidades)`.
- **compound**: Um termo que forma um composto com sua cabeça. **Exemplo**: Livro telefônico. `compound(livro, telefônico)`. Variações de **compound** são, **compound:lvc**, um verbo auxiliar em uma construção de verbo leve. **Exemplo**: Eles

preferem o trem. compound: lvc(preferem, o); **compound:prt**, partícula de verbo frasal. **Exemplo**: Eles desligaram a estação. compound:prt(desligaram, desligar); **compound:redup**, elementos em compostos por reduplicação. **Exemplo**: Homens grandes (+ênfase) estão jogando jogos. compound:redup(grandes, ênfase); **compound:svc**, compostos de verbos seriais. **Exemplo**: Musa veio pegar a faca. compound:svc(veio, pegar).

- **conj**: Um coordenado ligado por conjunção. **Exemplo**: Pedro é grande e honesto. conj(é, honesto).
- **cop**: Uma palavra que liga um sujeito a um predicado, tipicamente “ser”. **Exemplo**: Tiago é honesto. cop(é, é).
- **csbj**: Um oracional que funciona como sujeito sintático. **Exemplo**: Que ele mentiu me surpreendeu. csbj(surpreendeu, mentiu). As variações de **csbj** são, **csbj:outer**, sujeito de uma oração copulativa cujo predicado é uma oração. **Exemplo**: Caminhar nas montanhas é experimentar o melhor da natureza. csbj:outer(é, caminhar) e **csbj:pass** sujeito passivo de uma oração. **Exemplo**: Que ela mentiu foi suspeitado por todos. csbj:pass(for, mentiu).
- **dep**: Dependência não especificada. Usado quando nenhuma outra etiqueta se aplica. **Exemplo**: Meu amigo não faz isso tão bem. dep(faz, não).
- **det**: Determinante. Uma palavra que introduz uma frase nominal e expressa noção gramatical como definitude. **Exemplo**: O homem está aqui. det(está, o). As variações de **det** são, **det:numgov** quantificador pronominal que governa o caso do substantivo. **Exemplo**: Quantos homens jogaram cartas. det:numgov(homens, quantos); **det:nummod** quantificador pronominal que concorda em caso com o substantivo. **Exemplo**: Com quantos homens eu joguei cartas. det:nummod(homens, quantos); **det:poss** determinante possessivo. **Exemplo**: Vai ser meu cuidado verificar. det:poss(é, meu).
- **discourse**: Um marcador discursivo ou conector. **Exemplo**: Eu tenho 21 anos, bem, vou fazer em novembro. discourse(tenho, bem).
- **dislocated**: Elementos deslocados. Elementos no início ou fim de uma cláusula relacionados a uma posição interna. **Exemplo**: Elefante, tópico, nariz, sujeito, longo. dislocated(é, elefante).
- **expl**: Expletivo. Um pronome sem conteúdo semântico em posição de argumento central. **Exemplo**: Há um fantasma no quarto. expl(há, há). As variações de **expl** são **expl:impers** Expletivo em construções impessoais. **Exemplo**: Espera-se que viaje. expl:impers(espera-se, se); **expl:pass** pronome refle-

xivo usado no passivo reflexivo. **Exemplo:** É dito facilmente. expl:pass(dito, é);
expl:pv Clítico reflexivo com verbo inerentemente reflexivo. **Exemplo:** Martin teme animais. expl:pv(teme, se).

- **fixed:** Uma expressão fixa composta de múltiplas palavras. **Exemplo:** Eu gosto de cachorros assim como de gatos. fixed(gosto, assim).
- **flat:** Expressão plana. Uma expressão multi-palavra onde a cabeça não é clara. **Exemplo:** Hillary Rodham Clinton. flat(Hillary, Rodham). Algumas variações de **flat** são **flat:foreign** Termos estrangeiros em uma expressão plana. **Exemplo:** Azul no Rosto. flat:foreign(Azul, no) e **flat:name**, componentes de nomes próprios. **Exemplo:** Eis o discurso de Tiziana Maiolo. flat:name(Tiziana, Maiolo).
- **goeswith:** Um espaço ou hífen inserido incorretamente. **Exemplo:** Eles vêm aqui sem permissão legal. goeswith(vêm, aqui).
- **iobj:** Objeto indireto. O objeto indireto de um verbo. **Exemplo:** Ele me deu um aumento. iobj(deu, me).
- **list:** Lista. Usado para cadeias de itens comparáveis **Exemplo:** Steve Jones sj@abc.xyz Universidade do Arizona. list(Steve, sj@abc.xyz).
- **mark:** Uma palavra que marca uma cláusula como subordinada. **Exemplo:** Forças se envolveram em luta após insurgentes atacarem. mark(atacarem, após).
- **nmod:** Modificador nominal. Dependentes nominais de outro nominal, como atributo ou complemento genitivo. **Exemplo:** O escritório da presidente. nmod(escritório, presidente). As variações de **nmod** são, **nmod:poss** modificador possessivo marcado com genitivo. **Exemplo:** O livro de Marie. nmod:poss(livro, Marie) e **nmod:tmod:** Modificador que especifica tempo. **Exemplo:** Você está livre para o almoço algum dia esta semana? nmod:tmod(dia, semana).
- **nsubj:** Sujeito nominal. O sujeito sintático e proto-agente de uma cláusula. **Exemplo:** Clinton derrotou Dole. nsubj(derrotou, Clinton). As variações de **nsubj** são **nsubj:outer** sujeito de cláusula copulativa cujo predicado é cláusula. **Exemplo:** O problema é que isso nunca foi tentado. nsubj:outer(é, problema). **nsubj:pass**, frase nominal sujeito de cláusula passiva. **Exemplo:** Schwarzenberg foi derrotado por Zeman. nsubj:pass(foi, Schwarzenberg)
- **nummod:** Um número que modifica um nominal. **Exemplo:** Sam comeu 3 ovelhas. nummod(comeu, 3). A variação do **nummod** é **nummod:gov**, modificador numérico que governa o caso do substantivo. **Exemplo:** Cinco homens jogaram cartas. nummod:gov(homens, cinco).

- **obj**: O argumento central secundário de um verbo. **Exemplo**:: Ela me deu um aumento. obj(deu, aumento)
- **obl**: Nominal oblíquo. Nominal não central atuando como argumento ou adjunto. **Exemplo**: Ontem à noite, nadei na piscina. obl(nadei, noite). As variações do **obl** são, **obl:agent** Agentes de verbos passivos. **Exemplo**: O gato foi perseguido pelo cachorro. obl:agent(perseguido, cachorro). **obl:arg** Argumentos oblíquos distintos de adjuntos. **Exemplo**: Confio no instinto dele. obl:arg(confio, instinto). **obl:lmod** modificador especificando localização. **Exemplo**: Dirija por aquela estrada, você recebe instruções nela. obl:lmod(estrada, por). **obl:tmod**, modificador especificando tempo. **Exemplo**: Ontem à noite, nadei na piscina. obl:tmod(nadei, noite).
- **orphan**: Usado em elipse de cabeça onde promoção resultaria em relação não natural. **Exemplo**: Marie ganhou ouro e Peter bronze. orphan(ganhou, Peter).
- **parataxis**: Relação entre elementos justapostos sem coordenação explícita. **Exemplo**: Vamos encarar, estamos irritados. parataxis(estamos, encarar).
- **punct** Qualquer peça de pontuação em uma cláusula. **Exemplo**: Vá para casa! punct(vá, !).
- **reparandum** Indica disfluências em reparos de fala. **Exemplo**: Vá para a direi-para a esquerda. reparandum(esquerda, direi).
- **root**: Aponta para a raiz da sentença. **Exemplo**: Eu amo batatas fritas francesas. root(amo, ROOT).
- **vocative**: Marca um participante do diálogo endereçado. **Exemplo**: Rapazes, relaxem!. vocative(relaxem, rapazes).
- **xcomp**: Complemento clausal aberto. Cláusula sem sujeito próprio, compartilhando com a cláusula matriz. **Exemplo**: Esperamos que eles mudem de ideia. xcomp(esperamos, mudem).

.9 Resultado: Busca de Hiperparametros Avaliados

.10 Resultado: Treinamento Modelos Avaliados

.11 Resultado: Validação Modelos Avaliados

.12 Frequência das Tags e instabilidade

Tabela 16 – Resultados dos experimentos com mBERT (Linear)

Trial	Mean LAS	Std LAS	Learning Rate	Weight Decay	Warmup Ratio
1	0.885058	0.006380	0.000036	0.196730	0.420505
3	0.884417	0.006255	0.000034	0.129981	0.399405
2	0.880640	0.005876	0.000029	0.216720	0.498847
7	0.869962	0.004572	0.000021	0.192942	0.381000
9	0.869878	0.004654	0.000021	0.101991	0.438022
5	0.859672	0.004352	0.000017	0.130814	0.347393
6	0.857565	0.004750	0.000016	0.240849	0.462004
4	0.855767	0.004519	0.000016	0.182946	0.305832
0	0.854372	0.004344	0.000015	0.108845	0.389569
8	0.839878	0.005863	0.000012	0.292299	0.366276

Tabela 17 – Resultados dos experimentos com BERTimbau-Base (Linear)

Trial	Mean LAS	Std LAS	Learning Rate	Weight Decay	Warmup Ratio
4	0.908692	0.005915	0.000044	0.214306	0.367460
0	0.906738	0.007174	0.000041	0.285090	0.419238
8	0.905483	0.006857	0.000034	0.272302	0.311038
6	0.905474	0.007546	0.000039	0.215511	0.471784
7	0.904887	0.005722	0.000033	0.220988	0.342784
3	0.902560	0.006051	0.000031	0.108977	0.451542
9	0.896193	0.006148	0.000025	0.182029	0.398756
1	0.870862	0.007976	0.000015	0.242467	0.300276
5	0.869476	0.008027	0.000015	0.177760	0.469774
2	0.863406	0.007812	0.000014	0.126214	0.452487

Tabela 18 – Resultados dos experimentos com BERTimbau-Large (Linear)

Trial	Mean LAS	Std LAS	Learning Rate	Weight Decay	Warmup Ratio
2	0.925713	0.002624	0.000038	0.126153	0.403698
1	0.925066	0.002479	0.000029	0.214964	0.349256
6	0.924980	0.002818	0.000025	0.169615	0.432424
7	0.922653	0.007954	0.000046	0.124523	0.417075
4	0.922128	0.003095	0.000020	0.198165	0.423723
0	0.920267	0.003849	0.000019	0.168731	0.461301
8	0.918754	0.003464	0.000017	0.232271	0.408937
5	0.916199	0.003033	0.000015	0.110463	0.328853
9	0.911381	0.004579	0.000013	0.283695	0.318210
3	0.901764	0.004516	0.000010	0.173628	0.433841

Tabela 19 – Resultados dos experimentos com modernJabuticaBERT-Base-1k (Linear)

Trial	Mean LAS	Std LAS	Learning Rate	Weight Decay	Warmup Ratio
3	0.825660	0.012153	0.000049	0.149677	0.435433
1	0.818081	0.012015	0.000046	0.274902	0.346220
4	0.816004	0.006392	0.000041	0.185580	0.483414
7	0.764761	0.021123	0.000029	0.240583	0.388557
2	0.760194	0.025240	0.000029	0.124826	0.354958
0	0.726410	0.011366	0.000022	0.227242	0.359989
6	0.690056	0.025080	0.000015	0.262750	0.484973
8	0.675433	0.007994	0.000013	0.198784	0.451598
9	0.675348	0.017714	0.000013	0.163394	0.356288
5	0.634587	0.010357	0.000010	0.205283	0.352006

Tabela 20 – Resultados dos experimentos com mBERT (Biaffine)

Trial	Mean LAS	Std LAS	Learning Rate	Weight Decay	Warmup Ratio
8	0.833259	0.005840	0.000030	0.237547	0.327849
2	0.832299	0.003669	0.000027	0.124994	0.472556
3	0.831644	0.003540	0.000021	0.295586	0.383554
7	0.831031	0.004162	0.000021	0.203839	0.469646
9	0.826412	0.003292	0.000047	0.124138	0.306292
0	0.826105	0.003719	0.000016	0.255733	0.313032
1	0.826067	0.004637	0.000048	0.171161	0.344204
5	0.824835	0.003819	0.000017	0.250622	0.466429
4	0.812010	0.004949	0.000013	0.288138	0.487158
6	0.807205	0.004583	0.000012	0.102060	0.464783

Tabela 21 – Resultados dos experimentos com BERTimbau Base (Biaffine)

Trial	Mean LAS	Std LAS	Learning Rate	Weight Decay	Warmup Ratio
6	0.863257	0.003916	0.000031	0.128855	0.410144
2	0.862503	0.005636	0.000031	0.296217	0.356121
5	0.861592	0.004908	0.000029	0.263890	0.405889
9	0.861510	0.006410	0.000037	0.282828	0.460082
7	0.860582	0.006276	0.000023	0.164334	0.308330
3	0.860180	0.009311	0.000046	0.259406	0.484412
8	0.860107	0.007456	0.000048	0.233779	0.437821
4	0.860095	0.005602	0.000024	0.101319	0.476664
1	0.857603	0.005675	0.000019	0.247379	0.475622
0	0.839351	0.006066	0.000011	0.222736	0.476501

Tabela 22 – Resultados dos experimentos com BERTimbau Large (Biaffine)

Trial	Mean LAS	Std LAS	Learning Rate	Weight Decay	Warmup Ratio
0	0.887366	0.006817	0.000030	0.116483	0.417290
4	0.885584	0.009068	0.000018	0.147112	0.307553
7	0.885084	0.005630	0.000017	0.116782	0.488933
8	0.878930	0.013033	0.000016	0.294734	0.385474
6	0.878894	0.016560	0.000019	0.124715	0.302827
5	0.877394	0.010739	0.000018	0.146182	0.425536
3	0.874395	0.018661	0.000049	0.175856	0.439879
2	0.874045	0.020230	0.000039	0.142492	0.330243
9	0.862019	0.016939	0.000049	0.184893	0.482496
1	0.861821	0.025160	0.000036	0.210631	0.458091

Tabela 23 – Resultados dos experimentos com JabuticaBERT (Biaffine)

Trial	Mean LAS	Std LAS	Learning Rate	Weight Decay	Warmup Ratio
6	0.818719	0.025180	0.000035	0.158349	0.302796
3	0.808947	0.034193	0.000045	0.168110	0.473429
5	0.792397	0.033073	0.000036	0.169727	0.347370
1	0.747085	0.047024	0.000020	0.122321	0.322066
7	0.702396	0.048477	0.000021	0.105447	0.449218
2	0.674189	0.066415	0.000022	0.294030	0.468165
8	0.664172	0.041053	0.000020	0.257532	0.373692
4	0.621514	0.053323	0.000013	0.179855	0.457334
9	0.588667	0.061754	0.000013	0.257362	0.415505
0	0.560711	0.036098	0.000011	0.233061	0.344830

Tabela 24 – Resultados do treinamento por época para o modelo mBERT (Linear)

Epoch	Training Loss	Validation Loss	UAS	LAS	UPOS Accuracy
1	9.784786	6.575181	0.112768	0.090611	0.777003
2	5.680443	4.400321	0.161281	0.144849	0.954856
3	4.105455	3.332325	0.228167	0.214088	0.970919
4	3.175950	2.573829	0.345504	0.330271	0.976228
5	2.490882	2.061388	0.472258	0.456102	0.976459
6	1.987491	1.735117	0.557976	0.541082	0.977705
7	1.616225	1.439901	0.649788	0.630631	0.979228
8	1.349968	1.284263	0.698071	0.679930	0.980336
9	1.139671	1.184959	0.720227	0.700379	0.980798
10	0.982604	1.117782	0.743630	0.722904	0.980428
11	0.867634	1.072143	0.755908	0.735275	0.981444
12	0.765230	0.984507	0.800775	0.779819	0.981167
13	0.690611	0.994038	0.798329	0.776588	0.980936
14	0.629238	0.966288	0.804791	0.784297	0.982321
15	0.588444	0.966376	0.827133	0.806361	0.980798
16	0.541444	1.077217	0.812315	0.789282	0.980198
17	0.506546	1.051257	0.804653	0.784250	0.980382
18	0.453457	0.984063	0.835441	0.813839	0.979459
19	0.397804	1.128150	0.777511	0.758263	0.981121
20	0.347073	1.026234	0.844904	0.824040	0.979459
21	0.303266	0.957418	0.873200	0.851828	0.982044
22	0.268382	0.992165	0.872877	0.851736	0.980105
23	0.246756	1.146256	0.804053	0.783789	0.980567
24	0.202736	1.027355	0.873661	0.851782	0.980244
25	0.173859	0.998692	0.886309	0.864937	0.980936
26	0.150304	1.039968	0.884832	0.863322	0.981444
27	0.125447	0.954554	0.896418	0.875877	0.981582
28	0.106853	1.054003	0.888109	0.868076	0.981213
29	0.093519	0.969124	0.902557	0.883355	0.982690
30	0.080641	1.009441	0.903527	0.884278	0.983106
31	0.068464	1.025494	0.904865	0.884694	0.983152
32	0.057137	1.040346	0.902419	0.883586	0.982598
33	0.043171	1.020306	0.910543	0.890371	0.982229
34	0.033366	1.067205	0.907542	0.888294	0.982875
35	0.026204	1.039534	0.914143	0.895079	0.983475
36	0.022319	1.046280	0.916174	0.897341	0.984213
37	0.016075	1.069643	0.915482	0.896510	0.983798
38	0.014255	1.071471	0.913959	0.896141	0.984213
39	0.011247	1.063762	0.916359	0.897618	0.983752
40	0.008834	1.065175	0.916636	0.898172	0.983475

Tabela 25 – Resultados do treinamento por época - Bertimbau-Base (Linear)

Epoch	Training Loss	Validation Loss	UAS	LAS	UPOS Accuracy
1	10.372640	7.025009	0.106139	0.083424	0.761578
2	5.646849	4.342153	0.146057	0.131556	0.969333
3	3.935340	3.168088	0.245283	0.233848	0.979417
4	2.926033	2.341689	0.404647	0.390353	0.982587
5	2.203649	1.774119	0.559021	0.543739	0.985602
6	1.695705	1.416780	0.647123	0.631374	0.987058
7	1.327780	1.137280	0.722127	0.705598	0.988409
8	1.060664	0.982709	0.745985	0.730132	0.988981
9	0.866159	0.871984	0.796975	0.781901	0.988305
10	0.722717	0.837625	0.803212	0.787307	0.988617
11	0.620572	0.807819	0.817194	0.801445	0.987577
12	0.534410	0.768929	0.839337	0.823224	0.988929
13	0.464331	0.859188	0.800821	0.786527	0.988201
14	0.417534	0.830679	0.819845	0.806435	0.988357
15	0.372993	0.757753	0.860700	0.844327	0.987993
16	0.332850	0.713535	0.879983	0.864130	0.987785
17	0.285960	0.742749	0.878684	0.863143	0.988097
18	0.248571	0.743175	0.891211	0.875513	0.988617
19	0.213999	0.767109	0.886948	0.871355	0.988721
20	0.183067	0.735175	0.903581	0.887104	0.988045
21	0.160177	0.772020	0.894121	0.879620	0.988409
22	0.138446	0.763390	0.907636	0.892510	0.988721
23	0.119382	0.813734	0.902958	0.887936	0.987941
24	0.102905	0.760321	0.912521	0.896876	0.987993
25	0.091771	0.766994	0.912833	0.896876	0.989293
26	0.071640	0.825432	0.902646	0.888872	0.989397
27	0.061192	0.829208	0.906596	0.892458	0.988721
28	0.053004	0.806675	0.915224	0.900982	0.988305
29	0.041582	0.830843	0.915016	0.901294	0.988825
30	0.035287	0.782852	0.926296	0.912054	0.989345
31	0.029436	0.827815	0.918967	0.905400	0.989345
32	0.024266	0.892762	0.916992	0.902906	0.988825
33	0.017819	0.867486	0.920526	0.905608	0.989345
34	0.013759	0.837314	0.926763	0.912521	0.989345
35	0.010295	0.885999	0.922033	0.908051	0.989500
36	0.008239	0.842982	0.927335	0.913301	0.989449
37	0.006734	0.847912	0.927803	0.913977	0.989760
38	0.004899	0.865180	0.927647	0.914029	0.989708
39	0.004361	0.860932	0.928583	0.915328	0.989500
40	0.003333	0.862260	0.928271	0.914912	0.989656

Tabela 26 – Resultados do treinamento por época - BERTimbau-Large (Linear)

Epoch	Training Loss	Validation Loss	UAS	LAS	UPOS Accuracy
1	9.876456	5.880204	0.109257	0.088466	0.852487
2	4.857795	3.572459	0.186808	0.169292	0.972348
3	3.236737	2.503597	0.310307	0.295597	0.980196
4	2.358057	1.880307	0.460107	0.445709	0.983263
5	1.785010	1.428331	0.605853	0.589272	0.985654
6	1.372180	1.150877	0.688082	0.673684	0.988357
7	1.067261	0.923696	0.774157	0.757056	0.988565
8	0.863936	0.845216	0.784656	0.770362	0.989604
9	0.713046	0.817122	0.810229	0.796663	0.988097
10	0.613907	0.822485	0.803940	0.789698	0.988929
11	0.527644	0.695295	0.848589	0.835334	0.989085
12	0.471626	0.712244	0.866677	0.852487	0.986278
13	0.427845	0.759945	0.835179	0.820625	0.988045
14	0.379899	0.737954	0.852175	0.837725	0.988929
15	0.357007	0.732695	0.876865	0.861791	0.989033
16	0.327183	0.762744	0.867249	0.850824	0.987733
17	0.315721	0.720203	0.889443	0.873330	0.986018
18	0.259311	0.679714	0.895213	0.880243	0.989137
19	0.219348	0.663645	0.901086	0.887000	0.988929
20	0.192366	0.618323	0.927907	0.912729	0.988409
21	0.157941	0.643311	0.922293	0.907895	0.989604
22	0.137575	0.682865	0.916888	0.901606	0.988669
23	0.114743	0.664303	0.925308	0.910546	0.989397
24	0.099422	0.647212	0.930454	0.914808	0.988981
25	0.081560	0.669267	0.928427	0.913665	0.989085
26	0.066682	0.710384	0.926348	0.911170	0.989500
27	0.057516	0.701615	0.924372	0.910286	0.988409
28	0.043702	0.705665	0.934716	0.920526	0.990124
29	0.037281	0.754914	0.930558	0.916108	0.989293
30	0.026778	0.735343	0.935340	0.920786	0.988877
31	0.022306	0.760168	0.931233	0.917875	0.990228
32	0.016596	0.780705	0.933468	0.919850	0.989708
33	0.011084	0.763185	0.939654	0.926607	0.990384
34	0.008637	0.788380	0.939498	0.925568	0.989033
35	0.006158	0.761091	0.943084	0.930350	0.989656
36	0.003569	0.774513	0.943604	0.930662	0.990488
37	0.002618	0.777236	0.942928	0.929414	0.989812
38	0.002136	0.768134	0.946463	0.933001	0.990176
39	0.001743	0.783440	0.943136	0.929830	0.990124
40	0.002136	0.777716	0.944280	0.930974	0.990124

Tabela 27 – Resultados do treinamento por época para o modelo JabutিকাBERT (Linear)

Epoch	Training Loss	Validation Loss	UAS	LAS	UPOS Accuracy
1	9.803177	6.818684	0.111143	0.082859	0.687876
2	5.411188	4.301930	0.183661	0.160527	0.930556
3	3.695162	3.186497	0.261536	0.240852	0.961208
4	2.804156	2.537012	0.350916	0.332392	0.969182
5	2.213399	2.118947	0.444823	0.426922	0.974125
6	1.782714	1.845442	0.480915	0.462350	0.974872
7	1.451035	1.699578	0.512647	0.493542	0.975661
8	1.190737	1.508857	0.584832	0.563941	0.977613
9	0.968267	1.443441	0.612369	0.592350	0.979358
10	0.796088	1.276843	0.696972	0.673797	0.979441
11	0.679309	1.343255	0.661793	0.640902	0.979026
12	0.579242	1.124826	0.762180	0.740707	0.981642
13	0.506046	1.131747	0.763010	0.742161	0.980521
14	0.446337	1.237296	0.731860	0.711924	0.980936
15	0.399411	1.152712	0.784026	0.761307	0.980147
16	0.366940	1.198923	0.768077	0.749263	0.981642
17	0.333551	1.097938	0.802384	0.781950	0.981933
18	0.298900	1.121107	0.817253	0.796445	0.980355
19	0.253773	1.084486	0.827553	0.807036	0.981725
20	0.212675	1.124565	0.812809	0.792831	0.982805
21	0.162066	1.123202	0.828176	0.809029	0.982058
22	0.129207	1.059192	0.855547	0.837438	0.982971
23	0.104424	1.082864	0.859825	0.840470	0.983096
24	0.078587	1.129784	0.852266	0.833742	0.982348
25	0.059787	1.204039	0.848362	0.829962	0.983096
26	0.042106	1.168908	0.858122	0.840595	0.984093
27	0.032310	1.141175	0.867052	0.849151	0.984176
28	0.022569	1.221780	0.859492	0.841550	0.984342
29	0.018911	1.220996	0.863770	0.846617	0.984633
30	0.015608	1.273313	0.853553	0.835777	0.985131
31	0.008594	1.264320	0.858080	0.841882	0.985463
32	0.006704	1.229602	0.871288	0.854135	0.985006
33	0.004620	1.281679	0.866304	0.849109	0.985048
34	0.003282	1.267262	0.868256	0.850438	0.984840
35	0.003048	1.264561	0.871869	0.853678	0.985380
36	0.002288	1.239354	0.877103	0.859534	0.985546
37	0.001067	1.238567	0.876770	0.859576	0.986045
38	0.000790	1.241940	0.878847	0.862026	0.985671
39	0.000674	1.248937	0.876355	0.859409	0.985837
40	0.000532	1.245967	0.877892	0.860822	0.985754

Tabela 28 – Resultados do treinamento por época para o modelo mBERT (Biaffine)

Epoch	Training Loss	Validation Loss	UAS	LAS	UPOS Accuracy
1	9.943626	6.761981	0.131785	0.099197	0.487768
2	5.900317	3.591131	0.245153	0.215565	0.892125
3	3.770547	2.585422	0.336134	0.311623	0.959010
4	2.853947	2.038078	0.431453	0.410081	0.967504
5	2.283768	1.669670	0.521695	0.502031	0.970919
6	1.873197	1.441020	0.595089	0.575517	0.975905
7	1.584075	1.325079	0.630355	0.611752	0.976597
8	1.357088	1.251108	0.671436	0.651819	0.978167
9	1.179196	1.171223	0.703979	0.683069	0.977705
10	1.028053	1.070744	0.737722	0.718011	0.981305
11	0.904653	1.099649	0.737952	0.717411	0.981536
12	0.814786	1.027820	0.758955	0.738322	0.981859
13	0.730801	1.011102	0.777234	0.757801	0.982321
14	0.667866	1.005841	0.789420	0.769249	0.981075
15	0.603963	1.060188	0.786835	0.766571	0.980198
16	0.525432	1.003042	0.810654	0.789974	0.980198
17	0.459322	1.011761	0.808853	0.787666	0.980521
18	0.418677	1.021525	0.824409	0.803822	0.981075
19	0.372103	1.012364	0.834749	0.813239	0.981167
20	0.336289	1.007745	0.835441	0.816654	0.982506
21	0.293265	1.096428	0.831425	0.812731	0.981813
22	0.262227	1.110891	0.838765	0.819886	0.981444
23	0.233395	1.149395	0.837703	0.819609	0.983013
24	0.204886	1.164659	0.840011	0.821917	0.981398
25	0.184191	1.199049	0.841627	0.823024	0.981952
26	0.162222	1.197551	0.848135	0.828794	0.981813
27	0.146732	1.246256	0.855705	0.837518	0.982367
28	0.119596	1.263536	0.855613	0.836180	0.982367
29	0.111399	1.355260	0.848458	0.829302	0.982875
30	0.095316	1.402977	0.853259	0.833503	0.982044
31	0.086596	1.440888	0.851828	0.833549	0.982875
32	0.074913	1.435824	0.854921	0.836918	0.982644
33	0.064436	1.466542	0.855382	0.838026	0.982598
34	0.056028	1.511367	0.856859	0.839688	0.982967
35	0.051092	1.531169	0.861614	0.844212	0.982506
36	0.039822	1.676730	0.857229	0.839180	0.982921
37	0.038069	1.719510	0.861706	0.843427	0.982598
38	0.031814	1.738831	0.862306	0.844719	0.982506
39	0.028887	1.775959	0.861798	0.843935	0.982921
40	0.025581	1.784062	0.863506	0.845689	0.982736

Tabela 29 – Resultados de treinamento do BERTimbau Base (Biaffine)

Epoch	Training Loss	Validation Loss	UAS	LAS	UPOS Accuracy
1	10.744893	7.806105	0.124643	0.061074	0.242840
2	7.384517	4.804198	0.265139	0.220542	0.700348
3	4.488308	2.925152	0.375071	0.342637	0.929154
4	3.135164	2.153723	0.443786	0.420136	0.960081
5	2.378841	1.599825	0.577629	0.556734	0.969749
6	1.815575	1.256761	0.671397	0.651697	0.977857
7	1.430790	1.073896	0.720360	0.700764	0.978897
8	1.154242	0.953516	0.760175	0.742138	0.981808
9	0.973175	0.874549	0.784136	0.769583	0.984563
10	0.829021	0.815433	0.808462	0.793129	0.986382
11	0.717660	0.765756	0.823120	0.809034	0.986798
12	0.603471	0.740093	0.834971	0.821560	0.987369
13	0.514159	0.758722	0.834035	0.819897	0.987265
14	0.455546	0.794554	0.841520	0.827954	0.988045
15	0.407162	0.729942	0.854254	0.838921	0.988513
16	0.364358	0.775365	0.859504	0.845678	0.988929
17	0.332764	0.778382	0.856333	0.842975	0.988981
18	0.298891	0.777890	0.866001	0.852487	0.989085
19	0.251013	0.900337	0.854046	0.841000	0.989085
20	0.219512	0.823003	0.867977	0.854930	0.989708
21	0.183658	0.893747	0.866053	0.853319	0.988513
22	0.161404	0.874244	0.871719	0.858672	0.990020
23	0.135765	0.966728	0.871303	0.858361	0.989968
24	0.122133	0.982696	0.874318	0.861687	0.989397
25	0.104440	1.061165	0.873018	0.860076	0.989708
26	0.087850	1.060507	0.876709	0.865014	0.989449
27	0.079003	1.147307	0.872135	0.860752	0.989241
28	0.065216	1.166297	0.880659	0.868548	0.989345
29	0.058847	1.169885	0.879048	0.867613	0.989864
30	0.050158	1.244593	0.881647	0.869796	0.990124
31	0.044464	1.272071	0.881855	0.870419	0.990280
32	0.035928	1.311424	0.885337	0.873590	0.990332
33	0.032463	1.345211	0.886533	0.875097	0.990488
34	0.029251	1.463682	0.879464	0.868548	0.989812
35	0.023208	1.501278	0.885389	0.874162	0.990384
36	0.017859	1.541077	0.885285	0.874058	0.990540
37	0.016408	1.597981	0.884505	0.872966	0.990488
38	0.015029	1.656509	0.882010	0.870783	0.990540
39	0.013188	1.663841	0.882634	0.871095	0.990644
40	0.011540	1.660877	0.883570	0.872135	0.990696

Tabela 30 – Resultados de treinamento do BERTimbau Large (Biaffine)

Epoch	Training Loss	Validation Loss	UAS	LAS	UPOS Accuracy
1	10.512833	7.375972	0.131920	0.085763	0.433494
2	6.025387	3.373855	0.306461	0.273871	0.915016
3	3.445530	2.314519	0.402724	0.377774	0.965383
4	2.559693	1.673118	0.535787	0.515359	0.974063
5	1.879633	1.258283	0.651177	0.631374	0.979677
6	1.416966	1.047377	0.706690	0.690421	0.981912
7	1.108526	0.909266	0.750819	0.736109	0.984511
8	0.893560	0.784839	0.799418	0.785020	0.985966
9	0.748319	0.752893	0.828317	0.812932	0.986330
10	0.626098	0.729485	0.846614	0.831176	0.986954
11	0.547924	0.670887	0.862363	0.847549	0.987525
12	0.464688	0.711784	0.844535	0.830501	0.988877
13	0.412185	0.672016	0.874838	0.861843	0.988929
14	0.367261	0.659369	0.880191	0.865897	0.988877
15	0.336941	0.662387	0.878216	0.864962	0.989293
16	0.308120	0.692987	0.880815	0.866989	0.989293
17	0.287169	0.735954	0.878632	0.864806	0.989449
18	0.246611	0.723502	0.883206	0.867405	0.989449
19	0.227644	0.756360	0.884298	0.871511	0.989552
20	0.191040	0.730509	0.897084	0.884090	0.990020
21	0.153508	0.762474	0.894381	0.880815	0.989864
22	0.130891	0.828688	0.898435	0.884817	0.989293
23	0.112817	0.856280	0.893913	0.880815	0.990748
24	0.100263	0.829937	0.904361	0.891107	0.990124
25	0.087344	0.892806	0.895993	0.882686	0.989708
26	0.070365	0.907649	0.904829	0.892770	0.989552
27	0.059181	1.019990	0.899995	0.887988	0.989604
28	0.051826	1.019897	0.904153	0.892562	0.988981
29	0.043866	1.076574	0.903945	0.891366	0.989500
30	0.038549	1.107225	0.900567	0.888716	0.989137
31	0.033144	1.207375	0.897812	0.885597	0.989449
32	0.026743	1.332794	0.894849	0.882946	0.989760
33	0.024400	1.224570	0.904829	0.893394	0.989345
34	0.019296	1.348810	0.901762	0.889911	0.990176
35	0.014300	1.334421	0.906284	0.894589	0.989293
36	0.011883	1.402111	0.907428	0.895473	0.989708
37	0.009700	1.477472	0.904153	0.893238	0.989968
38	0.006236	1.521442	0.904465	0.893446	0.990072
39	0.005849	1.510334	0.905608	0.894433	0.989968
40	0.004790	1.505415	0.905193	0.893965	0.989968

Tabela 31 – Resultados de treinamento do JabuticaBERT (Biaffine)

Epoch	Training Loss	Validation Loss	UAS	LAS	UPOS Accuracy
1	9.354714	6.127495	0.146571	0.104955	0.613947
2	5.221494	3.597127	0.262242	0.228849	0.885742
3	3.473840	2.674963	0.337708	0.311418	0.944428
4	2.671645	2.197135	0.402625	0.378120	0.960917
5	2.151891	1.899414	0.464219	0.441126	0.969141
6	1.731539	1.634654	0.540557	0.517756	0.971716
7	1.359810	1.370629	0.644889	0.621672	0.973211
8	1.051941	1.913915	0.472858	0.455372	0.974000
9	0.837061	1.112549	0.744736	0.720979	0.978070
10	0.690763	1.086161	0.769406	0.744362	0.980022
11	0.555576	1.055949	0.786975	0.763052	0.980147
12	0.492671	1.024116	0.793620	0.771400	0.979898
13	0.414462	0.987794	0.827138	0.802259	0.980853
14	0.338138	1.366410	0.760934	0.739752	0.980313
15	0.271236	1.124688	0.823857	0.803132	0.981185
16	0.233744	1.070727	0.846991	0.825684	0.982639
17	0.185835	1.105435	0.847905	0.825809	0.983262
18	0.150538	1.184686	0.848029	0.827719	0.982930
19	0.121581	1.177323	0.863397	0.842173	0.982390
20	0.103997	1.338099	0.857333	0.837521	0.982722
21	0.085576	1.278244	0.859077	0.840470	0.984674
22	0.078455	1.319509	0.861237	0.843253	0.983927
23	0.061212	1.501688	0.849483	0.830253	0.983802
24	0.053116	1.533521	0.865349	0.848320	0.984882
25	0.045079	1.441699	0.878058	0.859576	0.985173
26	0.035138	1.551463	0.874943	0.856710	0.984467
27	0.025113	1.762924	0.870000	0.852100	0.985090
28	0.028401	1.851935	0.873531	0.855671	0.984010
29	0.025035	1.909725	0.873821	0.855007	0.985173
30	0.016642	1.900559	0.870997	0.853138	0.985006
31	0.015411	1.856799	0.875483	0.856959	0.984799
32	0.015004	1.980777	0.878016	0.860282	0.984591
33	0.008519	2.185527	0.867301	0.849400	0.984965
34	0.006564	2.129559	0.874112	0.856045	0.985297
35	0.004540	2.199263	0.874528	0.856502	0.985006
36	0.004031	2.201184	0.878058	0.860406	0.985339
37	0.002096	2.248716	0.878473	0.860572	0.985339
38	0.002245	2.231646	0.880633	0.862151	0.985505
39	0.001431	2.268286	0.880924	0.862774	0.985546
40	0.001049	2.287051	0.880093	0.862068	0.985380

Tabela 32 – Resultados do treinamento por época para o modelo mBERT (Linear) Sem UPOS

Epoch	Training Loss	Validation Loss	UAS	LAS
1	8.150240	6.619714	0.102105	0.039836
2	5.619763	4.416279	0.176099	0.152234
3	4.065923	3.299242	0.242799	0.224289
4	3.170976	2.611413	0.347858	0.324040
5	2.545582	2.127528	0.443778	0.419544
6	2.090486	1.790254	0.518002	0.495015
7	1.745438	1.508040	0.623061	0.602890
8	1.468736	1.436210	0.572840	0.554607
9	1.254815	1.181029	0.716073	0.694516
10	1.095208	1.113833	0.717965	0.696917
11	0.954613	1.059393	0.753693	0.732598
12	0.848605	1.008936	0.764633	0.744184
13	0.755302	1.042695	0.749908	0.730336
14	0.685851	0.932580	0.810146	0.787482
15	0.637814	0.984839	0.782312	0.760617
16	0.586609	0.928459	0.814346	0.792236
17	0.551760	1.056953	0.770126	0.747784
18	0.511376	1.034414	0.780881	0.759278
19	0.449649	0.802804	0.854782	0.832256
20	0.404061	0.881222	0.846843	0.825748
21	0.342129	0.871673	0.848274	0.826902
22	0.316446	0.849857	0.867338	0.847858
23	0.276518	0.857203	0.874954	0.853859
24	0.242369	0.865407	0.870338	0.848643
25	0.216404	0.854697	0.884555	0.862722
26	0.187581	0.808410	0.892633	0.871076
27	0.170229	0.848775	0.894895	0.873800
28	0.144753	0.888662	0.879108	0.859583
29	0.128580	0.849000	0.895772	0.875415
30	0.113630	0.850119	0.897618	0.876708
31	0.095868	0.855741	0.899326	0.878877
32	0.083291	0.855123	0.905050	0.884694
33	0.071942	0.919655	0.895864	0.876016
34	0.061711	0.881402	0.906665	0.886078
35	0.053539	0.867034	0.909204	0.890048
36	0.046673	0.876005	0.909297	0.890740
37	0.039507	0.875080	0.912482	0.893879
38	0.032562	0.893581	0.911281	0.892818
39	0.029647	0.888009	0.913543	0.894572
40	0.026736	0.890262	0.912435	0.893510

Tabela 33 – Resultados do treinamento por época - Bertimbau-Base (Linear) Sem UPOS

Epoch	Training Loss	Validation Loss	UAS	LAS
1	8.150240	6.619714	0.102105	0.039836
2	5.619763	4.416279	0.176099	0.152234
3	4.065923	3.299242	0.242799	0.224289
4	3.170976	2.611413	0.347858	0.324040
5	2.545582	2.127528	0.443778	0.419544
6	2.090486	1.790254	0.518002	0.495015
7	1.745438	1.508040	0.623061	0.602890
8	1.468736	1.436210	0.572840	0.554607
9	1.254815	1.181029	0.716073	0.694516
10	1.095208	1.113833	0.717965	0.696917
11	0.954613	1.059393	0.753693	0.732598
12	0.848605	1.008936	0.764633	0.744184
13	0.755302	1.042695	0.749908	0.730336
14	0.685851	0.932580	0.810146	0.787482
15	0.637814	0.984839	0.782312	0.760617
16	0.586609	0.928459	0.814346	0.792236
17	0.551760	1.056953	0.770126	0.747784
18	0.511376	1.034414	0.780881	0.759278
19	0.449649	0.802804	0.854782	0.832256
20	0.404061	0.881222	0.846843	0.825748
21	0.342129	0.871673	0.848274	0.826902
22	0.316446	0.849857	0.867338	0.847858
23	0.276518	0.857203	0.874954	0.853859
24	0.242369	0.865407	0.870338	0.848643
25	0.216404	0.854697	0.884555	0.862722
26	0.187581	0.808410	0.892633	0.871076
27	0.170229	0.848775	0.894895	0.873800
28	0.144753	0.888662	0.879108	0.859583
29	0.128580	0.849000	0.895772	0.875415
30	0.113630	0.850119	0.897618	0.876708
31	0.095868	0.855741	0.899326	0.878877
32	0.083291	0.855123	0.905050	0.884694
33	0.071942	0.919655	0.895864	0.876016
34	0.061711	0.881402	0.906665	0.886078
35	0.053539	0.867034	0.909204	0.890048
36	0.046673	0.876005	0.909297	0.890740
37	0.039507	0.875080	0.912482	0.893879
38	0.032562	0.893581	0.911281	0.892818
39	0.029647	0.888009	0.913543	0.894572
40	0.026736	0.890262	0.912435	0.893510

Tabela 34 – Resultados do treinamento por época - BERTimbau-Large (Linear) Sem UPOS

Epoch	Training Loss	Validation Loss	UAS	LAS
1	8.215374	6.113919	0.080098	0.022506
2	4.986060	3.656616	0.191902	0.171942
3	3.265369	2.515021	0.315609	0.299080
4	2.405930	1.903592	0.456573	0.438432
5	1.830124	1.506409	0.550340	0.532460
6	1.446923	1.301511	0.596653	0.579344
7	1.177829	0.982229	0.764801	0.746712
8	0.968323	0.890154	0.781693	0.761214
9	0.795304	0.751432	0.831072	0.811944
10	0.694583	0.793444	0.807630	0.790893
11	0.605494	0.714480	0.836478	0.820001
12	0.537141	0.746705	0.825303	0.809554
13	0.483831	0.712548	0.837414	0.820885
14	0.430120	0.621788	0.874734	0.858984
15	0.385220	0.668646	0.871563	0.855918
16	0.363455	0.704325	0.853059	0.838089
17	0.347504	0.661008	0.875825	0.857685
18	0.279107	0.856187	0.822704	0.807630
19	0.240815	0.661839	0.880919	0.865845
20	0.208194	0.631141	0.888768	0.872343
21	0.180183	0.610746	0.912625	0.897604
22	0.156965	0.562344	0.920630	0.905037
23	0.129332	0.597815	0.919954	0.905608
24	0.112699	0.568391	0.929830	0.915224
25	0.097474	0.572644	0.930194	0.913093
26	0.081098	0.623580	0.923697	0.909611
27	0.067831	0.609429	0.935859	0.921618
28	0.059689	0.560806	0.941525	0.927283
29	0.049726	0.610842	0.934664	0.920786
30	0.041135	0.603998	0.938718	0.923697
31	0.029775	0.617214	0.940330	0.926711
32	0.023638	0.617630	0.945319	0.931545
33	0.020226	0.636995	0.942669	0.929466
34	0.015905	0.626582	0.947866	0.933209
35	0.010595	0.658026	0.945579	0.931805
36	0.009660	0.641626	0.946983	0.933572
37	0.007608	0.650092	0.947866	0.933728
38	0.004905	0.648605	0.948126	0.934404
39	0.003914	0.652137	0.949114	0.935392
40	0.003465	0.655823	0.948490	0.934924

Tabela 35 – Resultados do treinamento por época para o modelo JabuticaBERT (Linear) Sem UPOS

Epoch	Training Loss	Validation Loss	UAS	LAS
1	8.393768	6.428660	0.086680	0.025377
2	5.378833	4.437797	0.180836	0.147984
3	3.875141	3.311655	0.257383	0.231756
4	2.987793	2.653744	0.344021	0.316443
5	2.397018	2.231774	0.433027	0.405283
6	1.972485	1.994680	0.455912	0.428999
7	1.637555	1.737154	0.503427	0.479628
8	1.357048	1.536455	0.594717	0.567180
9	1.131365	1.410976	0.637787	0.612119
10	0.953565	1.314130	0.667359	0.641774
11	0.826351	1.260309	0.692113	0.666736
12	0.722533	1.260356	0.692362	0.667234
13	0.639915	1.404780	0.637413	0.616688
14	0.566562	1.221430	0.699464	0.676870
15	0.495308	1.158324	0.744320	0.718860
16	0.467804	1.191760	0.734643	0.710886
17	0.436218	1.280523	0.713170	0.690202
18	0.385081	1.088923	0.782074	0.756739
19	0.327297	1.119284	0.764506	0.739793
20	0.293407	0.997818	0.821697	0.799352
21	0.235995	1.028641	0.818790	0.795863
22	0.187532	1.001492	0.840553	0.817585
23	0.157733	1.050212	0.832039	0.808448
24	0.129583	1.069665	0.840262	0.817627
25	0.095414	1.049261	0.839598	0.818333
26	0.078788	1.058600	0.849857	0.828010
27	0.057904	1.042759	0.855630	0.833493
28	0.041248	1.049101	0.858413	0.837646
29	0.034084	1.068309	0.861195	0.840429
30	0.022675	1.088302	0.855381	0.834074
31	0.016738	1.093525	0.859659	0.838435
32	0.010978	1.152028	0.855879	0.836317
33	0.006988	1.115545	0.862151	0.842630
34	0.004810	1.139670	0.864435	0.843170
35	0.003518	1.137361	0.866429	0.845911
36	0.002794	1.146439	0.864975	0.843917
37	0.002158	1.161517	0.866429	0.845703
38	0.001923	1.162787	0.867924	0.846908
39	0.001706	1.164500	0.867425	0.846866
40	0.001558	1.168091	0.867508	0.846908

Tabela 36 – Resultados do treinamento por época para o modelo mBERT (Biaffine) Sem UPOS

Epoch	Training Loss	Validation Loss	UAS	LAS
1	7.759808	5.437845	0.124908	0.082210
2	5.402070	3.582396	0.219442	0.182838
3	3.775309	2.580805	0.310423	0.283466
4	2.868056	2.008706	0.405234	0.380124
5	2.279641	1.610968	0.510663	0.486337
6	1.879259	1.421743	0.574409	0.552576
7	1.581678	1.249412	0.638155	0.615630
8	1.358529	1.194827	0.662482	0.642310
9	1.173147	1.083292	0.699732	0.679237
10	1.034544	1.022796	0.730613	0.710072
11	0.914724	0.945467	0.757109	0.737906
12	0.814309	0.929939	0.777049	0.755678
13	0.728459	0.974770	0.777142	0.755816
14	0.662365	0.929487	0.804330	0.781112
15	0.595818	0.877709	0.806684	0.786789
16	0.510786	1.020973	0.786143	0.766848
17	0.460190	0.874932	0.828564	0.808761
18	0.423094	0.864438	0.839457	0.820255
19	0.372433	0.889751	0.841904	0.821086
20	0.343712	0.853045	0.851136	0.831472
21	0.308123	0.891527	0.851551	0.831841
22	0.280008	0.859199	0.862537	0.843704
23	0.249363	0.927171	0.851459	0.833318
24	0.230431	0.981467	0.853397	0.833872
25	0.202458	0.992962	0.861660	0.842596
26	0.191651	1.000617	0.847027	0.827594
27	0.177132	0.943651	0.863091	0.844165
28	0.151807	1.057460	0.865122	0.846520
29	0.138156	1.013918	0.872738	0.853859
30	0.125380	1.141869	0.855613	0.837426
31	0.114610	1.041115	0.875369	0.858475
32	0.102079	1.124564	0.868861	0.851689
33	0.093654	1.122144	0.877539	0.860367
34	0.084596	1.149899	0.876477	0.859537
35	0.073892	1.209540	0.873338	0.855705
36	0.068892	1.195242	0.879616	0.862906
37	0.064009	1.216493	0.878508	0.861198
38	0.057695	1.222018	0.879570	0.862583
39	0.051822	1.230577	0.881555	0.864660
40	0.047604	1.241329	0.881693	0.864845

Tabela 37 – Resultados de treinamento do BERTimbau Base (Biaffine) Sem UPOS

Epoch	Training Loss	Validation Loss	UAS	LAS
1	8.261358	5.832484	0.085607	0.029679
2	5.918988	4.157014	0.212277	0.161079
3	4.171156	2.796542	0.343209	0.306097
4	3.020113	2.151899	0.408493	0.381933
5	2.360491	1.625537	0.525079	0.502157
6	1.817737	1.236911	0.647487	0.626488
7	1.426862	1.023299	0.713707	0.693695
8	1.153245	0.908387	0.751702	0.733042
9	0.964673	0.802758	0.792141	0.773845
10	0.828843	0.755234	0.810905	0.795000
11	0.725128	0.717474	0.829097	0.811944
12	0.637698	0.722164	0.835594	0.819689
13	0.562536	0.685269	0.857581	0.841104
14	0.494730	0.683316	0.856282	0.841104
15	0.421545	0.663316	0.860596	0.845158
16	0.369445	0.722794	0.854462	0.840168
17	0.334421	0.658971	0.870991	0.855710
18	0.299391	0.655941	0.878164	0.862779
19	0.256314	0.672381	0.879568	0.863870
20	0.227256	0.697399	0.880035	0.865586
21	0.194420	0.708089	0.892510	0.879048
22	0.174149	0.764380	0.885597	0.871563
23	0.150600	0.755259	0.886948	0.872447
24	0.131660	0.785224	0.893030	0.878684
25	0.121811	0.773659	0.892874	0.879516
26	0.105457	0.824429	0.893809	0.879516
27	0.093952	0.843190	0.894433	0.879983
28	0.081563	0.864881	0.894901	0.881075
29	0.071073	0.888191	0.895941	0.882426
30	0.062444	0.908094	0.898643	0.885025
31	0.056762	0.928456	0.899735	0.886117
32	0.048228	0.978915	0.898124	0.885233
33	0.043846	1.008818	0.895941	0.883050
34	0.039522	1.045037	0.895265	0.882218
35	0.035929	1.039541	0.899891	0.886221
36	0.030681	1.070201	0.897708	0.884661
37	0.029352	1.089875	0.897604	0.884609
38	0.025439	1.100933	0.900151	0.887052
39	0.021568	1.147125	0.896928	0.883778
40	0.020110	1.114936	0.900203	0.886948

Tabela 38 – Resultados de treinamento do BERTimbau Large (Biaffine) Sem UPOS

Epoch	Training Loss	Validation Loss	UAS	LAS
1	7.970501	5.626317	0.102500	0.055564
2	5.501441	3.528320	0.246115	0.211342
3	3.500025	2.342637	0.381309	0.351941
4	2.537255	1.698753	0.497427	0.472738
5	1.903137	1.287655	0.627371	0.604553
6	1.460230	0.983849	0.715994	0.695722
7	1.125629	0.850731	0.765788	0.746453
8	0.911996	0.751084	0.792297	0.775716
9	0.767728	0.702197	0.820729	0.805655
10	0.657252	0.670040	0.838921	0.823536
11	0.571510	0.599906	0.863714	0.847913
12	0.496154	0.594108	0.867665	0.852747
13	0.434177	0.701172	0.840584	0.826446
14	0.390089	0.660707	0.873694	0.857893
15	0.352079	0.618206	0.884038	0.869172
16	0.309127	0.656994	0.880035	0.865689
17	0.289667	0.669776	0.887260	0.872499
18	0.255981	0.661114	0.883414	0.871095
19	0.227414	0.660025	0.899891	0.886585
20	0.193257	0.661338	0.903321	0.889651
21	0.158622	0.704594	0.901398	0.888404
22	0.142006	0.735202	0.897344	0.882998
23	0.126697	0.765111	0.895629	0.882478
24	0.113648	0.721873	0.900878	0.887104
25	0.093369	0.794418	0.904049	0.890951
26	0.074199	0.830130	0.904361	0.890951
27	0.069375	0.816384	0.910858	0.898695
28	0.057987	0.840667	0.906128	0.893238
29	0.045908	0.898323	0.910806	0.898332
30	0.042089	0.897851	0.909091	0.895889
31	0.035482	0.949999	0.908259	0.895733
32	0.030325	0.949684	0.912314	0.900307
33	0.026720	1.027281	0.910182	0.897604
34	0.022483	1.019642	0.911222	0.899215
35	0.018302	1.081149	0.913769	0.901242
36	0.014730	1.113931	0.909923	0.896928
37	0.013268	1.139365	0.911274	0.898903
38	0.009692	1.113553	0.913769	0.901398
39	0.009604	1.157441	0.913769	0.901606
40	0.008185	1.166806	0.913093	0.90113

Tabela 39 – Resultados de treinamento do JabuticaBERT (Biaffine) Sem UPOS

Epoch	Training Loss	Validation Loss	UAS	LAS
1	7.470867	5.216873	0.108734	0.081156
2	4.841999	3.536611	0.227354	0.180753
3	3.438372	2.582774	0.321427	0.289156
4	2.616234	2.082240	0.402791	0.374424
5	2.087964	1.758234	0.473232	0.446484
6	1.683992	1.503693	0.558957	0.532417
7	1.360143	1.313698	0.639905	0.613365
8	1.077190	1.209507	0.682477	0.655730
9	0.861823	1.066322	0.745317	0.717407
10	0.693986	1.048824	0.756365	0.731570
11	0.575143	1.018641	0.790838	0.764838
12	0.496442	0.912803	0.811480	0.787100
13	0.410853	1.006194	0.805956	0.782157
14	0.331785	0.905823	0.833617	0.811771
15	0.291736	0.978692	0.830502	0.807949
16	0.254058	0.893040	0.860988	0.839723
17	0.198626	0.980588	0.858620	0.836774
18	0.149987	1.069976	0.856128	0.833119
19	0.141900	1.491528	0.798148	0.777256
20	0.114110	1.089457	0.866179	0.846991
21	0.091424	1.199443	0.870416	0.850770
22	0.082324	1.131833	0.871952	0.850355
23	0.067043	1.262494	0.875940	0.856710
24	0.055294	1.205063	0.884122	0.864560
25	0.043282	1.368658	0.874569	0.854301
26	0.040146	1.362649	0.884371	0.864601
27	0.035361	1.371629	0.881547	0.862607
28	0.028529	1.406064	0.883540	0.863230
29	0.027225	1.443492	0.884537	0.865473
30	0.019172	1.605498	0.884163	0.865307
31	0.017265	1.534620	0.892387	0.872368
32	0.012036	1.575720	0.888566	0.869460
33	0.010778	1.663178	0.888067	0.868630
34	0.009578	1.786623	0.887984	0.868464
35	0.006085	1.743962	0.893010	0.874735
36	0.004764	1.773588	0.891473	0.873406
37	0.003392	1.789937	0.893841	0.874901
38	0.002817	1.814949	0.895377	0.876563
39	0.001928	1.841999	0.896083	0.877227
40	0.001563	1.861034	0.894713	0.876230

Tabela 40 – Selected training epochs of PortParser.

Epoch	Train Loss	Val Loss	UPOS	UAS	LAS
1	5.917	1.771	75.71	70.27	95.94
10	0.121	0.616	95.14	93.60	98.97
20	0.059	0.710	95.48	93.96	99.06
30	0.044	0.693	95.73	94.24	99.08
40	0.038	0.776	95.72	94.31	98.91
50	0.012	0.854	96.05	94.72	98.99
60	0.008	0.924	96.07	94.79	99.03

Tabela 41 – Resultados validação - mBert (Linear)

Training Loss	Validation Loss	Epoch	UAS	LAS	UPOS Accuracy
0.008834	1.065175	40	0.916636	0.898172	0.983475

Tabela 42 – Resultados validação - Bertimbau-Base (Linear)

Training Loss	Validation Loss	Epoch	UAS	LAS	UPOS Accuracy
0.003333	0.860932	40	0.928583	0.915328	0.989500

Tabela 43 – Resultados validação - Bertimbau-Large (Linear)

Training Loss	Validation Loss	Epoch	UAS	LAS	UPOS Accuracy
0.002136	0.777716	40	0.944280	0.930974	0.990124

Tabela 44 – Resultados validação - JabuticaBERT (Linear)

Training Loss	Validation Loss	Epoch	UAS	LAS	UPOS Accuracy
0.000532	1.245967	40	0.877892	0.860822	0.985754 height

Tabela 45 – Resultados validação - mBert (Biaffine)

Training Loss	Validation Loss	Epoch	UAS	LAS	UPOS Accuracy
0.025581	1.784062	40	0.863506	0.845689	0.982736

Tabela 46 – Resultados validação - BERTimbau Base (Biaffine)

Training Loss	Validation Loss	Epoch	UAS	LAS	UPOS Accuracy
0.011540	1.660877	40	0.883570	0.872135	0.990696

Tabela 47 – Resultados validação - BERTimbau Large (Biaffine)

Training Loss	Validation Loss	Epoch	UAS	LAS	UPOS Accuracy
0.004790	1.505415	40	0.905193	0.893965	0.989968

Tabela 48 – Resultados validação - JabuticaBERT (Biaffine)

Training Loss	Validation Loss	Epoch	UAS	LAS	UPOS Accuracy
0.001049	2.287051	40	0.880093	0.862068	0.985380

Tabela 49 – Resultados validação - mBert (Linear) Sem UPOS

Training Loss	Validation Loss	Epoch	UAS	LAS
—	—	—	0.913543	0.894572

Tabela 50 – Resultados validação - Bertimbau-Base (Linear) Sem UPOS

Training Loss	Validation Loss	Epoch	UAS	LAS
—	—	—	0.929778	0.916420

Tabela 51 – Resultados validação - Bertimbau-Large (Linear) Sem UPOS

Training Loss	Validation Loss	Epoch	UAS	LAS
–	–	–	0.949114	0.935392

Tabela 52 – Resultados validação - JabuticaBERT (Linear) Sem UPOS

Training Loss	Validation Loss	Epoch	UAS	LAS
–	–	–	0.867924	0.846908

Tabela 53 – Resultados validação - mBert (Biaffine) Sem UPOS

Training Loss	Validation Loss	Epoch	UAS	LAS
0.047604	1.241329	40	0.881693	0.864845

Tabela 54 – Resultados validação - BERTimbau Base (Biaffine) Sem UPOS

Training Loss	Validation Loss	Epoch	UAS	LAS
0.020110	1.100933	40	0.900151	0.887052

Tabela 55 – Resultados validação - BERTimbau Large (Biaffine) Sem UPOS

Training Loss	Validation Loss	Epoch	UAS	LAS
0.008185	1.157441	40	0.913769	0.901606

Tabela 56 – Resultados validação - JabuticaBERT (Biaffine) Sem UPOS

Training Loss	Validation Loss	Epoch	UAS	LAS
0.001563	1.841999	40	0.896083	0.877227

Tabela 57 – Resultados validação - PortParser

Training Loss	Validation Loss	Epoch	UAS	LAS	UPOS Accuracy
–	–	–	0.960840	0.946129	0.990947

Tabela 58 – Acurácia por rótulo — mBERT (mB). **Lin** = Linear com UPOS, **Biaff** = Biaffine com UPOS, **sLin** = sem UPOS Linear, **sBiaff** = sem UPOS Biaffine. **Negrito** = melhor por rótulo.

DEPREL	N	Lin	Biaff	sLin	sBiaff
det	4,853	0.9948	0.9944	0.9936	0.9951
nsubj	1,895	0.9546	0.9530	0.9546	0.9536
root	1,683	0.9750	0.9774	0.9733	0.9750
obj	1,476	0.9587	0.9587	0.9600	0.9621
xcomp	475	0.9242	0.9284	0.9305	0.9326
punct	4,443	0.9986	0.9984	0.9986	0.9989
mark	828	0.9674	0.9686	0.9686	0.9698
advcl	437	0.8741	0.8764	0.8741	0.8924
case	4,532	0.9958	0.9958	0.9949	0.9954
obl	1,746	0.9170	0.9026	0.9049	0.9066
amod	1,318	0.9545	0.9598	0.9568	0.9522
conj	960	0.9344	0.9417	0.9448	0.9406
cc	833	0.9940	0.9916	0.9940	0.9928
nmod	2,607	0.9425	0.9448	0.9444	0.9344
advmod	1,222	0.9664	0.9681	0.9656	0.9632
flat:name	686	0.9738	0.9781	0.9781	0.9825
ccomp	218	0.9358	0.9174	0.9174	0.9174
cop	595	0.9832	0.9849	0.9832	0.9899
acl	307	0.8599	0.8306	0.8599	0.8436
nummod	341	0.9472	0.9589	0.9472	0.9472
acl:relcl	401	0.9526	0.9451	0.9501	0.9501
ccomp:speech	130	0.9154	0.8692	0.9154	0.8923
parataxis	160	0.7063	0.6937	0.6813	0.7250
csubj	83	0.8193	0.7470	0.8193	0.7831
aux:pass	203	0.9951	0.9901	0.9951	1.0000
appos	208	0.9183	0.9375	0.9087	0.9279
fixed	230	0.9696	0.9696	0.9739	0.9783
nsubj:pass	144	0.9861	0.9722	0.9792	0.9792
aux	145	0.9655	0.9517	0.9586	0.9448
nsubj:outer	15	0.8667	0.8000	0.8000	0.8000
obl:agent	110	0.9636	0.9636	0.9636	0.9364
expl:impers	27	0.8148	0.8148	0.7778	0.8519
expl	124	0.8952	0.9113	0.9113	0.9113
discourse	51	0.6078	0.5686	0.5882	0.5490
orphan	14	0.2857	0.1429	0.0714	0.2857
dislocated	17	0.3529	0.5294	0.4118	0.3529
flat	17	1.0000	1.0000	1.0000	1.0000
flat:foreign	15	0.8000	0.8000	0.6000	0.6000
iobj	19	0.7368	0.7895	0.7368	0.7895
vocative	6	0.1667	0.1667	0.0000	0.3333
Global	33,574	0.9665	0.9657	0.9657	0.9655

UPOS	N	Lin	Biaff
DET	4,904	0.9951	0.9951
NOUN	6,309	0.9886	0.9892
VERB	3,368	0.9857	0.9866
PUNCT	4,443	0.9986	0.9986
SCONJ	456	0.9408	0.9518
ADP	5,019	0.9980	0.9974
ADJ	1,691	0.9568	0.9574
CCONJ	823	0.9927	0.9915
ADV	1,272	0.9764	0.9780
PROPN	2,209	0.9959	0.9964
AUX	968	0.9979	0.9969
NUM	622	0.9695	0.9775
PRON	1,286	0.9821	0.9813
SYM	120	1.0000	1.0000
X	85	0.7059	0.7412
INTJ	5	0.8000	0.6000
Global	33,580	0.9888	0.9893

Tabela 59 – Acurácia por rótulo — BERTimbau Base (BtB). **Lin**=Linear com UPOS, **Bi-aff**=Biaffine com UPOS, **sLin**=sem UPOS Linear, **sBiaff**=sem UPOS Biaffine. **Ne-grito**=melhor por rótulo.

DEPREL	N	Lin	Biaff	sLin	sBiaff
det	4,853	0.9963	0.9963	0.9965	0.9967
nsubj	1,895	0.9641	0.9689	0.9641	0.9668
root	1,683	0.9804	0.9863	0.9828	0.9846
obj	1,476	0.9763	0.9763	0.9722	0.9824
xcomp	475	0.9579	0.9495	0.9516	0.9621
punct	4,443	0.9984	0.9986	0.9991	0.9993
mark	828	0.9807	0.9807	0.9819	0.9855
advcl	437	0.9176	0.9268	0.9039	0.9016
case	4,532	0.9971	0.9982	0.9978	0.9976
obl	1,746	0.9318	0.9376	0.9341	0.9296
amod	1,318	0.9697	0.9689	0.9605	0.9742
conj	960	0.9510	0.9500	0.9385	0.9573
cc	833	0.9964	0.9964	0.9976	0.9964
nmod	2,607	0.9490	0.9517	0.9524	0.9532
advmod	1,222	0.9673	0.9755	0.9705	0.9697
flat:name	686	0.9723	0.9679	0.9694	0.9679
ccomp	218	0.9404	0.9404	0.9541	0.9312
cop	595	0.9933	0.9966	0.9966	0.9933
acl	307	0.8567	0.8893	0.8697	0.8730
nummod	341	0.9501	0.9619	0.9501	0.9531
acl:relcl	401	0.9551	0.9601	0.9526	0.9576
ccomp:speech	130	0.9462	0.9231	0.9385	0.9231
parataxis	160	0.7438	0.6875	0.7688	0.7562
csubj	83	0.8554	0.9277	0.9157	0.9398
aux:pass	203	1.0000	0.9901	0.9951	0.9951
appos	208	0.9279	0.9375	0.9279	0.9423
fixed	230	0.9783	0.9739	0.9652	0.9739
nsubj:pass	144	0.9931	0.9514	0.9861	0.9792
aux	145	0.9724	0.9724	0.9724	0.9655
nsubj:outer	15	0.8667	0.7333	0.8667	0.7333
obl:agent	110	0.9636	0.9909	0.9818	0.9727
expl:impers	27	0.8889	0.9259	0.9259	0.8889
expl	124	0.9194	0.9274	0.9113	0.8871
discourse	51	0.6863	0.6471	0.6863	0.6667
orphan	14	0.1429	0.1429	0.1429	0.2143
dislocated	17	0.4706	0.5294	0.3529	0.4706
flat	17	1.0000	1.0000	1.0000	1.0000
flat:foreign	15	0.7333	0.7333	0.7333	0.8667
iobj	19	0.8947	0.8947	0.9474	0.8947
vocative	6	0.3333	0.3333	0.5000	0.1667
Global	33,574	0.9734	0.9750	0.9734	0.9749

UPOS	N	Lin	Biaff
DET	4,904	0.9969	0.9967
NOUN	6,309	0.9921	0.9933
VERB	3,368	0.9911	0.9911
PUNCT	4,443	0.9984	0.9986
SCONJ	456	0.9605	0.9649
ADP	5,019	0.9992	0.9990
ADJ	1,691	0.9746	0.9746
CCONJ	823	0.9951	0.9951
ADV	1,272	0.9796	0.9827
PROPN	2,209	0.9964	0.9959
AUX	968	0.9979	1.0000
NUM	622	0.9759	0.9807
PRON	1,286	0.9860	0.9852
SYM	120	1.0000	1.0000
X	85	0.7647	0.7529
INTJ	5	0.8000	0.8000
Global	33,580	0.9922	0.9927

Tabela 60 – Acurácia por rótulo — BERTimbau Large (BtL). **Lin**=Linear com UPOS, **Biaff**=Biaffine com UPOS, **sLin**=sem UPOS Linear, **sBiaff**=sem UPOS Biaffine. **Ne-grito**=melhor por rótulo.

DEPREL	N	Lin	Biaff	sLin	sBiaff
det	4,853	0.9965	0.9953	0.9963	0.9963
nsubj	1,895	0.9699	0.9736	0.9704	0.9715
root	1,683	0.9857	0.9881	0.9822	0.9899
obj	1,476	0.9776	0.9810	0.9756	0.9770
xcomp	475	0.9579	0.9579	0.9579	0.9579
punct	4,443	0.9984	0.9982	0.9986	0.9989
mark	828	0.9831	0.9855	0.9795	0.9867
advcl	437	0.9359	0.9176	0.9153	0.9108
case	4,532	0.9967	0.9978	0.9965	0.9969
obl	1,746	0.9341	0.9376	0.9364	0.9376
amod	1,318	0.9734	0.9810	0.9689	0.9750
conj	960	0.9604	0.9635	0.9521	0.9656
cc	833	0.9964	0.9928	0.9952	0.9940
nmod	2,607	0.9586	0.9582	0.9501	0.9513
advmod	1,222	0.9746	0.9763	0.9722	0.9730
flat:name	686	0.9796	0.9738	0.9767	0.9825
ccomp	218	0.9541	0.9541	0.9587	0.9633
cop	595	0.9950	0.9983	0.9983	0.9950
acl	307	0.8958	0.8925	0.8893	0.8795
nummod	341	0.9560	0.9589	0.9501	0.9648
acl:relcl	401	0.9676	0.9626	0.9576	0.9576
ccomp:speech	130	0.9385	0.9385	0.9538	0.9462
parataxis	160	0.7312	0.7812	0.7500	0.7562
csbj	83	0.9036	0.9398	0.9036	0.9277
aux:pass	203	0.9951	0.9951	1.0000	0.9901
appos	208	0.9375	0.9327	0.9279	0.9231
fixed	230	0.9739	0.9739	0.9783	0.9783
nsubj:pass	144	0.9931	0.9861	0.9861	0.9861
aux	145	0.9586	0.9793	0.9586	0.9448
nsubj:outer	15	0.9333	0.8000	0.8667	0.8000
obl:agent	110	1.0000	0.9727	0.9818	0.9818
expl:impers	27	0.8889	0.8519	0.8148	0.8519
expl	124	0.9274	0.9274	0.9194	0.9677
discourse	51	0.7059	0.7451	0.6275	0.6667
orphan	14	0.3571	0.4286	0.2143	0.2143
dislocated	17	0.4706	0.5882	0.3529	0.4706
flat	17	1.0000	1.0000	1.0000	1.0000
flat:foreign	15	0.9333	0.8667	0.8000	0.8667
iobj	19	0.8947	0.8947	0.8421	0.8947
vocative	6	0.6667	0.5000	0.6667	0.3333
Global	33,574	0.9771	0.9780	0.9748	0.9766

UPOS	N	Lin	Biaff
DET	4,904	0.9967	0.9957
NOUN	6,309	0.9921	0.9933
VERB	3,368	0.9920	0.9923
PUNCT	4,443	0.9982	0.9984
SCONJ	456	0.9649	0.9759
ADP	5,019	0.9984	0.9988
ADJ	1,691	0.9787	0.9840
CCONJ	823	0.9951	0.9939
ADV	1,272	0.9843	0.9843
PROPN	2,209	0.9964	0.9959
AUX	968	0.9969	0.9990
NUM	622	0.9775	0.9807
PRON	1,286	0.9883	0.9907
SYM	120	0.9917	0.9917
X	85	0.8353	0.7765
INTJ	5	0.8000	0.8000
Global	33,580	0.9928	0.9934

Tabela 61 – Acurácia por rótulo — JabuticaBERT (JaB). **Lin**=Linear com UPOS, **Bi-aff**=Biaffine com UPOS, **sLin**=sem UPOS Linear, **sBiaff**=sem UPOS Biaffine. **Ne-grito**=melhor por rótulo.

DEPREL	N	Lin	Biaff	sLin	sBiaff
det	4,853	0.9936	0.9936	0.9920	0.9928
nsubj	1,895	0.9530	0.9546	0.9520	0.9520
root	1,683	0.9786	0.9840	0.9762	0.9780
obj	1,476	0.9627	0.9634	0.9593	0.9593
xcomp	475	0.9326	0.9305	0.8947	0.9326
punct	4,443	0.9991	0.9991	0.9989	0.9991
mark	828	0.9771	0.9758	0.9638	0.9722
advcl	437	0.8993	0.8810	0.8856	0.8810
case	4,532	0.9951	0.9956	0.9943	0.9962
obl	1,746	0.9158	0.9055	0.9015	0.9021
amod	1,318	0.9514	0.9552	0.9476	0.9476
conj	960	0.9313	0.9260	0.9198	0.9240
cc	833	0.9916	0.9940	0.9928	0.9892
nmod	2,607	0.9379	0.9428	0.9367	0.9382
advmod	1,222	0.9648	0.9648	0.9558	0.9599
flat:name	686	0.9519	0.9548	0.9490	0.9577
ccomp	218	0.9174	0.9174	0.9174	0.8945
cop	595	0.9899	0.9899	0.9832	0.9899
acl	307	0.8306	0.8306	0.8274	0.8176
nummod	341	0.9384	0.9472	0.9443	0.9443
acl:relcl	401	0.9476	0.9576	0.9476	0.9401
ccomp:speech	130	0.8769	0.8615	0.8846	0.8462
parataxis	160	0.6625	0.6813	0.6813	0.7188
csbj	83	0.7831	0.8193	0.7711	0.7711
aux:pass	203	0.9951	0.9852	0.9852	0.9803
appos	208	0.9087	0.9038	0.9231	0.9135
fixed	230	0.9565	0.9565	0.9478	0.9304
nsubj:pass	144	0.9722	0.9583	0.9514	0.9653
aux	145	0.9241	0.9448	0.9241	0.9241
nsubj:outer	15	0.8000	0.6667	0.8000	0.8000
obl:agent	110	0.9455	0.9545	0.9364	0.9455
expl:impers	27	0.8889	0.8519	0.8889	0.8889
expl	124	0.9032	0.9274	0.9032	0.8871
discourse	51	0.6471	0.5882	0.5882	0.6667
orphan	14	0.2143	0.3571	0.0714	0.0000
dislocated	17	0.4118	0.2353	0.3529	0.2353
flat	17	0.9412	0.9412	0.9412	0.9412
flat:foreign	15	0.5333	0.7333	0.3333	0.4667
iobj	19	0.7895	0.7895	0.7368	0.6842
vocative	6	0.1667	0.0000	0.0000	0.1667
Global	33,574	0.9647	0.9651	0.9608	0.9622

UPOS	N	Lin	Biaff
DET	4,904	0.9931	0.9941
NOUN	6,309	0.9887	0.9914
VERB	3,368	0.9881	0.9863
PUNCT	4,443	0.9991	0.9991
SCONJ	456	0.9627	0.9583
ADP	5,019	0.9968	0.9976
ADJ	1,691	0.9503	0.9556
CCONJ	823	0.9891	0.9939
ADV	1,272	0.9780	0.9748
PROPN	2,209	0.9914	0.9914
AUX	968	0.9897	0.9948
NUM	622	0.9662	0.9727
PRON	1,286	0.9798	0.9798
SYM	120	0.9917	1.0000
X	85	0.7647	0.7647
INTJ	5	0.8000	0.8000
Global	33,580	0.9881	0.9892

Tabela 62 – Instabilidade por rótulo de dependência (DEPREL) — mBERT (mB). As 20 categorias com maior número de esquecimentos (Lin) são exibidas. Fg. máximo e Ac.F máxima por linha em **negrito**.

DEPREL	N	Lin		Biaff		sLin		sBiaff	
		Fg.	Ac.F	Fg.	Ac.F	Fg.	Ac.F	Fg.	Ac.F
nmod	2,035	1,517	0.9263	1,543	0.9366	1,263	0.9425	1,403	0.9351
obl	1,348	1,307	0.8895	1,423	0.8709	1,192	0.8642	1,187	0.8858
amod	1,190	544	0.9521	544	0.9420	470	0.9471	438	0.9471
nsubj	1,444	533	0.9550	472	0.9460	440	0.9488	485	0.9536
conj	832	525	0.9315	519	0.9339	373	0.9447	444	0.9423
advcl	406	524	0.8448	538	0.8498	433	0.8276	401	0.8621
acl	270	430	0.8037	462	0.8074	226	0.8148	373	0.8185
root	1,508	407	0.9808	398	0.9761	346	0.9781	379	0.9748
obj	1,143	349	0.9615	390	0.9615	312	0.9589	351	0.9563
parataxis	122	201	0.6885	226	0.7131	164	0.7377	248	0.7705
xcomp	390	198	0.9487	227	0.9385	224	0.9436	189	0.9564
case	2,239	184	0.9924	169	0.9942	167	0.9951	139	0.9924
flat:name	561	178	0.9608	137	0.9643	225	0.9590	215	0.9661
acl:relcl	342	171	0.9708	233	0.9620	176	0.9795	226	0.9766
advmod	602	159	0.9718	112	0.9718	153	0.9734	95	0.9751
appos	165	116	0.9273	137	0.8970	102	0.9273	136	0.9212
mark	404	107	0.9678	137	0.9728	104	0.9653	108	0.9678
det	2,406	106	0.9954	122	0.9958	104	0.9958	108	0.9946
nsubj:pass	127	95	0.8976	132	0.9449	61	0.9213	92	0.9213
ccomp	192	92	0.9635	134	0.9479	67	0.9375	134	0.9375
Global	21,664	8,501	0.9561	8,867	0.9539	7,140	0.9551	7,808	0.9566

Legenda: Lin = Linear com UPOS; Biaff = Biaffine com UPOS; sLin = sem UPOS Linear; sBiaff = sem UPOS Biaffine. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 63 – Instabilidade por categoria morfossintática (UPOS) — mBERT (mB). Apenas modelos MTL. Fg. máximo e Ac.F máxima por linha em **negrito**.

UPOS	N	Lin		Biaff	
		Fg.	Ac.F	Fg.	Ac.F
ADJ	1,611	851	0.9497	766	0.9423
VERB	3,075	769	0.9805	559	0.9805
NOUN	5,041	756	0.9808	669	0.9804
PROPN	1,821	213	0.9907	141	0.9907
ADV	631	134	0.9794	101	0.9826
ADP	2,473	121	0.9943	103	0.9968
DET	2,425	119	0.9959	106	0.9959
PRON	655	115	0.9832	117	0.9740
SCONJ	227	97	0.9471	105	0.9515
X	63	67	0.6667	44	0.6825
AUX	490	52	0.9898	56	0.9898
NUM	392	49	0.9847	32	0.9796
CCONJ	412	30	0.9927	21	0.9854
SYM	98	28	0.9898	16	0.9898
PUNCT	2,242	6	1.0000	1	1.0000
INTJ	8	2	0.1250	0	0.0000
Global	21,664	3,409	0.9835	2,837	0.9827

Legenda: Lin = Linear com UPOS; Biaff = Biaffine com UPOS. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 64 – UAS (HEAD) por época de avaliação — mBERT (mB). Valor máximo por linha em **negrito**.

Época	Lin	Biaff	sLin	sBiaff
1	0.1128	0.1318	0.1021	0.1249
6	0.5580	0.5951	0.5180	0.5744
11	0.7559	0.7380	0.7537	0.7571
16	0.8123	0.8107	0.8143	0.7861
21	0.8732	0.8314	0.8483	0.8516
26	0.8848	0.8481	0.8926	0.8470
31	0.9049	0.8518	0.8993	0.8754
36	0.9162	0.8572	0.9093	0.8796

Legenda: **Lin** = Linear com UPOS; **Biaff** = Biaffine com UPOS; **sLin** = sem UPOS Linear; **sBiaff** = sem UPOS Biaffine. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 65 – Instabilidade por rótulo de dependência (DEPREL) — BERTimbau Base (BtB). As 20 categorias com maior número de esquecimentos (Lin) são exibidas. Fg. máximo e Ac.F máxima por linha em **negrito**.

DEPREL	N	Lin		Biaff		sLin		sBiaff	
		Fg.	Ac.F	Fg.	Ac.F	Fg.	Ac.F	Fg.	Ac.F
nmod	1,729	982	0.9491	1,026	0.9468	659	0.9445	1,093	0.9491
obl	1,132	775	0.9090	897	0.8975	700	0.9011	789	0.9099
amod	927	229	0.9676	221	0.9709	190	0.9644	180	0.9676
obj	862	224	0.9710	202	0.9745	142	0.9710	162	0.9733
advcl	333	223	0.8829	343	0.8529	217	0.8889	243	0.8589
conj	630	216	0.9635	287	0.9651	196	0.9619	217	0.9651
nsubj	1,243	214	0.9678	265	0.9702	188	0.9670	224	0.9662
parataxis	101	178	0.7525	167	0.7525	170	0.7228	139	0.7525
flat:name	532	173	0.9680	150	0.9756	80	0.9680	236	0.9662
acl	186	163	0.8441	202	0.8817	137	0.8387	239	0.8441
root	1,144	141	0.9851	213	0.9790	149	0.9808	194	0.9764
xcomp	293	112	0.9659	137	0.9454	96	0.9556	112	0.9693
advmod	575	95	0.9809	73	0.9826	60	0.9809	74	0.9722
appos	153	92	0.9150	126	0.9346	75	0.9412	123	0.9150
acl:relcl	252	86	0.9762	101	0.9802	58	0.9722	65	0.9683
mark	404	78	0.9777	54	0.9827	65	0.9827	58	0.9777
fixed	117	71	0.9915	46	0.9744	41	0.9915	54	0.9658
case	2,234	69	0.9960	67	0.9978	41	0.9969	64	0.9982
ccomp	139	64	0.9281	58	0.9209	47	0.9209	51	0.9209
ccomp:speech	118	58	0.9661	68	0.9407	32	0.9492	76	0.9576
Global	19,239	4,664	0.9695	5,265	0.9689	3,702	0.9679	4,852	0.9684

Legenda: Lin = Linear com UPOS; Biaff = Biaffine com UPOS; sLin = sem UPOS Linear; sBiaff = sem UPOS Biaffine. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 66 – Instabilidade por categoria morfossintática (UPOS) — BERTimbau Base (BtB). Apenas modelos MTL. Fg. máximo e Ac.F máxima por linha em **negrito**.

UPOS	N	Lin		Biaff	
		Fg.	Ac.F	Fg.	Ac.F
NOUN	3,819	386	0.9890	333	0.9893
ADJ	1,247	299	0.9727	254	0.9759
VERB	2,301	237	0.9865	226	0.9883
PROPN	1,822	174	0.9923	112	0.9951
X	91	116	0.7582	57	0.7802
ADV	602	88	0.9850	39	0.9884
SCONJ	227	78	0.9515	45	0.9736
DET	2,403	65	0.9971	47	0.9971
ADP	2,468	59	0.9984	16	0.9992
CCONJ	408	37	0.9926	19	0.9926
PRON	622	32	0.9904	47	0.9904
NUM	406	27	0.9852	30	0.9852
AUX	474	11	0.9958	12	0.9937
SYM	98	11	0.9796	24	0.9694
INTJ	9	4	0.1111	4	0.0000
PUNCT	2,242	3	0.9996	7	1.0000
Global	19,239	1,627	0.9895	1,272	0.9907

Legenda: Lin = Linear com UPOS; Biaff = Biaffine com UPOS. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 67 – UAS (HEAD) por época de avaliação — BERTimbau Base (BtB). Valor máximo por linha em **negrito**.

Época	Lin	Biaff	sLin	sBiaff
1	0.1061	0.1246	0.0690	0.0856
6	0.6471	0.6714	0.5982	0.6475
11	0.8172	0.8231	0.8139	0.8291
16	0.8800	0.8595	0.8582	0.8545
21	0.8941	0.8661	0.9000	0.8925
26	0.9026	0.8767	0.9120	0.8938
31	0.9190	0.8819	0.9209	0.8997
36	0.9273	0.8853	0.9266	0.8977

Legenda: **Lin** = Linear com UPOS; **Biaff** = Biaffine com UPOS; **sLin** = sem UPOS Linear; **sBiaff** = sem UPOS Biaffine. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 68 – Instabilidade por rótulo de dependência (DEPREL) — BERTimbau Large (BtL). As 20 categorias com maior número de esquecimentos (Lin) são exibidas. Fg. máximo e Ac.F máxima por linha em **negrito**.

DEPREL	N	Lin		Biaff		sLin		sBiaff	
		Fg.	Ac.F	Fg.	Ac.F	Fg.	Ac.F	Fg.	Ac.F
nmod	1,729	923	0.9468	2,057	0.9514	895	0.9479	895	0.9514
obl	1,132	900	0.9178	1,895	0.9037	896	0.9117	866	0.9002
amod	927	270	0.9590	1,122	0.9547	222	0.9579	206	0.9655
nsubj	1,243	264	0.9735	1,344	0.9791	219	0.9702	272	0.9767
advcl	333	229	0.8979	559	0.9099	204	0.8889	293	0.8769
root	1,144	225	0.9816	480	0.9825	142	0.9843	143	0.9851
acl	186	191	0.8226	339	0.8710	168	0.8710	221	0.8763
conj	630	189	0.9667	792	0.9794	154	0.9730	204	0.9746
parataxis	101	189	0.7327	213	0.7624	167	0.7624	162	0.7723
obj	862	189	0.9722	923	0.9710	147	0.9733	179	0.9745
flat:name	532	188	0.9718	678	0.9624	250	0.9680	179	0.9774
appos	153	118	0.9346	266	0.8954	119	0.9281	80	0.9281
xcomp	293	94	0.9693	349	0.9693	114	0.9659	92	0.9590
acl:relcl	252	90	0.9802	313	0.9921	80	0.9841	91	0.9802
advmod	575	89	0.9826	628	0.9843	60	0.9774	89	0.9757
det	2,384	71	0.9979	231	0.9971	74	0.9975	61	0.9971
case	2,234	68	0.9973	499	0.9982	74	0.9960	85	0.9973
mark	404	67	0.9876	488	0.9802	59	0.9728	64	0.9802
fixed	117	59	0.9744	141	0.9658	51	0.9744	37	0.9744
obl:agent	68	57	0.8971	39	0.9412	40	0.9559	41	0.9412
Global	19,239	4,948	0.9709	15,504	0.9717	4,545	0.9707	4,732	0.9713

Legenda: Lin = Linear com UPOS; Biaff = Biaffine com UPOS; sLin = sem UPOS Linear; sBiaff = sem UPOS Biaffine. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 69 – Instabilidade por categoria morfossintática (UPOS) — BERTimbau Large (BtL). Apenas modelos MTL. Fg. máximo e Ac.F máxima por linha em **negrito**.

UPOS	N	Lin		Biaff	
		Fg.	Ac.F	Fg.	Ac.F
NOUN	3,819	385	0.9898	948	0.9890
ADJ	1,247	324	0.9703	1,497	0.9671
VERB	2,301	303	0.9857	1,496	0.9870
PROPN	1,822	163	0.9912	1,880	0.9929
X	91	108	0.8571	55	0.8571
ADV	602	99	0.9867	659	0.9900
DET	2,403	78	0.9975	1,222	0.9971
PRON	622	70	0.9904	651	0.9887
SCONJ	227	68	0.9604	276	0.9559
CCONJ	408	46	0.9877	429	0.9902
ADP	2,468	44	0.9992	774	0.9992
SYM	98	35	0.9898	56	0.9796
NUM	406	29	0.9852	412	0.9852
AUX	474	27	0.9958	500	0.9937
INTJ	9	11	0.2222	10	0.2222
PUNCT	2,242	11	1.0000	1,364	1.0000
Global	19,239	1,801	0.9901	12,229	0.9900

Legenda: Lin = Linear com UPOS; Biaff = Biaffine com UPOS. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 70 – UAS (HEAD) por época de avaliação — BERTimbau Large (BtL). Valor máximo por linha em **negrito**.

Época	Lin	Biaff	sLin	sBiaff
1	0.1093	0.1363	0.0801	0.1025
6	0.6881	0.3065	0.5967	0.7160
11	0.8486	0.7508	0.8365	0.8637
16	0.8672	0.8445	0.8531	0.8800
21	0.9223	0.8786	0.9126	0.9014
26	0.9263	0.8984	0.9237	0.9044
31	0.9312	0.9000	0.9403	0.9083
36	0.9436	0.8948	0.9470	0.9099

Legenda: **Lin** = Linear com UPOS; **Biaff** = Biaffine com UPOS; **sLin** = sem UPOS Linear; **sBiaff** = sem UPOS Biaffine. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 71 – Instabilidade por rótulo de dependência (DEPREL) — JabuticabERT (JaB). As 20 categorias com maior número de esquecimentos (Lin) são exibidas. Fg. máximo e Ac.F máxima por linha em **negrito**.

DEPREL	N	Lin		Biaff		sLin		sBiaff	
		Fg.	Ac.F	Fg.	Ac.F	Fg.	Ac.F	Fg.	Ac.F
obl	1,519	1,513	0.8940	1,797	0.8835	1,563	0.8670	2,748	0.8729
nmod	2,226	1,420	0.9447	1,862	0.9447	1,542	0.9353	2,561	0.9420
amod	1,421	621	0.9444	638	0.9388	692	0.9402	1,969	0.9395
nsubj	1,512	588	0.9590	712	0.9544	562	0.9517	1,821	0.9537
advcl	448	500	0.8772	736	0.8504	494	0.8259	859	0.8304
conj	887	485	0.9188	620	0.9278	592	0.9256	1,225	0.9448
obj	1,248	469	0.9696	446	0.9696	467	0.9615	1,498	0.9647
root	1,639	399	0.9805	532	0.9780	480	0.9786	920	0.9793
acl	298	382	0.8523	520	0.8490	420	0.7886	647	0.7987
flat:name	626	307	0.9505	265	0.9329	343	0.9313	926	0.9297
xcomp	455	254	0.9385	234	0.9429	278	0.9011	620	0.9297
case	3,241	241	0.9948	188	0.9941	187	0.9929	622	0.9948
parataxis	128	235	0.7422	267	0.7266	260	0.6250	286	0.7266
acl:relcl	375	167	0.9867	249	0.9813	183	0.9600	606	0.9600
appos	180	163	0.9222	210	0.8833	194	0.8778	314	0.8667
det	2,406	159	0.9950	144	0.9963	177	0.9958	454	0.9946
advmod	648	141	0.9769	188	0.9691	246	0.9660	789	0.9645
nsubj:pass	150	128	0.9267	145	0.9200	125	0.8867	151	0.9133
ccomp	208	128	0.8654	173	0.9038	132	0.8606	312	0.8942
mark	472	127	0.9746	157	0.9725	172	0.9534	567	0.9682
Global	24,077	9,256	0.9585	11,115	0.9557	9,999	0.9487	22,076	0.9533

Legenda: Lin = Linear com UPOS; Biaff = Biaffine com UPOS; sLin = sem UPOS Linear; sBiaff = sem UPOS Biaffine. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 72 – Instabilidade por categoria morfossintática (UPOS) — JabuticaBERT (JaB). Apenas modelos MTL. Fg. máximo e Ac.F máxima por linha em **negrito**.

UPOS	N	Lin		Biaff	
		Fg.	Ac.F	Fg.	Ac.F
NOUN	5,602	849	0.9884	588	0.9879
ADJ	1,847	770	0.9524	803	0.9545
VERB	3,390	674	0.9835	560	0.9829
PROPN	1,969	246	0.9883	246	0.9873
DET	2,425	173	0.9951	122	0.9959
ADP	3,558	164	0.9978	96	0.9975
PRON	634	159	0.9858	166	0.9858
ADV	676	133	0.9793	132	0.9734
SCONJ	227	118	0.9515	101	0.9339
AUX	504	85	0.9861	52	0.9921
X	85	59	0.7294	84	0.6941
NUM	405	53	0.9753	74	0.9728
CCONJ	411	41	0.9878	32	0.9927
SYM	96	30	0.9583	28	0.9583
PUNCT	2,240	14	0.9996	9	0.9991
INTJ	8	2	0.1250	7	0.2500
Global	24,077	3,570	0.9858	3,100	0.9854

Legenda: Lin = Linear com UPOS; Biaff = Biaffine com UPOS. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

Tabela 73 – UAS (HEAD) por época de avaliação — JabuticaBERT (JaB). Valor máximo por linha em **negrito**.

Época	Lin	Biaff	sLin	sBiaff
1	0.1111	0.1466	0.0867	0.4927
6	0.4809	0.5406	0.4559	0.4732
11	0.6618	0.7870	0.6921	0.7564
16	0.7681	0.8470	0.7346	0.8305
21	0.8282	0.8591	0.8188	0.8662
26	0.8581	0.8749	0.8499	0.8746
31	0.8581	0.8755	0.8597	0.8842
36	0.8771	0.8781	0.8650	0.8930

Legenda: **Lin** = Linear com UPOS; **Biaff** = Biaffine com UPOS; **sLin** = sem UPOS Linear; **sBiaff** = sem UPOS Biaffine. Fg. = *forgetting events* (acerto → erro); Ac.F = acurácia final no conjunto de teste. Fg. máximo e Ac.F máxima por linha em **negrito**.

APÊNDICE A – Um Apêndice

Anexos

ANEXO A – Um Anexo

ANEXO B – Outro Anexo